Relation Algebra

§1. Notation

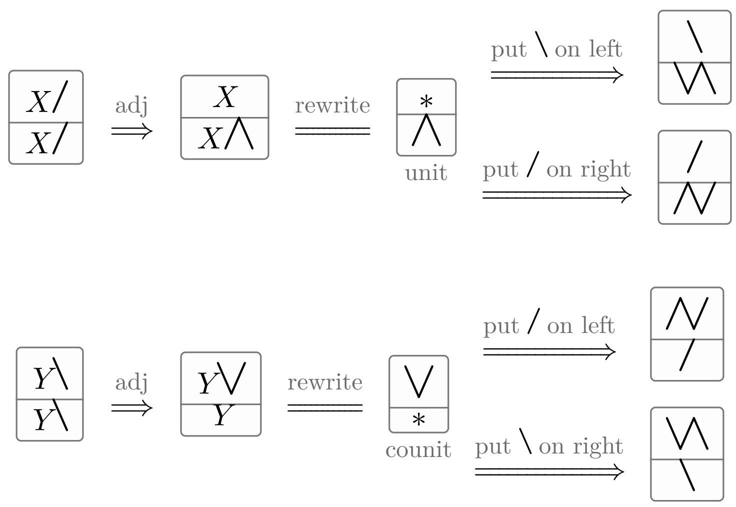
Traditionaly Adjunction F -| G is defined as $F(x) \leq y$ iff $x \leq G(y)$, and you derive laws like this:

$$F(x) \leq F(x) \implies x \leq G(F(x)) \implies F(x) \leq F(G(F(x)))$$
$$G(y) \leq G(y) \implies F(G(y)) \leq y \implies G(F(G(y))) \leq G(y)$$

By define $X : * \mapsto x$, $Y : * \mapsto y$ and use diagram order, application can be replaced by composition: $XF \leq Y$ iff $X \leq YG$ (1a)

- write F as $/$, G as $\backslash$
- render $a \leq b$ as $\boxed{\frac{a}{b}}$
- **rewrite** $X \leq XF$ as $1 \leq F$

$$\boxed{\frac{X/}{Y}} = \boxed{\frac{X}{Y\backslash}} \tag{adj}$$



now we have $/\!\backslash\!/ = /$ and $\backslash\!/\!\backslash = \backslash$, and you just discovered string diagrams.

§1.1. Adjunctions

(1.1a)

F⊣G	F's type	monad FG	𝟙⊑FG	GF⊑𝟙	F preserves ∪	G preserves ∩	FGF=F	GFG=G
◁⊣▷	A→ A⊗A	◁▷=𝟙	𝟙⊑◁▷	▷◁⊑𝟙	(R∪S)◁= R◁∪S◁	(R∩S)▷= R▷∩S▷	◁▷◁=◁	▷◁▷=▷
⊸⊣⟜	A→𝕀	⊸⟜=⊤	𝟙⊑⊸⟜	⟜⊸⊑𝟙	(R∪S)⊸= R⊸∪S⊸	(R∩S)⟜= R⟜∩S⟜	⊸⟜⊸=⊸	⟜⊸⟜=⟜
°⊣°	(A→B)→ (B→A)	°°=𝟙	R=R°°	R=R°°	(R∪S)°= R°∪S°	(R∩S)°= R°∩S°	R°°°=R°	R°°°=R°
⟜◁⊣▷⊸	(X⊗A→Y)→ (X→Y⊗A)	𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	=	=	=	=
▷⊸⊣⟜◁	(X→Y⊗A)→ (X⊗A→Y)	𝟙	(𝟙⊗⟜◁) (▷⊸⊗𝟙)=𝟙	(⟜◁⊗𝟙) (𝟙⊗▷⊸)=𝟙	=	=	=	=
Δ⊣∩	(A→B)→ (A→B)^2	R↦R∩R=R	R⊑R∩R	R∩S⊑R	Δ(R∪S)= ΔR∪ΔS	(R∩T)∩(S∩U)= (R∩S)∩(T∩U)	R∩R=R	R∩R=R
∪⊣Δ	(A→B)^2→ (A→B)	(R,S)↦ (R∪S,R∪S)	R⊑R∪S	R∪R⊑R	(R∪T)∪(S∪U)= (R∪S)∪(T∪U)	Δ(R∩S)= ΔR∩ΔS	R∪R=R	R∪R=R
⊥⊣!	{*}→ (A→B)	—	—	⊥⊑R	—	—	—	—
·S⊣/S	(A→B)→ (A→C)	S/S	R⊑(RS)/S	(T/S)S⊑T	(R∪T)S= RS∪TS	(R∩T)/S= R/S∩T/S	((RS)/S)S =RS	((T/S)S)/S =T/S
S·⊣S\	(B→C)→ (A→C)	S\S	R⊑S\(SR)	S(S\T)⊑T	S(R∪T)= SR∪ST	S\(R∩T)= S\R∩S\T	S(S\(SR)) =SR	S\(S(S\T)) =S\T
R∩⊣R⇒	(A→B)→ (A→B)	X↦R⇒(X∩R)	X⊑R⇒(X∩R)	R∩(R⇒Y)⊑Y	R∩(X∪Y)= (R∩X)∪(R∩Y)	R⇒(X∩Y)= (R⇒X)∩(R⇒Y)	R∩(R⇒(X∩R)) =X∩R	R⇒(R∩(R⇒Y)) =R⇒Y
𝓓⊣·⊤	(A→B)→ Cor A	R↦(𝓓R)⊤	R⊑(𝓓R)⊤	𝓓(A⊤)⊑A	𝓓(R∪S)= 𝓓R∪𝓓S	(A∩B)⊤= A⊤∩B⊤	𝓓((𝓓R)⊤) =𝓓R	(𝓓(A⊤))⊤ =A⊤
𝓡⊣⊤·	(A→B)→ Cor B	R↦⊤(𝓡R)	R⊑⊤(𝓡R)	𝓡(⊤A)⊑A	𝓡(R∪S)= 𝓡R∪𝓡S	⊤(A∩B)= ⊤A∩⊤B	𝓡(⊤(𝓡R)) =𝓡R	⊤(𝓡(⊤A)) =⊤A
·f⊣·f°	(A→B)→ (A→C)	ff°	𝟙⊑ff°	f°f⊑𝟙	(R∪S)f= Rf∪Sf	(R∩S)f°= Rf°∩Sf°	ff°f=f	f°ff°=f°
f°·⊣f·	(A→C)→ (B→C)	ff°	𝟙⊑ff°	f°f⊑𝟙	f°(X∪Y)= f°X∪f°Y	f(X∩Y)= fX∩fY	ff°f=f	f°ff°=f°
i⊣E	Map↪Rel	E	$\frac{𝟙}{\ni}$:A→EA	∋:EB→B	—	—	$\frac{\ni}{\ni}$=𝟙	$\frac{R}{\ni}$∋=R
·∋⊣ $\frac{\ }{\ni}$	Map(A,EB)→ (A→B)	𝟙	$\frac{f\ni}{\ni}$=f	$\frac{R}{\ni}$∋=R	—	—	$\frac{f\ni}{\ni}$∋ =f∋	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$
$\frac{\ }{\ni}$ ⊣·∋	(A→B)→ Map(A,EB)	𝟙	$\frac{R}{\ni}$∋=R	$\frac{f\ni}{\ni}$=f	—	—	$\frac{\frac{R}{\ni}\ni}{\ni}$ =$\frac{R}{\ni}$	$\frac{f\ni}{\ni}$∋ =f∋

§1.2. Composing adjunctions

(1.2a)

$$\underset{\text{f a map}}{\boxed{\frac{X}{f(R/T)}}} \;\underset{f°\cdot\dashv f\cdot}{\Longleftrightarrow}\; \boxed{\frac{f°X}{R/T}} \;\underset{\cdot T\dashv /T}{\Longleftrightarrow}\; \boxed{\frac{(f°X)T}{R}}$$

$$\underset{\text{associativity}}{\Longleftrightarrow}\; \boxed{\frac{f°(XT)}{R}} \;\underset{f°\cdot\dashv f\cdot}{\Longleftrightarrow}\; \boxed{\frac{XT}{fR}} \;\underset{\cdot T\dashv /T}{\Longleftrightarrow}\; \boxed{\frac{X}{(fR)/T}}$$

$$\underset{\text{indirect equality}}{\Longrightarrow}\; \boxed{\frac{f(R/T)}{(fR)/T}}$$

	$\cdot T$	$T\cdot$	$\cdot g$	$g^\circ\cdot$	$T\cap$	$^\circ$	$\text{⟜}\lhd$	Δ
$\cdot S$	$R/(ST)=(R/T)/S$	$T\backslash(R/S)=(T\backslash R)/S$	$R/(Sg)=(Rg^\circ)/S$	$g(R/S)=(gR)/S$	—	$(Y/S)^\circ=S^\circ\backslash(Y^\circ)$	$(RS)^\wedge=R^\wedge(S\otimes\mathbb{1})$	$(T_1\cap T_2)/S=T_1/S\cap T_2/S$
$S\cdot$	$S\backslash(R/T)=(S\backslash R)/T$	$(TS)\backslash R=S\backslash(T\backslash R)$	$S\backslash(Rg^\circ)=(S\backslash R)g^\circ$	$(g^\circ S)\backslash R=S\backslash(gR)$	—	$(S\backslash Y)^\circ=Y^\circ/S^\circ$	$((S\otimes\mathbb{1})R)^\wedge=SR^\wedge$	$S\backslash(T_1\cap T_2)=S\backslash T_1\cap S\backslash T_2$
$\cdot f$	$R/(fT)=(R/T)f^\circ$	$T\backslash(Rf^\circ)=(T\backslash R)f^\circ$	$(fg)^\circ=g^\circ f^\circ$	—	—	$(fY)^\circ=Y^\circ f^\circ$	—	$(T_1\cap T_2)f^\circ=T_1f^\circ\cap T_2f^\circ$
$f^\circ\cdot$	$f(R/T)=(fR)/T$	$(Tf^\circ)\backslash R=f(T\backslash R)$	—	$(fg)^\circ=g^\circ f^\circ$	—	$(fY)^\circ=Y^\circ f^\circ$	—	$f(T_1\cap T_2)=fT_1\cap fT_2$
$R\cap$	—	—	—	—	$(R\cap T)\Rightarrow Y=R\Rightarrow(T\Rightarrow Y)$	$(R\Rightarrow Y)^\circ=R^\circ\Rightarrow(Y^\circ)$	—	$R\Rightarrow(T_1\cap T_2)=(R\Rightarrow T_1)\cap(R\Rightarrow T_2)$
$^\circ$	$(Y/T)^\circ=T^\circ\backslash(Y^\circ)$	$(T\backslash Y)^\circ=Y^\circ/T^\circ$	$(gY)^\circ=Y^\circ g^\circ$	$(gY)^\circ=Y^\circ g^\circ$	$(T\Rightarrow Y)^\circ=T^\circ\Rightarrow(Y^\circ)$	$Y^{\circ\circ}=Y$	$(R^\wedge)^\circ=(R^\circ\otimes\mathbb{1})(\mathbb{1}\otimes\rhd\text{⊸})$ both bends: the $^\circ$ section's display	$(T_1\cap T_2)^\circ=T_1^\circ\cap T_2^\circ$
$\text{⟜}\lhd$	$(RT)^\wedge=R^\wedge(T\otimes\mathbb{1})$	$((T\otimes\mathbb{1})R)^\wedge=TR^\wedge$	—	—	—	$(R^\wedge)^\circ=(R^\circ\otimes\mathbb{1})(\mathbb{1}\otimes\rhd\text{⊸})$ both bends: the $^\circ$ section's display	—	—
$\cup$	$(X_1\cup X_2)T=X_1T\cup X_2T$	$T(X_1\cup X_2)=TX_1\cup TX_2$	$(X_1\cup X_2)g=X_1g\cup X_2g$	$g^\circ(X_1\cup X_2)=g^\circ X_1\cup g^\circ X_2$	$T\cap(X_1\cup X_2)=(T\cap X_1)\cup(T\cap X_2)$	$(X_1\cup X_2)^\circ=X_1^\circ\cup X_2^\circ$	—	—
$\bot$	$\bot T=\bot$	$T\bot=\bot$	$\bot g=\bot$	$g^\circ\bot=\bot$	$T\cap\bot=\bot$	$\bot^\circ=\bot$	—	—

(1.2b)

§1.3. Adjoint triples

Beyond $\cup\dashv\Delta\dashv\cap$, which §1.1 already carries as two rows, the table holds one adjoint triple with content: $\exists\_f\dashv f*\dashv\forall\_f$ along a map f, once on each side of the composite. $f : A\to B$ throughout this subsection.

$$\cdot f\dashv\cdot f^\circ\dashv/f^\circ \qquad (1.3a)$$
$$f^\circ\cdot\dashv f\cdot\dashv f\backslash$$

The first two links of each chain are the map shunting rules, and the third is §1.1′s division row read at a chosen divisor: $\cdot S\dashv/S$ at $S{:=}f^\circ$, and $S\cdot\dashv S\backslash$ at $S{:=}f$.

The third link is not the first over again. For a map f, $/f$ collapses to $\cdot f^\circ$; being a map is exactly what that collapse spends $R/(fS)=(R/S)f^\circ$. — f° is not a map, so $/f^\circ$ does not collapse in turn, and the three operators stay distinct. On $\mathrm{Rel}(\mathbb{I},A)=EA$ they are the image triple:

operator	acts by	name
$\cdot f$	$S\mapsto f[S]$	direct image, $\exists\_f$
$\cdot f^\circ$	$T\mapsto f^{-1}[T]$	inverse image, $f*$
$/f^\circ$	$S\mapsto\{b : f^{-1}(b)\subseteq S\}$	$\forall\_f$

(1.3b)

§1.1′s $\mathcal{D}\dashv\cdot\top$ is this same chain along the projection $A\otimes B\to A$, read through $\mathrm{Rel}(A,B)=E(A\otimes B)$: $\mathcal{D}R=\{(a,a) : \exists b.\ aRb\}$, $A\top$ is the relation that ignores b altogether, and the third link is $R\mapsto\mathbb{1}\cap R/\top=\{(a,a) : \forall b.\ aRb\}$.

$$\mathcal{D}\dashv\cdot\top\dashv\mathbb{1}\cap\cdot/\top \qquad (1.3c)$$

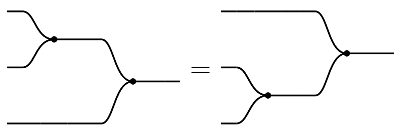
Three is where it stops, and one map breaks both ends. Take $A=\{a_1,a_2\}$ and $B=\{b\}$: $f[\{a_1\}]\cap f[\{a_2\}]=\{b\}$ while $f[\{a_1\}\cap\{a_2\}]=\varnothing$, so $\cdot f$ does not preserve meets and has no left adjoint, and the same two subsets give $\forall\_f(\{a_1\}\cup\{a_2\})=\{b\}$ against $\forall\_f\{a_1\}\cup\forall\_f\{a_2\}=\varnothing$, so $/f^\circ$ does not preserve joins and has no right adjoint. A monic f restores the binary case and no more — $\top f$ still reaches only $\mathrm{im}\ f$, and $\forall\_f\bot=B\setminus\mathrm{im}\ f$ is still not $\bot$. The chain extends only when f° is a map as well, and then $/f^\circ=\cdot f^{\circ\circ}=\cdot f$ and it repeats forever. For an S with neither S nor S° a map there is no triple at all: $\cdot S\dashv/S$ is two links and stops.

The rows that chain forever say nothing by it: $^\circ$ is an order-isomorphism of hom-posets, so it is adjoint to itself on both sides and $^\circ\dashv{}^\circ\dashv{}^\circ$ has no end, and §1.1′s other row of that kind, $\text{⟜}\lhd\dashv\rhd\text{⊸}$, goes the same way.

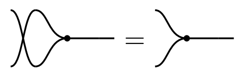
§2. Relations

> Rel is a poset-enriched category with (⊗, °) where ⊗ is commutative cartisian product, and converse `°⊣°`, and
>
> C: `(▷ : A⊗A→A, ∘− : 𝕀→A)` a commutative monoid with `C°⊣C`. We write `C°` as `(◁ : A→A⊗A, −∘ : A→𝕀)` (2a)

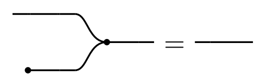
(2b)



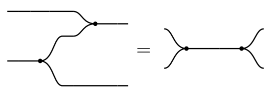
▷ associative



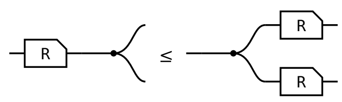
▷ commutative



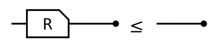
∘− is its unit



Frobenius, one half — the other is its °



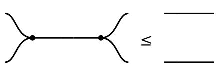
R◁≤◁(R⊗R)



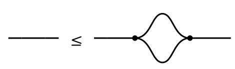
R−∘≤−∘

§2.1. ∀ object A, `(A,◁,−∘)⊣(A,▷,∘−)`

(2.1a)



▷◁≤𝟙



𝟙≤◁▷

∘−−∘≤𝟙

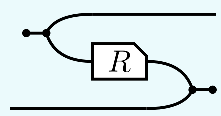
𝟙≤−∘∘−

The last of these is the only one that makes a picture **bigger**, and it is worth a name: **a wire is below the cut wire**. In `Rel` it reads `{(a,a)}⊆A×A` — cut a wire and its two ends stop having to agree, so cutting can only add pairs. It is the one weakening this calculus gives away for free.

§3. ° : $\mathcal{C}^{op} \longrightarrow \mathcal{C}$ is a 2 functor

(3a)

° is primitive, part of the data the first section lists. It turns both of `R`'s wires round, and the Frobenius structure DRAWS that — the picture and the formula below are `R°`, not its definition:



`R°=(⟜◁⊗𝟙)(𝟙⊗R⊗𝟙)(𝟙⊗▷⊸)`

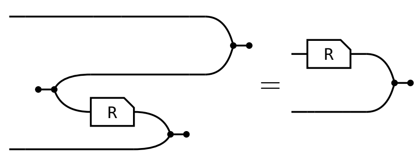
where `⟜◁ : 𝕀→a⊗a` opens a pair of wires out of nothing and `▷⊸ : b⊗b→𝕀` closes one, so the input of `R°` is where the output of `R` was. Taking that formula as the definition would now be circular: `◁` and `⊸` are `▷°` and `⟜°`. The laws of ° are that it is a contravariant 2-functor `° : 𝒞ᵒᵖ→𝒞`:

(i) `𝟙°=𝟙` (ii) `(RS)°=S°R°` (iii) `(R⊗S)°=R°⊗S°` (iv) `R≤S` implies `R°≤S°`

§3.1. The slide

The one rule (ii) and (iii) use, and each of them uses it twice. A converse facing a merge on the lower strand is the box itself, upright, on the upper one:

(3.1a)

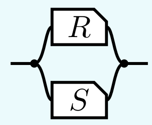


`R°` is DEFINED as the bending of `(R⊗𝟙)▷⊸`, so the slide claims only that unbending it gives that back — and unbending undoes bending for every arrow. That is the snake above with a passenger: the `⟜◁` bends the `a` strand down and the `▷⊸` brings it back up, while the `b` strand rides through untouched. Nothing else is spent below.

§4. ∩ is a commutative idempotent monoid on every hom-set

(4a)

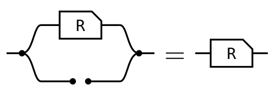
The **meet** of R,S : a→b, the paper's **convolution**, is R∩S:=◁(R⊗S)▷ — copy the input, run R and S on the two copies, merge the results — so what comes out is what both of them do.



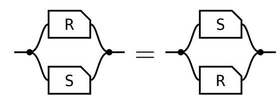
transcribed: a definition has no statement to export

On every hom-set it is associative, commutative and idempotent, with unit the maximal arrow T=⊸∘∘⟜ (the paper's Lemma 4.11).

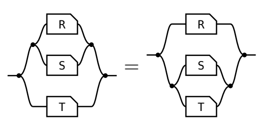
(4b)



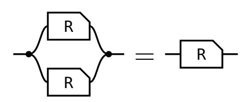
unit: one half of T per end — the merge's unit law absorbs the ∘—, the copy's counit law the —∘



commutative: σ crosses R⊗S by naturality and is absorbed by cocommutativity and commutativity



associative: coassociativity and associativity; ⊗ re-brackets for nothing, being strict here

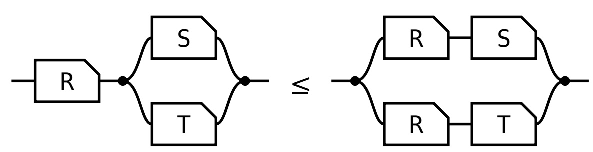


idempotent: the one that is not bookkeeping — the lax copy law is the whole of it, worked in allegory2

So ≤ is the order this monoid induces. R∩S≤R comes from the unit, and idempotency turns anything under both S and T into something under S∩T, since R=R∩R≤S∩T.

And one law relating ∩ to composition, which is **not** an equation:

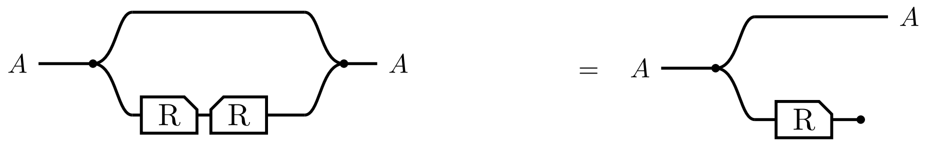
(4c)

semi-distributivity, and what supplies it	picture
R (S∩T)⊑RS∩RT — the lax copy law. Equality exactly when R is single valued: the Maps section's F(R∩S)=FR∩FS.	

§5. Domain and range

> The **domain** `Dom(R)≜𝟙∩RR°` and the **range** `Ran R≜Dom(R°)`. (5a)

(5b)



▷ lands the return leg back on the value ◁ handed out,
so `R°▷` cuts to ⊸ — Frobenius

Running `R` and throwing the result away leaves only the fact that `R` could fire, and `Ran R` is the same picture with the box mirrored. In `Rel` both steps are `{(a,a) : ∃b. a R b}`.

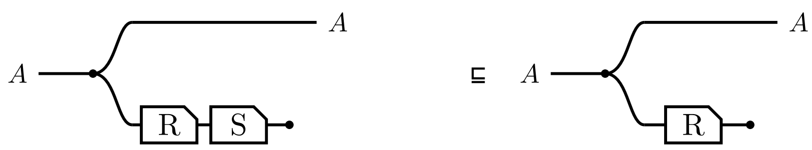
(5c)

`Dom(R)≜𝟙∩RR°`
`Dom(R)⊑A⟺R⊑AR`, for `A` coreflexive
`Dom(RS)⊑Dom(R)`
`Dom(R∩S)=𝟙∩SR°`
`R` entire ⟺`Dom(R)=𝟙⟺𝟙⊑RR°`
`R` simple ⟺`R°R⊑𝟙`
`R` a map ⟺`R` entire and simple
`R,S` entire ⟹`RS` entire — likewise simple, likewise maps
`RS` entire ⟹`R` entire

§5.1. Sliding the discard

`Dom(RS)⊑Dom(R)`, and a single glyph for `Dom` would have nothing to slide: with the box and the discard drawn apart, the law is one dot walking back along the lower strand.

(5.1a)



`S⊸⊑⊸`, the lax axiom for ⊸ in the first section
— the discard slides back past `S`

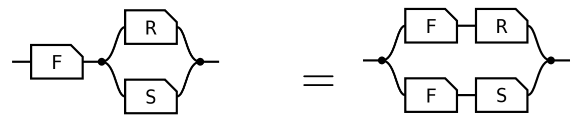
Equality is `S` entire, which is the same picture read as `Dom(R)=𝟙⟺R` entire.

§6. Maps

(6a)

surjective $\mathbb{1}\sqsubseteq R^\circ R$, that is, R° entire.	**single valued** $R^\circ R\sqsubseteq\mathbb{1}$
entire $\mathbb{1}\sqsubseteq RR^\circ$. With **single valued**, a **map**.	**injective** $RR^\circ\sqsubseteq\mathbb{1}$

(6b)

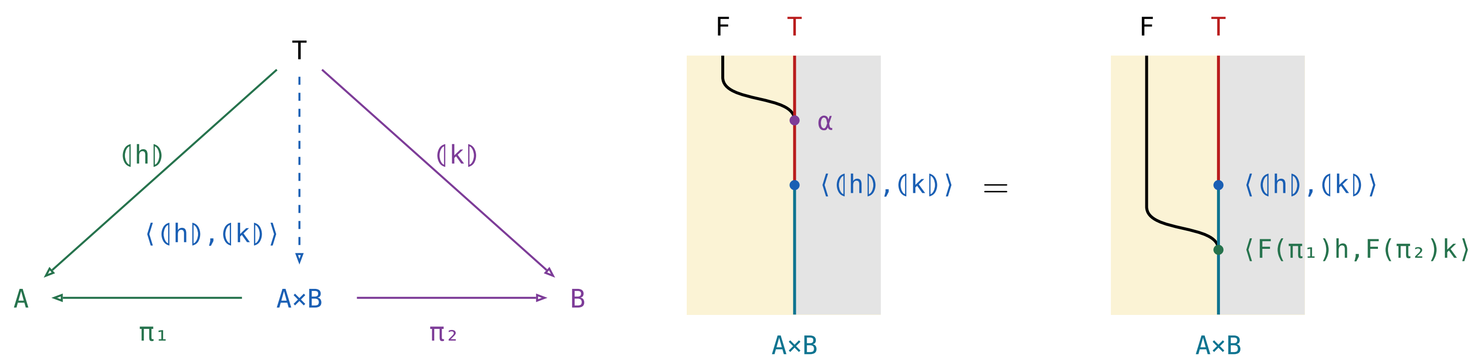
law	picture
$F(R\sqcap S)=FR\sqcap FS$ F single valued	 one person who admires both — a at A, b at B

§7. Reduce in $\mathcal{S}et$

(7a)

definition	type	note
`listr A::=nil|cons (A,listr A)`	$\mathcal{S}et\to\mathcal{S}et$	The cons-lists over `A`, the datatype every row below folds (B&dM p. 55).
$sum\triangleq⦇[zero,plus]⦈$	$listr\ Nat\to Nat$	$plus(a,b)=a+b$.
$length\triangleq⦇[zero,\pi_2\ succ]⦈$	$listr\ A\to Nat$	π_2 drops the head and keeps the count of the tail, `succ` adds one for the head.
$average\triangleq\langle sum,length\rangle\ div$	$listr\ Nat\to Real$	$div(m,n)=m/n$, with $div(0,0)=0$ so `average` is total. Traverses the list twice.
banana-split law $\langle⦇h⦈,⦇k⦈\rangle=⦇\langle F(\pi_1)h,F(\pi_2)k\rangle⦈$	$T\to A\times B$	Any fork of folds is a single fold, hence one traversal — F the base functor.
what it reduces to $\alpha\langle⦇h⦈,⦇k⦈\rangle=F(\langle⦇h⦈,⦇k⦈\rangle)\langle F(\pi_1)h,F(\pi_2)k\rangle$	$FT\to A\times B$	All that (11.6.1a) leaves to check: the fork satisfies the defining equation.
the instance $\langle sum,length\rangle=⦇[zeros,pluss]⦈$	$listr\ Nat\to Nat\times Nat$	$pluss(a,(b,n))=(a+b,n+1)$, so `average` runs in one pass.
$preds\ n=[n,n-1,\dots,1]$	$Nat\to[Nat]$	B&dM Ex 3.6 (p. 57): apply Ex 3.4 to write `preds` as $⦇k⦈\pi_1$.

(7b)



$\langle⦇h⦈,⦇k⦈\rangle=⦇\langle F(\pi_1)h,F(\pi_2)k\rangle⦈$ banana split

§7.1. Fokkinga's mutual recursion theorem

- $F\text{-}Alg(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $U : F\text{-}Alg(\mathcal{A})\to\mathcal{A}$ **creates** them: the product of $h : FA\to A$ and $k : FB\to B$ is carried by $A\times B$, with structure $\langle F(\pi_1)h, F(\pi_2)k\rangle : F(A\times B)\to A\times B$.
- π_1 and π_2 are homomorphisms out of it, and $\langle p,q\rangle : X\to A\times B$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (7a) IS that product's universal property read at the initial algebra: $(\!|h|\!)$ and $(\!|k|\!)$ are the unique homomorphisms to (A,h) and (B,k), so their fork is the unique homomorphism into the product, which is $(\!|\langle F(\pi_1)h, F(\pi_2)k\rangle|\!)$.
- Fokkinga's theorem is **strictly more general** and is not that product: in (7.1a) the two arrows leave $F(A\times B)$, not FA and FB, so each sees BOTH components. The product is the case where they factor as $F(\pi_1)h$ and $F(\pi_2)k$; B&dM's Ex 3.4 is the other special case (p. 58). What it says: **any algebra on a product carrier is folded by a fork**.

$$\begin{tikzcd} FT \arrow[r, "\alpha"] \arrow[d, "F(\langle f,g\rangle)"'] & T \arrow[d, dashed, "f"] \\ F(A\times B) \arrow[r, "h"'] & A \end{tikzcd} \qquad \begin{tikzcd} FT \arrow[r, "\alpha"] \arrow[d, "F(\langle f,g\rangle)"'] & T \arrow[d, dashed, "g"] \\ F(A\times B) \arrow[r, "k"'] & B \end{tikzcd}$$

$$\alpha f = F(\langle f,g\rangle)h \wedge \alpha g = F(\langle f,g\rangle)k \equiv \langle f,g\rangle = (\!|\langle h,k\rangle|\!) \tag{7.1a}$$

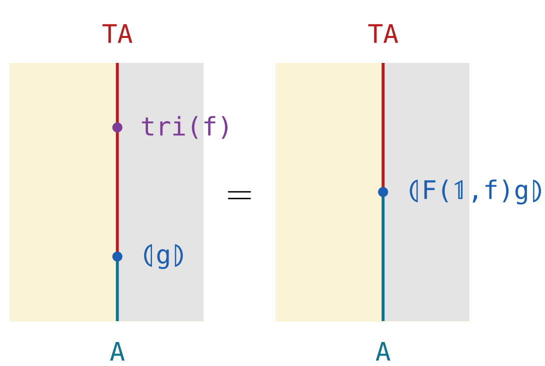
§7.2. Ruby triangles

(7.2a)

stage	definition
informally (p. 58)	$tri(f)\ [a_0,a_1,\dots,a_i,\dots,a_n]=[a_0,f\ a_1,\dots,f^i\ a_i,\dots,f^n\ a_n]$
for cons-lists (p. 59)	$tri(f)=(\!|[nil,(1\!\!1\times listr(f))\ cons]|\!)$
with the base functor named	$F(A,B)=1+A\times B$, $\alpha=[nil,cons]$, so $tri(f)=(\!|F(1\!\!1,listr(f))\alpha|\!)$
abstractly (p. 59)	F a bifunctor with initial type (α,T): $tri(f)=(\!|F(1\!\!1,T(f))\alpha|\!)$

For the definition to make sense $f : A\to A$ is required, and then $tri(f) : TA\to TA$.

$$\begin{tikzcd} TA \arrow[rr, "tri(f)"] \arrow[dr, dashed, "(\!|F(1\!\!1,f)g|\!)"'] & & TA \arrow[dl, dashed, "(\!|g|\!)"] \\ & A & \end{tikzcd}$$



$$tri(f)\ (\!|g|\!) = (\!|F(1\!\!1,f)g|\!) \quad\Longleftarrow\quad gf = F(f,f)g \tag{7.2b}$$

(7.2b) is Horner's rule, generalised from cons-lists to any initial type; B&dM call it that because for cons-lists it is the schoolbook method for evaluating a polynomial (p. 58). Fusion reduces it to $F(1\!\!1,T(f))\alpha(\!|g|\!)=F(1\!\!1,(\!|g|\!))F(1\!\!1,f)g$ (p. 59).

§7.3. Depth of a tree

(7.3a)

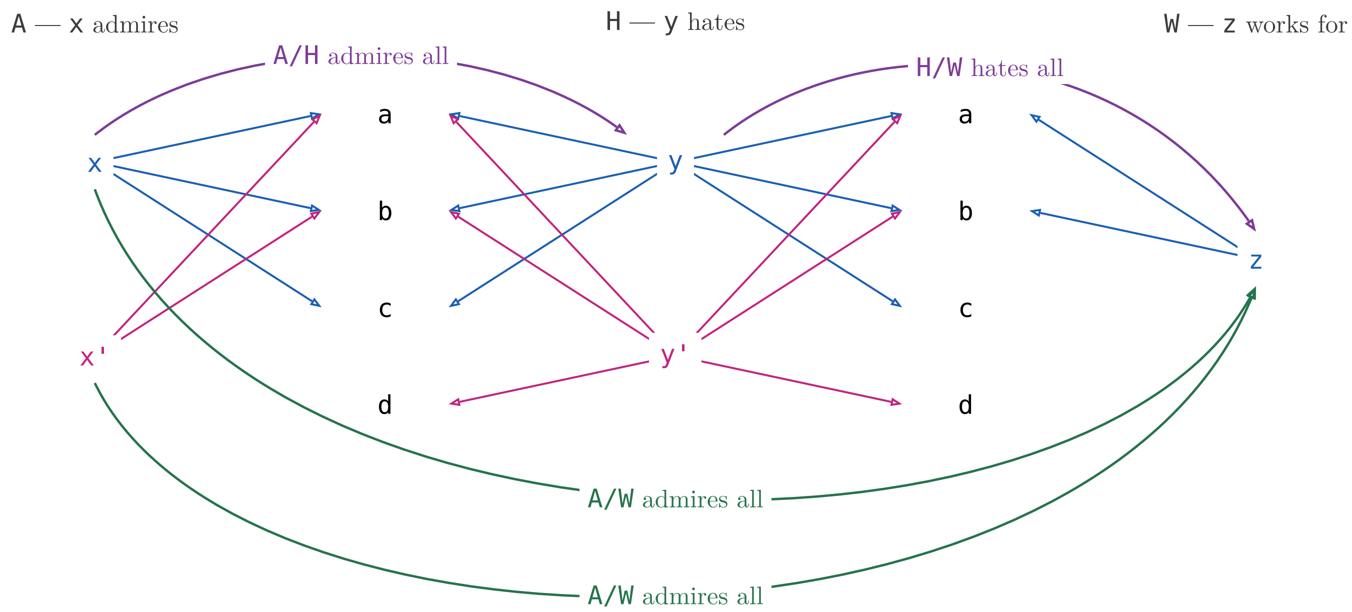
the datatype	`tree A::=tip A|bin (tree A,tree A)`, base functor `F(A,B)=A+B×B`, initial type `([tip,bin],tree)`
the fold	`⦇[g,h]⦈` is the unique `f` with `f (tip a)=g a` and `f (bin (x,y))=h (f x,f y)`
`tree(f)`	`tree(f)=⦇F(f,𝟙) [tip,bin]⦈`; pointwise `tree(f) (tip a)=tip (f a)` and `tree(f) (bin (x,y))=bin (tree(f) x,tree(f) y)`
`max`	`max=⦇[𝟙,bmax]⦈`, where `bmax (a,b)` is the larger of `a` and `b`
`depths`	`depths=tree(zero) tri(succ)` — replaces every tip by its depth in the tree, `zero` the constant function returning 0 and `succ` the successor
`depth`	`depth=depths max`

§8. / is all of

```
x (R/S) y⟺∀p. y S p→x R p
x (S\R) y⟺∀p. p S x→p R y
/compares images: S(y)⊆R(x).   \ compares preimages: S°(x)⊆R°(y).
example: A admires,H hates,W works for
x (A/H) y — x admires everyone y hates.
x (H\A) y — everyone who hates x admires y.
```
(8a)

§8.1. (R/S)(S/W)⊑R/W

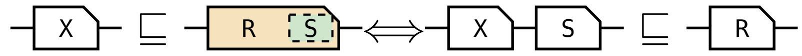
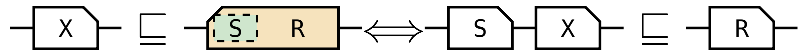
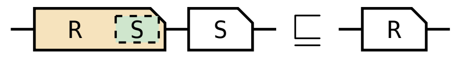
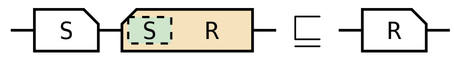
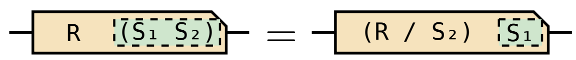
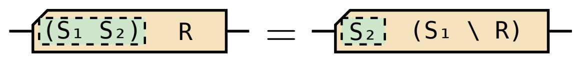
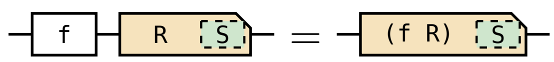
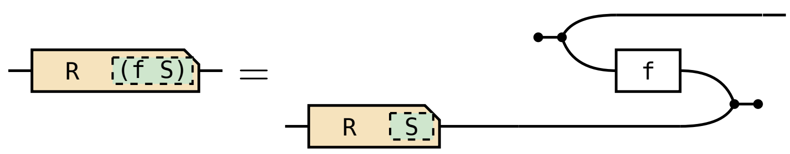
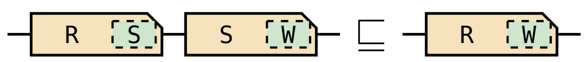
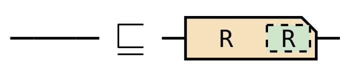
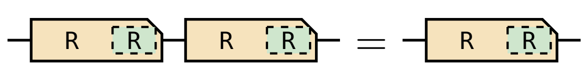
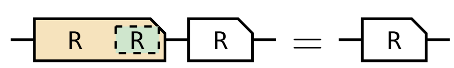
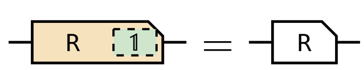
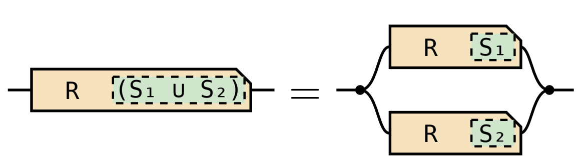
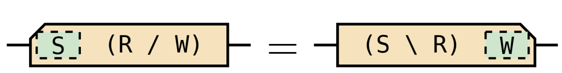
(8.1a)



x (A/H) y — x admires everyone y hates
y (H/W) z — y hates everyone z works for
x (A/W) z — x admires everyone z works for (8.1b)

(A/H)(H/W) is a path: x → y → z, and that is all of it. A/W also holds of x', who admires everyone z works for — but x' does not admire everyone anybody hates, so nothing composes to it. The missing path is exactly the strictness of (R/S)(S/W)⊑R/W.

(8.1c)

$X\sqsubseteq R/S \iff XS\sqsubseteq R$ X is any x-to-y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S, and R/S is the largest such.	
$X\sqsubseteq S\backslash R \iff SX\sqsubseteq R$ The mirror — divide on the left when x comes first.	
$(R/S)S\sqsubseteq R$ There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S=\varnothing$: R/S is everyone, $(R/S)S=\varnothing$.	
$S\ (S\backslash R)\sqsubseteq R$ The mirror.	
associate: $R/(S_1S_2)=(R/S_2)/S_1$ **A friend's enemy** is two hops: divide by the far end first.	
$(S_1S_2)\backslash R=S_2\backslash(S_1\backslash R)$ The mirror.	
maps: $f\ (R/S)=(fR)/S$ Rename x before or after dividing — the licence to write fR/S.	
$R/(fS)=(R/S)f^{\circ}$ Rename y: a map leaves a denominator as f° outside the box.	
$(R/S)(S/W)\sqsubseteq R/W$ Someone who admires all of a hate-set that already covers everyone z works for admires those people too.	
$\mathbb{1}\sqsubseteq R/R$ R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.	
$(R/R)(R/R)=R/R$ R/R is the preorder **admires at least as much as**, and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $\mathbb{1}\sqsubseteq R/R$ above it is an equality.	
$(R/R)R=R$ Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.	
$R/\mathbb{1}=R$ Dividing by $\mathbb{1}$: p's set is just $\{p\}$, so admiring all of it is admiring p.	
$R/(S_1\cup S_2)=R/S_1\cap R/S_2$ Admiring a combined hate-set is admiring each set in full.	
$S\backslash(R/W)=(S\backslash R)/W$ Which is why $S\backslash R/W$ needs no bracket.	

Fifteen laws, fifteen pictures, and not one shows a generator: ∩, ∪, ° and composition are what the Frobenius generators build, and / is none of those — it is posited, with nothing to unfold.

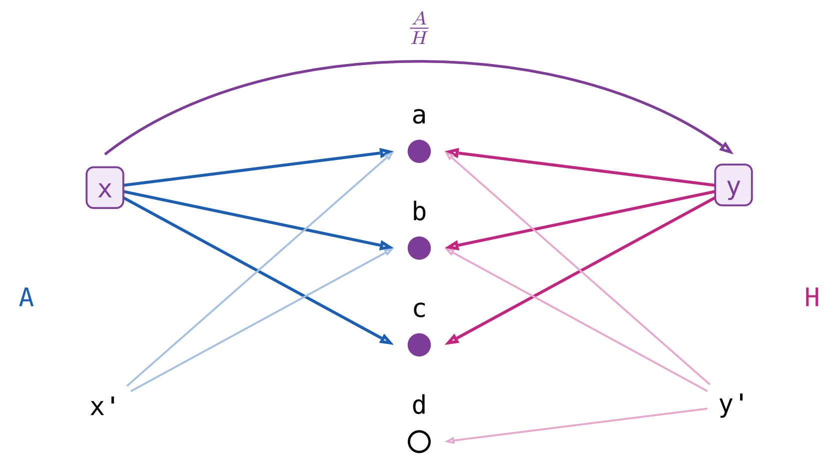
§9. $\frac{R}{S}$

> $\frac{R}{S}$ `≜(R/S)∩(S/R)°`. In `Rel` `x` and `y` has the same image: `∀p. (x R p⟺y S p)` (9a)

`x (A/H) y` — `x` admires everyone `y` hates (9b)
`x ((H/A)°) y` — `x` admires only people `y` hates
`x ((A/H)∩(H/A)°) y` — `x` admires only and all whom `y` hates

`x` admires exactly whom `y` hates: `/` is **all of**, $\frac{R}{S}$ is **only all of**.

(9c)



(9d)

law	the reading
$X \sqsubseteq \frac{R}{S} \iff XS \sqsubseteq R$ and `X°R⊑S`	`X` may pair `x` with `y` only when `x` admires exactly whom `y` hates. Both halves must typecheck, so the operation is **partial**.
$\left(\frac{R}{S}\right)^{\circ} = \frac{S}{R}$	Matching is symmetric.
$\frac{R}{S}\frac{S}{W} \sqsubseteq \frac{R}{W}$	And transitive.
$\frac{R}{S}S \sqsubseteq R$	$\left(\exists y.\, x\left(\frac{R}{S}\right)y \wedge ySp\right) \rightarrow xRp$ `x only admires whom y hates` $\frac{R}{S}S = \mathrm{Dom}\left(\frac{R}{S}\right)R$

$\frac{R}{R}R = R$	$\left(\exists y.\, x\left(\frac{R}{R}\right)y \wedge yRp\right) \iff xRp$
	`x and y admire the same people`
	`y=x always qualifies (𝟙⊑R%R below)`

$\mathbb{1} \sqsubseteq \frac{R}{R}$	$x\left(\frac{R}{R}\right)y$ if `x` and `y` admires the same peoples.
$\left(\frac{R}{R}\right)^{2} = \frac{R}{R}$	So the relation **admires the same people** is an equivalence relation.
$X \sqsubseteq \frac{R}{R} \iff XR \sqsubseteq R$, for symmetric `X`	The largest symmetric arrow that leaves `R` alone.
$\frac{R}{\mathbb{1}}$ is the **simple part** of `R`	The people who admire exactly one person and nobody else. It equals `R` only when `R` is simple, unlike `R/𝟙=R`.
`Dom` $\frac{R}{S}$ `=𝟙∩(R/S)(S/R)`	Its domain is the **domain of simplicity** of `R`.

§10. Freyd's Power allegories

(10a)

A **power allegory** is a division allegory with one unary operation on arrows, ∋ **epsiloff**, subject to

$$\ni_R\Box = R\Box,\quad \ni_R = \ni_{R\,\Box}$$
$$\mathbb{1} \sqsubseteq (R/\ni_R)(\ni_R/R) \qquad \ni_R \text{ is } \textbf{thick}$$
$$\frac{\ni_R}{\ni_R} = \mathbb{1} \qquad \ni_R \text{ is } \textbf{straight}$$

`R□` is `R`'s target, an identity arrow. For `R : A→B` write `∋ : EB→B`, dropping the subscript.

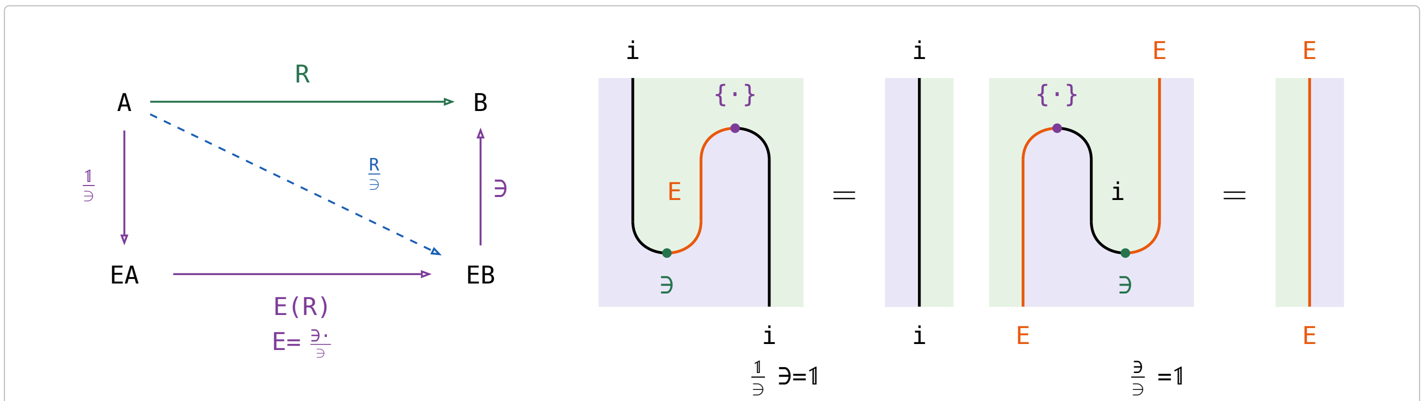
`∈≜∋° : A→EA`

(10b)

	law	the reading
1	$\ni_R\Box = R\Box,\ \ni_R = \ni_{R\,\Box}$	One ∋ per object, not per arrow.
2	∋ is **thick**	**Comprehension**: every `x` has a set of exactly the people `x` admires.
3	$\frac{R}{\ni}$ `: A→EB`, for `R : A→B`	convert a relation to a function. `a` $\frac{R}{\ni}$ `={b|a R b}`
4	$\frac{R}{\ni}$ is a **map**	
5	$\frac{R}{\ni}$ `∋=R`	
6	`F⊑` $\frac{F\ni}{\ni}$, `F` simple	A partial choice of sets is inside the total one.
7	$\frac{1}{\ni}$, the **singleton map**, monic	The one-person set.
8	$\frac{\ni}{\ni}$ `=𝟙`	Make the set of a set, then read it back one level down.
9	**fusion:** $\frac{fR}{\ni}$ `=f` $\frac{R}{\ni}$, `f` a map	Naturality of the unit, `f` $\frac{1}{\ni}$ `=` $\frac{1}{\ni}$ `(E(f))`.
10	$\frac{f}{\ni}$ `=f` $\frac{1}{\ni}$, `f` a map	Rename first or take singletons first — the fusion row above at `R=𝟙`.
11	$\frac{R}{S} = \frac{R}{\ni}\left(\frac{S}{\ni}\right)^{\circ}$	`x` and `y` match when `R` sends `x` and `S` sends `y` to the same set.
12	`E(R)≜` $\frac{\ni R}{\ni}$, `E≜` $\frac{\ni\cdot}{\ni}$	`E(R): EA→EB`, image of a set of A
13	$\frac{R}{\ni}$ `=` $\frac{1}{\ni}$ `E(R)`	$\frac{1}{\ni}$`: x↦{x}`
14	$\frac{S}{\ni}$ `E(R)=` $\frac{S}{\ni}$ $\frac{\ni R}{\ni}$ `=` $\frac{SR}{\ni}$	absorption — the monad's composition law, $\frac{S}{\ni}$ `⋄` $\frac{R}{\ni}$ `=` $\frac{SR}{\ni}$, §10.2
15	`subset≜∋/∋ : EA→EA`	`xs subset ys⟺∀a. ys∋a→xs∋a`, that is `ys⊆xs`, not `xs⊆ys`.

§10.1. i⊣E Power Allegory defined as adjunction

(10.1a)



$\frac{R}{\ni}$ `∋=R` EA is the powerset of A. B&dM write PA — standard mathematics, but here P is already the relator `P(R)`.

§10.2. $\mathcal{A}$≅Kleisli(E)

$$\mathsf{E(R)} \triangleq \tfrac{\ni \mathsf{R}}{\ni},\quad \{\cdot\} = \tfrac{\mathbb{1}}{\ni} \;:\; \mathsf{A \to EA},\quad \mathsf{union} = \mathsf{E}(\ni) = \tfrac{\ni\ni}{\ni} \;:\; \mathsf{E(EA) \to EA} \tag{10.2a}$$

$$\mathsf{f \diamond g} \triangleq \mathsf{f\ E(g)\ union} \;:\; \mathsf{A \to EC},\quad \text{for } \mathsf{f} : \mathsf{A \to EB} \text{ and } \mathsf{g} : \mathsf{B \to EC}$$

the monad is on Map($\mathcal{A}$), not on the allegory: E is a relator on all relations, but {·}, union and f E(g) union are maps, and the Kleisli construction happens where they live.

$$\boxed{\tfrac{\mathsf{S}}{\ni} \diamond \tfrac{\mathsf{R}}{\ni}} \;\overset{}{\underset{\text{definition of } \diamond,\ \mathsf{union=E(\ni)}}{=\!=}}\; \boxed{\tfrac{\mathsf{S}}{\ni}\ \mathsf{E}(\tfrac{\mathsf{R}}{\ni})\mathsf{E}(\ni)} \;\underset{\mathsf{E}\text{ a functor, } \mathsf{E(X)E(Y)=E(XY)}}{=\!=}\; \boxed{\tfrac{\mathsf{S}}{\ni}\ \mathsf{E}(\tfrac{\mathsf{R}}{\ni}\ni)} \tag{10.2b}$$

$$\underset{\text{(10b), } \frac{\mathsf{R}}{\ni}\ni=\mathsf{R}}{=\!=}\; \boxed{\tfrac{\mathsf{S}}{\ni}\ \mathsf{E(R)}} \;\underset{\text{(10b), absorption}}{=\!=}\; \boxed{\tfrac{\mathsf{SR}}{\ni}}$$

the first three steps only unfold ⋄ — E is a functor, so $\mathsf{E}(\tfrac{\mathsf{R}}{\ni})\mathsf{union}=\mathsf{E}(\tfrac{\mathsf{R}}{\ni}\ni)=\mathsf{E(R)}$ and the union is gone. Absorption, the row of (10b), is the whole law, and it is the functoriality of $\tfrac{\cdot}{\ni}$.

𝟙 goes to $\tfrac{\mathbb{1}}{\ni}$={·}, the Kleisli identity, by definition — so $\tfrac{\cdot}{\ni}$ is an isomorphism of categories $\mathcal{A}$≅Kleisli(E), and §10.1's i⊣E is the Kleisli adjunction. i is the identity on objects, so every object of the allegory is free.

§11. Relator

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from R to S IS R⊑S. A **relator** F : $\mathcal{C} \to \mathcal{D}$ is a 2-functor between allegories: (11a)

$$\mathsf{F(\mathbb{1})=\mathbb{1}} \qquad \mathsf{F(RS)=F(R)F(S)} \qquad \mathsf{R \sqsubseteq S \Longrightarrow F(R) \sqsubseteq F(S)}$$

Preserving ° is **not** asked for — ° is an identity-on-objects involution $\mathcal{C}^{op} \to \mathcal{C}$, no part of the 2-category.

(11b)

the statement	**B&dM**
For f a map, F(f) is a map and F(f°)=F(f)°.	Lemma 5.1
Over a **tabular** allegory a functor is a relator ⟺ it preserves °.	Theorem 5.1
F(R°)=F(R)° for every R, so F(R)° needs no bracket.	after Theorem 5.1, p. 113
Two relators agreeing on maps are equal.	Corollary 5.1
F(X∩Y)=F(X)∩F(Y) for X,Y coreflexive.	Ex 5.2
F(R∩S)⊑F(R)∩F(S), and strictly.	Ex 5.2, the restriction
F(Dom(R))=Dom(F(R)) for F preserving °.	—

The **power relator** P — xs P(R) ys⟺(∀a∈xs. ∃b∈ys. a R b)∧(∀b∈ys. ∃a∈xs. a R b) — is where the fourth is strict: for $\mathsf{R=\{(a_1,b_1),(a_2,b_2)\}}$ and $\mathsf{S=\{(a_1,b_2),(a_2,b_1)\}}$ the pair $\mathsf{(\{a_1,a_2\},\{b_1,b_2\})}$ is in P(R)∩P(S), while R∩S=∅.

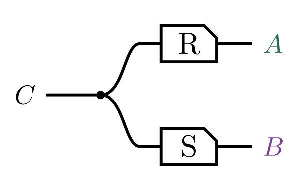
§11.1. Fork ⟨R,S⟩

The **fork** of R : C→A and S : C→B is $\mathsf{\langle R,S\rangle \triangleq R\pi_1{}^\circ \cap S\pi_2{}^\circ}$, where (π_1,π_2) is the tabulation of ⊤. (11.1a)

$$\mathsf{\langle R,S\rangle\pi_1 = Dom(S)R} \qquad \mathsf{\langle R,S\rangle\pi_2 = Dom(R)S} \tag{11.1b}$$

(11.1c)

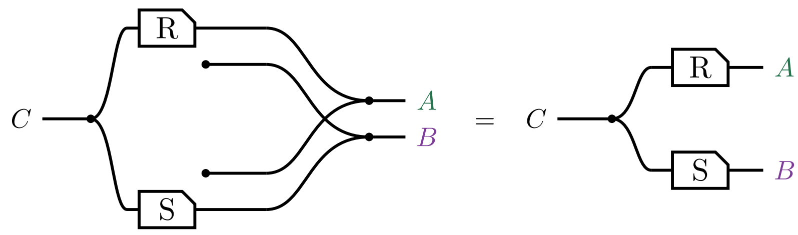
$$\begin{tikzcd} & A & \\ C \arrow[ur, "\mathsf{R}"] \arrow[dr, "\mathsf{S}"'] \arrow[rr, dashed, "\langle R,S\rangle"] & & A\times B \arrow[ul, "\pi_1"'] \arrow[dl, "\pi_2"] \\ & B & \end{tikzcd}$$



A domain is coreflexive, so $\langle R,S\rangle\pi_1 \sqsubseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f,g\rangle$ is unique. In Rel, $c\ \langle R,S\rangle\ (a,b)$ iff $c\ R\ a$ and $c\ S\ b$ — copy c, then R on one strand and S on the other, which is $\lhd(R\otimes S)$ on the right.

No $^\circ$ survives the translation. $\pi_1 = \mathbb{1}\otimes\multimap$ discards the second component, so $\pi_1{}^\circ = \mathbb{1}\otimes\multimapinv$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(11.1d)



$\multimapinv\!\rhd = \mathbb{1}$ on each half

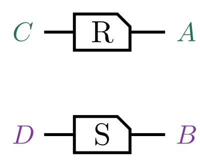
§11.1.1. Relational product $R\times S$

(11.1.1a)

> $R\times S \triangleq \langle \pi_1 R, \pi_2 S\rangle$, a relator in each argument but no longer a categorical product.

(11.1.1b)

$$\begin{tikzcd}
C \arrow[rr, "R"] & & A \\
C\times D \arrow[u, "\pi_1"] \arrow[d, "\pi_2"'] \arrow[rr, dashed, "R\times S"] \arrow[urr, phantom, "\sqsupseteq"] \arrow[drr, phantom, "\sqsubseteq"] & & A\times B \arrow[u, "\pi_1"'] \arrow[d, "\pi_2"] \\
D \arrow[rr, "S"] & & B
\end{tikzcd}$$

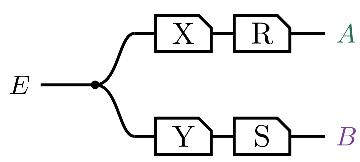


Right-then-up is $(R\times S)\pi_1$, up-then-right is $\pi_1 R$, and $(R\times S)\pi_1 \sqsubseteq \pi_1 R$, equality when S is entire. In Rel, $(c,d)\ (R\times S)\ (a,b)$ iff $c\ R\ a$ and $d\ S\ b$ — two strands side by side, no copy dot: $R\times S = R\otimes S$.

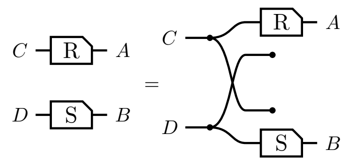
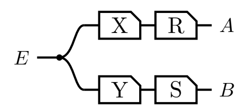
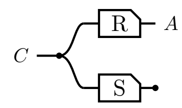
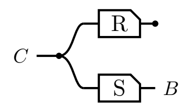
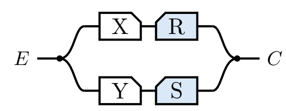
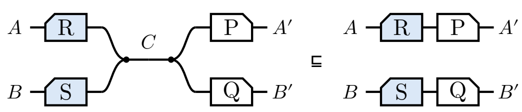
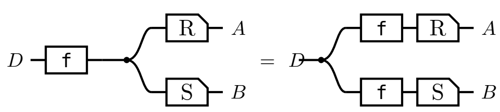
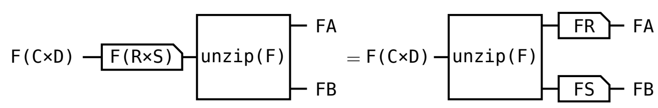
§11.1.2. Absorption

For $X : E \to C$ and $Y : E \to D$, $\langle X,Y\rangle(R\times S) = \langle XR, YS\rangle$. Both sides are this picture:

(11.1.2a)

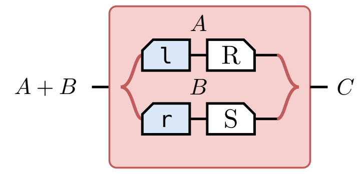
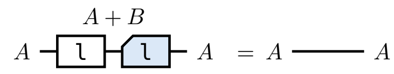
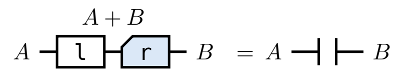
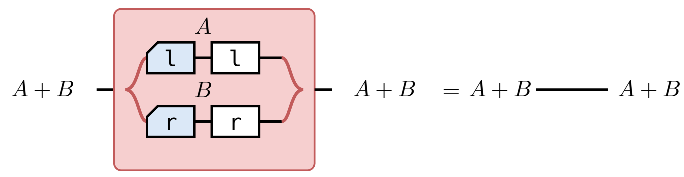


(11.1.2b)

the law	picture
1 $R\times S=\langle\pi_1 R,\pi_2 S\rangle$	
2 $\langle X,Y\rangle(R\times S)=\langle XR,YS\rangle$ both sides are the same strokes — (11.1.2a). Its $\sqsubseteq$ half is Ex 5.8, that half at $S:=\mathbb{1}$ and at $R:=\mathbb{1}$ B&dM's (5.4), (5.5) — stages of their proof of this row; Ex 5.6 is the $(R\times S)(U\times V)=(RU)\times(SV)$ it yields, at $R:=\mathbb{1}$ and $V:=\mathbb{1}$.	
3 $\langle R,S\rangle\pi_1=\mathrm{Dom}(S)R$ (11.1b)	
4 $\langle R,S\rangle\pi_2=\mathrm{Dom}(R)S$ (11.1b)	
5 $\langle X,Y\rangle\langle R,S\rangle^\circ=(XR^\circ)\cap(YS^\circ)$	
6 $\langle R,S\rangle^\circ\langle P,Q\rangle\sqsubseteq(R^\circ P)\times(S^\circ Q)$	
7 $f\langle R,S\rangle=\langle fR,fS\rangle$ f a map; it fails for an arbitrary arrow	
8 $F(R\times S)\mathrm{unzip}(F)=\mathrm{unzip}(F)(F(R)\times F(S))$ $\mathrm{unzip}(F)\triangleq\langle F(\pi_1),F(\pi_2)\rangle$, a map	
9 $g=\mathrm{curry}(f)\Longleftrightarrow(g\times\mathbb{1})\mathrm{eval}=f$ reading × as the relational product, does Rel have exponentials?	

§11.2. Coproduct $[R,S]\ :\ A+B\to C$

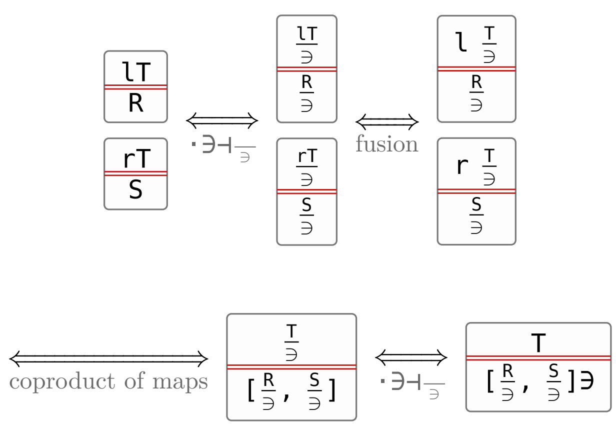
(11.2a)

the statement	picture
$[R,S] \triangleq l^\circ R \cup r^\circ S$ The tape is the union — a particle entering at A+B takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
$[R,S] = [\frac{R}{\ni}, \frac{S}{\ni}]\ni$	
$R+S \triangleq [Rl, Sr]$	
$l[R,S]=R$, $r[R,S]=S$, and $[R,S]$ is the only such arrow	
$ll^\circ = \mathbb{1} = rr^\circ$	
$lr^\circ = \bot = rl^\circ$	
$l^\circ l \cup r^\circ r = \mathbb{1}$	
$[U,V]^\circ[R,S] = U^\circ R \cup V^\circ S$	

§11.2.1. $[R,S] \triangleq [\frac{R}{\ni}, \frac{S}{\ni}]\ni$

The universal-property row is not free: l, r were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because $\frac{}{\ni}$ sends an arrow $A \to C$ to a map $A \to EC$ reversibly, so the map coproduct can be applied underneath it. For any $T : A+B \to C$,

(11.2.1a)

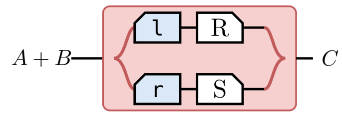
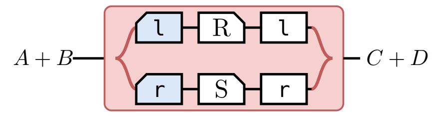
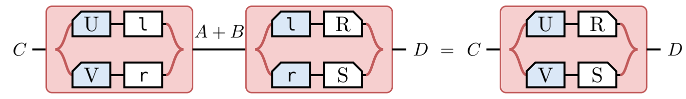
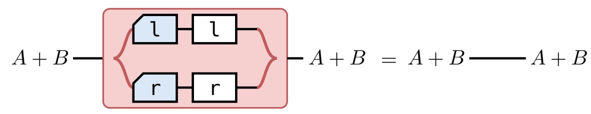


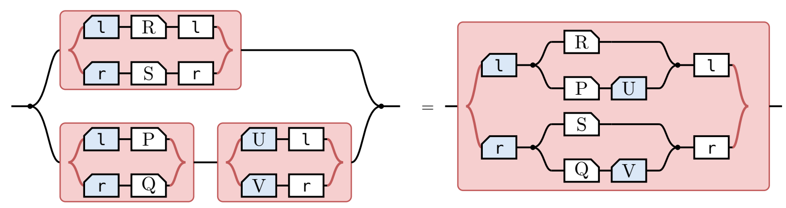
(11.2.1b)

$$\begin{tikzcd}
 & A \arrow[dl, "l"'] \arrow[dr, "\frac{R}{\ni}"] \arrow[rr, bend left, "R"] & & \\
A+B \arrow[rr, dashed, "{[\frac{R}{\ni}, \frac{S}{\ni}]}"] & & EC \arrow[r, "\ni"] & C \\
 & B \arrow[ul, "r"] \arrow[ur, "\frac{S}{\ni}"'] \arrow[urr, bend right, "S"'] & &
\end{tikzcd}$$

Nothing here holds only up to $\sqsubseteq$: every triangle commutes on the nose, which is the difference from the fork above. The border spells $[R,S]=[\frac{R}{\ni}, \frac{S}{\ni}]\ni$, and pushing $\ni$ into the union that $[\cdot,\cdot]$ on maps already is turns that back into the definition, $(l^\circ \frac{R}{\ni} \cup r^\circ \frac{S}{\ni})\ni = l^\circ R \cup r^\circ S$.

(11.2.1c)

B&dM	the law	picture
(5.9)	`[R,S]=(l°R)∪(r°S)` (11.2a)'s first row	
(5.10)	`R+S=[Rl,Sr]`	
(5.11)	`[U,V]°[R,S]=(U°R)∪(V°S)` (11.2a)'s last row	
Ex 5.12	`X≙[𝟙,⊥]=l°` and `Y≙[⊥,𝟙]=r°`, so `(Xl)∪(Yr)=[l,r]=𝟙` which is (5.9)	
Ex 5.13	prove (5.11), and say why duality does not carry it over from the product law	

Ex 5.14	`(R+S)∩([P,Q][U,V]°)`
	`=(R∩(PU°))+(S∩(QV°))`
	`[P,Q][U,V]°` is a full 2×2 of composites; R+S is diagonal, so the meet cuts the two off-diagonal branches
		

§11.3. The power relator $P(R)$

(11.3a)

For $R\ :\ A \to B$,

$$P(R) \triangleq ((\ni R)/\ni) \cap ((\ni R^\circ)/\ni)^\circ\ :\ EA \to EB$$

$$E(R) \triangleq \frac{\ni R}{\ni} = ((\ni R)/\ni) \cap (\ni/(\ni R))^\circ$$

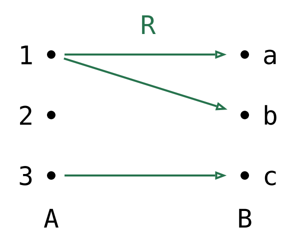
$$xs\ P(R)\ ys \iff (\forall a \in xs.\ \exists b \in ys.\ a\ R\ b) \land (\forall b \in ys.\ \exists a \in xs.\ a\ R\ b)$$

Every element of xs is related by R to some element of ys, and conversely.

(11.3b)

	in Rel	**in words**	**im(xs)**	**partner**
$(\ni R)/\ni$	$\forall y \in ys.\ \exists x \in xs.\ x\ R\ y$	$\forall y.$ some $xs\ R\ y$	$ys \subseteq im(xs)$	every y has a partner
$(\ni/(\ni R))^\circ$	$\forall y.\ (\exists x \in xs.\ x\ R\ y) \to y \in ys$	every y with some $xs\ R\ y$ is in ys	$ys \supseteq im(xs)$	ys leaves out no partner
$E(R)$	$ys = \{y \mid \exists x \in xs.\ x\ R\ y\}$	$ys =$ every y with some $xs\ R\ y$	$ys = im(xs)$	every y has a partner, and none is left out
$((\ni R^\circ)/\ni)^\circ$	$\forall x \in xs.\ \exists y \in ys.\ x\ R\ y$	$\forall x.\ x\ R$ some ys	—	every x has a partner
$P(R)$	$\forall y \in ys.\ \exists x \in xs.\ x\ R\ y$ and $\forall x \in xs.\ \exists y \in ys.\ x\ R\ y$	$\forall y.$ some $xs\ R\ y$ and $\forall x.\ x\ R$ some ys	—	every x and every y has a partner

(11.3c)



xs	$(\ni R)/\ni$	$(\ni/(\ni R))^\circ$	$E(R)$	$((\ni R^\circ)/\ni)^\circ$	$P(R)$
∅	∅	all 8 subsets of abc	∅	all 8 subsets of abc	∅
{1}	∅, a, b, ab	ab, abc	ab	a, b, ab, ac, bc, abc	a, b, ab
{2}	∅	all 8 subsets of abc	∅	none	none
{3}	∅, c	c, ac, bc, abc	c	c, ac, bc, abc	c
{1,2}	∅, a, b, ab	ab, abc	ab	none	none
{1,3}	all 8 subsets of abc	abc	abc	ac, bc, abc	ac, bc, abc
{2,3}	∅, c	c, ac, bc, abc	c	none	none
{1,2,3}	all 8 subsets of abc	abc	abc	none	none

(11.3d)

law	**the reading**
$X \sqsubseteq P(R) \iff X\ni \sqsubseteq \ni R$ and $X^\circ \ni \sqsubseteq \ni R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = P(R)^\circ$, and $R \sqsubseteq S \implies P(R) \sqsubseteq P(S)$.
$P(\mathbb{1}) = \frac{\ni}{\ni} = \mathbb{1}$	The straightness axiom verbatim: extensionality **is** P's unit law.
$P(f) = \frac{\ni f}{\ni}$, for f a map	In Rel, $xs\ P(f)\ ys \iff ys = \{f\ a \mid a \in xs\}$. The half at f° says every $a \in xs$ has its $f\ a$ on ys; f has just the one image per a, so that already says ys contains everything xs reaches, which is the fraction's second half. For a map the two definitions coincide.
$P(RS) = P(R)P(S)$	$\sqsupseteq$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(RS)$.

§11.4. Initial algebra

(11.4a)

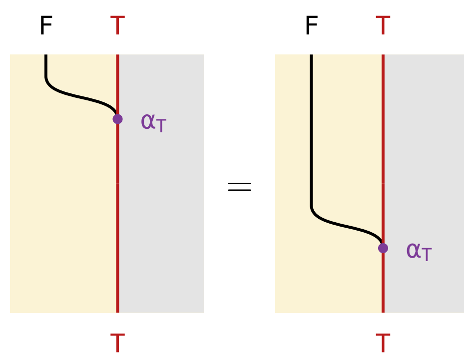
An **F-algebra** on A is a map $\alpha_{\mathsf{A}} : \mathsf{FA} \to \mathsf{A}$. An **F-homomorphism** from α_{A} to α_{B} is a map $\mathsf{h} : \mathsf{A} \to \mathsf{B}$ with $\alpha_{\mathsf{A}}\ \mathsf{h} = \mathsf{F(h)}\alpha_{\mathsf{B}}$. The **initial algebra** $\alpha_{\mathsf{T}} : \mathsf{FT} \to \mathsf{T}$ is the F-algebra with exactly one F-homomorphism $(\!|\alpha_{\mathsf{A}}|\!) : \mathsf{T} \to \mathsf{A}$ to every F-algebra α_{A}.

$$\begin{tikzcd}
\mathsf{FA} \arrow[r, "\alpha_{\mathsf{A}}"] \arrow[d, "\mathsf{F(h)}"'] & \mathsf{A} \arrow[d, "\mathsf{h}"] \\
\mathsf{FB} \arrow[r, "\alpha_{\mathsf{B}}"'] & \mathsf{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_{\mathsf{T}}"] \arrow[d, "\mathsf{F}((\!|\alpha_{\mathsf{A}}|\!))"'] & \mathsf{T} \arrow[d, dashed, "(\!|\alpha_{\mathsf{A}}|\!)"] \\
\mathsf{FA} \arrow[r, "\alpha_{\mathsf{A}}"'] & \mathsf{A}
\end{tikzcd}$$

§11.4.1. Reflection

(11.4.1a)

$$\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_{\mathsf{T}}"] \arrow[d, "\mathsf{F}(\mathbb{1})"'] & \mathsf{T} \arrow[d, dashed, "\mathbb{1}"] \\
\mathsf{FT} \arrow[r, "\alpha_{\mathsf{T}}"'] & \mathsf{T}
\end{tikzcd}$$



$(\!|\alpha_{\mathsf{T}}|\!) = \mathbb{1}$ (2.11)

Taking a value apart with α_{T} and putting it straight back is doing nothing.

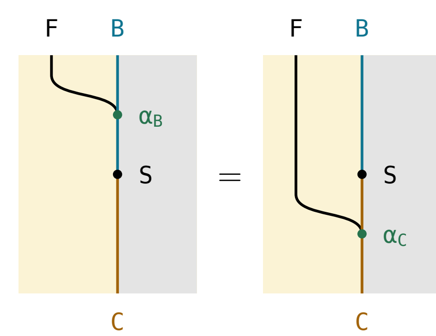
§11.4.2. Fusion

Fusion rewrites $(\!|\alpha_{\mathsf{B}}|\!)\mathsf{S}$ through a second algebra $\alpha_{\mathsf{C}} : \mathsf{FC} \to \mathsf{C}$ along an arrow $\mathsf{S} : \mathsf{B} \to \mathsf{C}$.

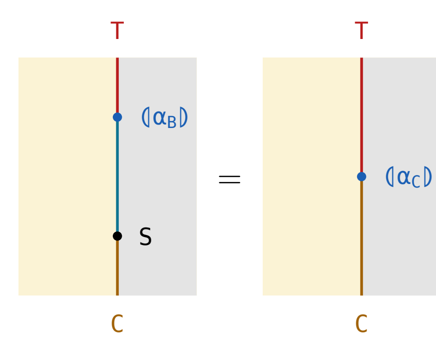
(11.4.2a)

$$\begin{tikzcd}
\mathsf{FT} \arrow[r, "\alpha_{\mathsf{T}}"] \arrow[d, "\mathsf{F}((\!|\alpha_{\mathsf{B}}|\!))"'] & \mathsf{T} \arrow[d, dashed, "(\!|\alpha_{\mathsf{B}}|\!)"] \\
\mathsf{FB} \arrow[r, "\alpha_{\mathsf{B}}"] \arrow[d, "\mathsf{F(S)}"'] & \mathsf{B} \arrow[d, "\mathsf{S}"] \\
\mathsf{FC} \arrow[r, "\alpha_{\mathsf{C}}"] & \mathsf{C}
\end{tikzcd}$$

the side condition



the conclusion



$(\!|\alpha_{\mathsf{B}}|\!)\mathsf{S} = (\!|\alpha_{\mathsf{C}}|\!) \Longleftarrow \alpha_{\mathsf{B}}\ \mathsf{S} = \mathsf{F(S)}\alpha_{\mathsf{C}}$ (2.12)

§11.5. Type relator

(11.5a)

Let F be a binary relator with initial type (α,T), so T is a type functor. $F(R,S)$ is its action on a pair, and $F(X)$ abbreviates $F(\mathbb{1},X)$, the F of the reduce section. For every object A the initial algebra is $\alpha : F(A,TA) \to TA$, among the maps. T acts on an arrow $R : A \to B$ by

$$T(R)=(\!|F(R,\mathbb{1})\alpha|\!) : TA \to TB$$

(11.5b)

name	law	what it says
the defining equation	$T(R)=(\!\|F(R,\mathbb{1})\alpha\|\!)$	Rebuild the structure with α, applying R to the parameter on the way.
functor	$T(\mathbb{1})=\mathbb{1}$ and $T(R)T(S)=T(RS)$	Acting by the identity changes nothing, and two actions in a row are one action.
type functor fusion	$T(R)(\!\|Q\|\!)=(\!\|F(R,\mathbb{1})Q\|\!)$	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because F is a bifunctor — $F(R,\mathbb{1})F(\mathbb{1},(\!\|Q\|\!))=F(R,(\!\|Q\|\!))=F(\mathbb{1},(\!\|Q\|\!))F(R,\mathbb{1})$.
naturality of α	$\alpha T(R)=F(R,T(R))\alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R)=F(R,T(R))$ to T.
type relator	$T(R)°=T(R°)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

§11.5.1. Type functor

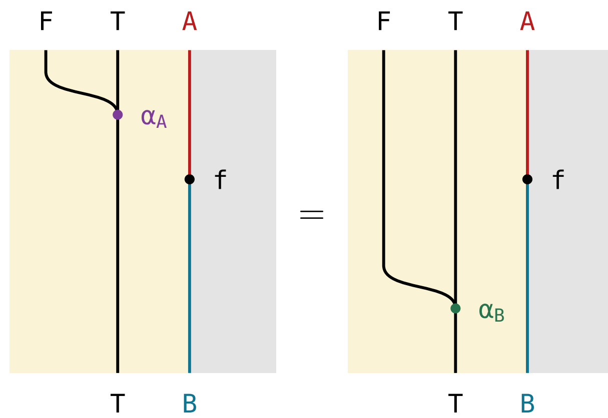
(11.5.1a)

Let $\mathtt{F}$ be a bifunctor taking both the parameter $\mathtt{A}$ and the recursive position $\mathtt{TA}$, with an initial algebra $\alpha_\mathtt{A} : \mathtt{F(A,TA)}{\to}\mathtt{TA}$ for every object $\mathtt{A}$. Then $\mathtt{T}$ is a functor, acting on a map $\mathtt{f : A{\to}B}$ by

$$\mathtt{T(f)} \triangleq \llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)}\alpha_\mathtt{B} \rrparenthesis : \mathtt{TA}{\to}\mathtt{TB}$$

(11.5.1b)

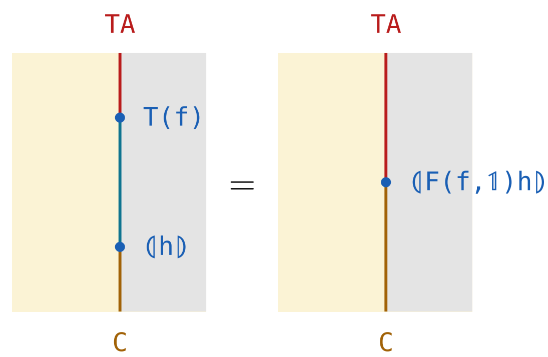
$$\begin{tikzcd}
\mathtt{F(A,TA)} \arrow[r, "\alpha_\mathtt{A}"] \arrow[d, "\mathtt{F(}\mathbb{1}\mathtt{,T(f))}"'] & \mathtt{TA} \arrow[d, dashed, "\mathtt{T(f)}"] \\
\mathtt{F(A,TB)} \arrow[r, "\mathtt{F(f,}\mathbb{1}\mathtt{)}\alpha_\mathtt{B}"] \arrow[d, "\mathtt{F(f,}\mathbb{1}\mathtt{)}"'] & \mathtt{TB} \\
\mathtt{F(B,TB)} \arrow[ur, "\alpha_\mathtt{B}"'] &
\end{tikzcd}$$



$$\alpha_\mathtt{A}\ \mathtt{T(f)}=\mathtt{F(f,T(f))}\alpha_\mathtt{B}$$

(11.5.1c)

$$\begin{tikzcd}
\mathtt{F(A,TA)} \arrow[r, "\alpha_\mathtt{A}"] \arrow[d, "\mathtt{F(}\mathbb{1}\mathtt{,T(f)}\llparenthesis \mathtt{h} \rrparenthesis\mathtt{)}"'] & \mathtt{TA} \arrow[d, dashed, "\llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)h} \rrparenthesis"'] \arrow[dr, dashed, "\mathtt{T(f)}"] & \\
 & & \mathtt{TB} \arrow[dl, dashed, "\llparenthesis \mathtt{h} \rrparenthesis"] \\
\mathtt{F(A,C)} \arrow[r, "\mathtt{F(f,}\mathbb{1}\mathtt{)h}"'] & \mathtt{C} &
\end{tikzcd}$$



$$\mathtt{T(f)}\llparenthesis \mathtt{h} \rrparenthesis=\llparenthesis \mathtt{F(f,}\mathbb{1}\mathtt{)h} \rrparenthesis$$

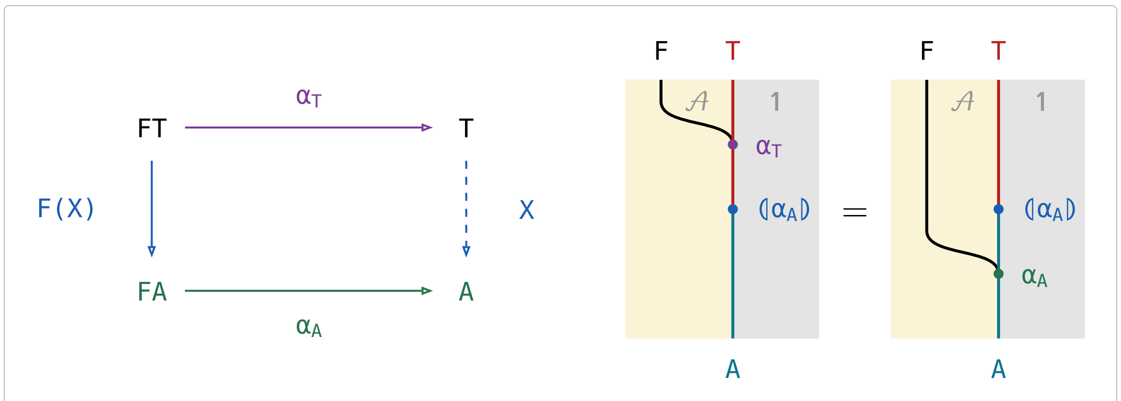
§11.6. Reduce

(11.6a)

> let F be a relator and has **initial algebra** $\alpha_T\ :\ FT \to T$ in the subcategory of functions. α_T is also initial in the allegory:

§11.6.1. The defining equation

(11.6.1a)



$X=(\!|\alpha_A|\!)\iff\alpha_T\ X=F(X)\alpha_A$

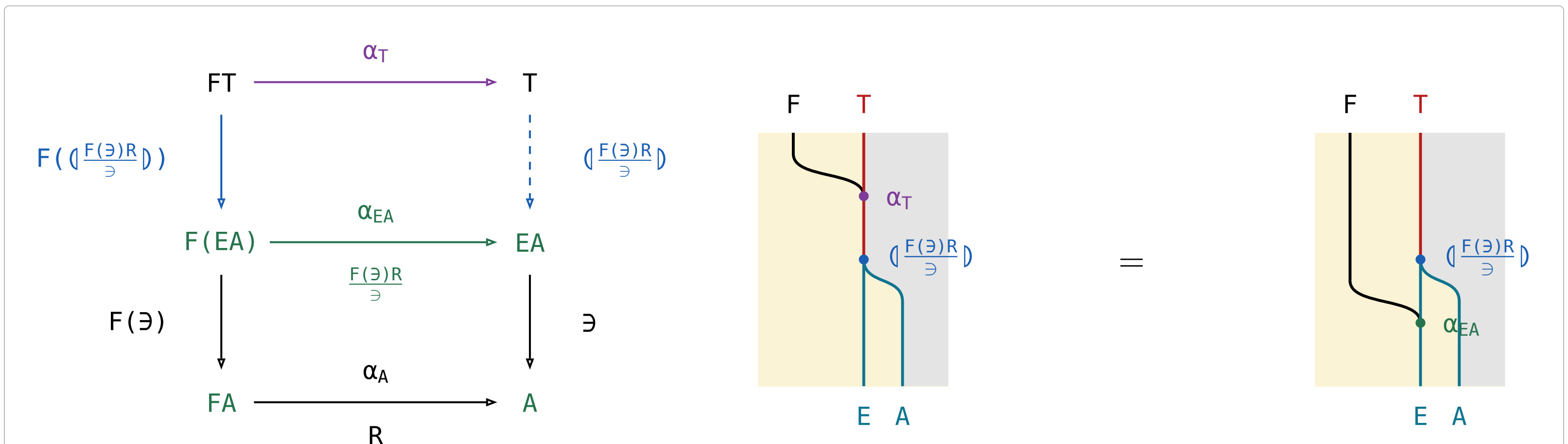
An F-algebra is a **weakened** natural transformation. A transformation $F\Rightarrow 1$ on $\mathcal{A}$ would need a component $FX\to X$ at every object and a commuting square at every arrow, but F-Algebra only need it works on T and A.

(11.6.1b)

⦇[c,f]⦈=reduce(c,f)		
part	**Nat**	**[A]**
datatype	`Nat::=zero|succ Nat`	`[A]::=nil|cons (A,[A])`
base functor F	`F(X)=1+X`	`F(X)=1+A×X`
initial algebra α	`α=[zero,succ]` `: 1+Nat→Nat`	`α=[nil,cons]` `: 1+A×[A]→[A]`
the fold, pointwise	`⦇[c,f]⦈ zero=c` `⦇[c,f]⦈ (succ n)=f (⦇[c,f]⦈n)`	`⦇[c,f]⦈ nil=c` `⦇[c,f]⦈ (cons (a,x))=f (a,⦇[c,f]⦈ x)`

§11.6.2. $(\!|R|\!)=(\!|\frac{F(\ni)R}{\ni}|\!)\ni$

(11.6.2a)



$\frac{1}{\ni}$ is the inverse of $\ni$

$\alpha_T\ (\!|\frac{F(\ni)R}{\ni}|\!)=F((\!|\frac{F(\ni)R}{\ni}|\!))\ \frac{F(\ni)R}{\ni}$

(11.6.2b)

$$\boxed{\frac{\alpha_T\ X}{F(X)R}} \underset{\cdot\ni\dashv\frac{\ }{\ni}}{\Longleftrightarrow} \boxed{\frac{\frac{\alpha_T\ X}{\ni}}{\frac{F(X)R}{\ni}}} \underset{\cdot\ni\dashv\frac{\ }{\ni}}{\Longleftrightarrow} \boxed{\frac{\frac{\alpha_T\ X}{\ni}}{\frac{F(\frac{X}{\ni}\ni)R}{\ni}}}$$

$$\underset{\text{relator, fusion twice}}{\Longleftrightarrow} \boxed{\frac{\alpha_T\ \frac{X}{\ni}}{F(\frac{X}{\ni})\ \frac{F(\ni)R}{\ni}}} \underset{\text{reduce of maps}}{\Longleftrightarrow} \boxed{\frac{\frac{X}{\ni}}{(\!|\frac{F(\ni)R}{\ni}|\!)}} \underset{\cdot\ni\dashv\frac{\ }{\ni}}{\Longleftrightarrow} \boxed{\frac{X}{(\!|\frac{F(\ni)R}{\ni}|\!)\ni}}$$

§12. Combinatorial functions

(12a)

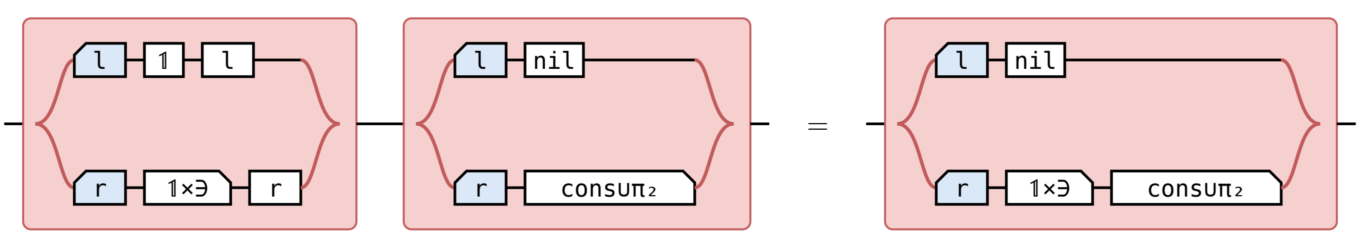
definition	type	note
$\mathtt{[A]::=nil\,|\,cons\ (A,[A])}$	$\mathcal{A}\to\mathcal{A}$	B&dM's `listr` under the short name it keeps from p. 125 on.
$\mathtt{list(R)}\triangleq(\!\lbrack\mathtt{[nil,(R\otimes 1)\ cons]}\rbrack\!)$	$\mathtt{[A]\to[B]}$	The relator's action on $\mathtt{R}\ :\ \mathtt{A\to B}$: one `R` per element, the shape untouched.
$\mathtt{subseq}\triangleq(\!\lbrack\mathtt{[nil,cons\cup\pi_2]}\rbrack\!)$	$\mathtt{[A]\to[A]}$	`xs subseq ys`: `ys` is `xs` with elements dropped — `cons` keeps the head, π_2 drops it.
$\mathtt{prefix}\triangleq(\!\lbrack\mathtt{[nil,nil\cup cons]}\rbrack\!)$ $\mathtt{=cat^\circ\ \pi_1=init^*}$	$\mathtt{[A]\to[A]}$	`ys` is an initial segment of `xs`; the first `nil` is where it stops early. $\mathtt{init}\triangleq\mathtt{snoc^\circ\ \pi_1}$.
$\mathtt{suffix}\triangleq\mathtt{cat^\circ\ \pi_2=tail^*}$	$\mathtt{[A]\to[A]}$	The dual, $\mathtt{tail}\triangleq\mathtt{cons^\circ\ \pi_2}$; as a reduce it needs snoc-lists.
$\mathtt{segment}\triangleq\mathtt{suffix\ prefix}$	$\mathtt{[A]\to[A]}$	A contiguous stretch of `xs`: a suffix, then a prefix of that.
$\mathtt{partition}\triangleq\mathtt{concat^\circ}$	$\mathtt{[A]\to[[A]]}$	`cat` restricted to a non-empty first argument, so `ys` is a list of non-empty segments of `xs`.
$\mathtt{concat}\triangleq(\!\lbrack\mathtt{[nil,cat]}\rbrack\!)$	$\mathtt{[[A]]\to[A]}$	Joins the segments back up, which is why its converse splits a list.
`inits`	$\mathtt{[A]\to[[A]]}$	Implements $\frac{\mathtt{prefix}}{\ni}$, listing the prefixes by increasing length.
`tails`	$\mathtt{[A]\to[[A]]}$	Implements $\frac{\mathtt{suffix}}{\ni}$ by decreasing length — the opposite order.
$\mathtt{filter(p)}\triangleq\frac{\mathtt{subseq\ list(p)}}{\ni}\ \mathtt{max(R)}$	$\mathtt{[A]\to[A]}$	The longest subsequence of `xs` whose every element passes `p`. Ex 7.41; `max(R)` is (13.1a)
$\mathtt{R}\triangleq\mathtt{length\le length^\circ}$	$\mathtt{[A]\to[A]}$	The preorder `filter` and `takewhile` maximise over: the longer list wins.
$\mathtt{takewhile(p)}\triangleq\frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{max(R)}$	$\mathtt{[A]\to[A]}$	The same with `prefix` for `subseq`: the longest prefix whose every element passes `p`. Ex 7.39
$\mathtt{mss}\triangleq\frac{\mathtt{segment\ sum}}{\ni}\ \mathtt{max(\le)}$	$\mathtt{[Int]\to Int}$	Maximum segment sum. `segment=suffix prefix` splits it into $\frac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{max(\le)}$ on each suffix. Ex 7.40

§12.1. $\frac{\mathtt{subseq}}{\ni}\ \mathtt{=}(\!\lbrack\mathtt{[nil}\ \frac{\mathbb{1}}{\ni}\mathtt{,}\langle\frac{\mathbb{1}\times\ni}{\ni}\ \mathtt{E(cons),\pi_2}\rangle\ \mathtt{cup]}\rbrack\!)$

$\mathtt{cup}\triangleq\frac{\pi_1\ni\cup\pi_2\ni}{\ni}\ :\ \mathtt{EA\times EA\to EA}$, so $\frac{\mathtt{R\cup S}}{\ni}=\langle\frac{\mathtt{R}}{\ni},\frac{\mathtt{S}}{\ni}\rangle\ \mathtt{cup}$. (12.1a)

$$\boxed{\frac{\mathtt{F(\ni)\ [nil,cons\cup\pi_2]}}{\ni}}\ \overset{}{\underset{\mathtt{FX=1+A\times X}}{=\!=}}\ \boxed{\frac{\mathtt{(\mathbb{1}+\mathbb{1}\times\ni)\ [nil,cons\cup\pi_2]}}{\ni}}\ \underset{\text{coproduct}}{=\!=}\ \boxed{\frac{\mathtt{[nil,(\mathbb{1}\times\ni)(cons\cup\pi_2)]}}{\ni}}$$

$$\underset{\text{coproduct of maps}}{=\!=}\ \boxed{\mathtt{[}\frac{\mathtt{nil}}{\ni}\mathtt{,}\ \frac{\mathtt{(\mathbb{1}\times\ni)(cons\cup\pi_2)}}{\ni}\mathtt{]}}\ \underset{\text{singleton}}{=\!=}\ \boxed{\mathtt{[nil}\ \frac{\mathbb{1}}{\ni}\mathtt{,}\ \frac{\mathtt{(\mathbb{1}\times\ni)(cons\cup\pi_2)}}{\ni}\mathtt{]}}$$ (12.1b)



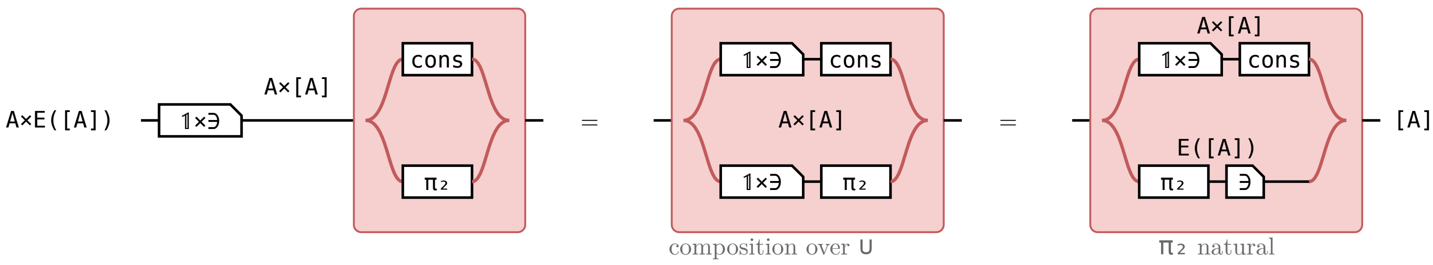
(12.1c)

B&dM p. 124, "coproduct": $\mathtt{R+S}\triangleq\mathtt{[Rl,Sr]}$ and $\mathtt{l[R,S]=R}$, $\mathtt{r[R,S]=S}$ — (11.2a).

$$\boxed{\frac{(\mathbb{1}\times\ni)(\mathtt{cons}\cup\pi_2)}{\ni}} \;\underset{\cup\text{ composes, }\pi_2\text{ natural}}{=\!=\!=}\; \boxed{\frac{(\mathbb{1}\times\ni)\ \mathtt{cons}\cup\pi_2\ni}{\ni}} \;\underset{\mathtt{cup}}{=\!=}\; \boxed{\langle\frac{(\mathbb{1}\times\ni)\ \mathtt{cons}}{\ni},\frac{\pi_2\ni}{\ni}\rangle\ \mathtt{cup}} \tag{12.1d}$$

$$\underset{\text{absorption, fusion}}{=\!=\!=}\; \boxed{\langle\frac{\mathbb{1}\times\ni}{\ni}\ E(\mathtt{cons}),\pi_2\rangle\ \mathtt{cup}}$$

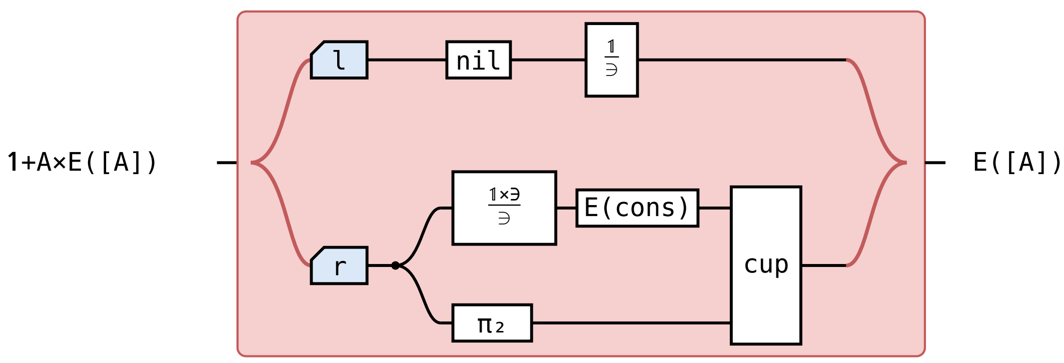
(12.1e)



B&dM p. 124, "composition over join, naturality of π_2": $T(X_1\cup X_2)=TX_1\cup TX_2$ — (1.2b); $(\mathbb{1}\times\ni)\pi_2=\pi_2\ni$ — (11.1.1b) at π_2, an equality because $\mathbb{1}$ is entire.

$$\begin{tikzcd}
A\times E([A]) \arrow[r, "\pi_2"] \arrow[d, "\mathbb{1}\times\ni"'] & E([A]) \arrow[d, "\ni"] \\
A\times[A] \arrow[r, "\pi_2"'] & [A]
\end{tikzcd} \tag{12.1f}$$

(12.1g)



B&dM §5.6, p. 124, which writes $P\mathtt{cons}$; $\mathtt{cons}$ is a map, and there $P(\mathtt{cons})=E(\mathtt{cons})$ — (11.3d).

§13. Optimisation Problems

§13.1. Minimum and maximum

For `R : A→A`, `min(R)≜∋∩(∈\R°) : EA→A`, `max(R)≜min(R°)`. (13.1a)

`xs (min(R)) x⟺x∈xs∧(∀y∈xs. x R y)`

`xs (min(R)) x⟺(x in xs) and all x R\: xs` in q

`min(R)=∋∩all R°` `all R≜∈\R`, q's `all`; the chains below keep it written `∈\`

(13.1b)

the law	what it says
`X⊑min(R)⟺X⊑∋` and `X°∋⊑R`	in the set, and below every element of it
$\frac{1}{\ni}$ `(∈\R)=R`	bounding a singleton is bounding its element
$\frac{S}{\ni}$ `(∈\R)=S°\R`	bound `S`'s image without building the set
`union≜` $\frac{\ni\ni}{\ni}$ `: E(EA)→EA`	flattens a set of sets
`union (∈\R)=∈\(∈\R)`	bound a union by bounding each member set
$\frac{1}{\ni}$ `min(R)=𝟙∩R`	a singleton's minimum is its element, where `R` is reflexive $\frac{S}{\ni}$ `min(R)` at `S:=𝟙`
$\frac{S}{\ni}$ `min(R)=S∩(S°\R°)`	an `S`-value that points to every `S`-value
$\frac{S}{\ni}$ `min(R)=` $\frac{S}{\ni}$ `min(R∩S°S)`	only `R` between values `S` gives one argument counts — context
`E(S) min(R)=(∋S)∩((∋S)°\R°)`	the same for the image of a set $\frac{S}{\ni}$ `min(R)` at `S:=∋S`
`P(f) min(R)=min(fRf°) f`	shunt a function through a minimum
`P(S) min(R)=(∋S)∩(∈\(SR°))` `R` reflexive	fusion with the power relator ⊒ is the only proof here that tabulates
`P(S) min(R)⊑(∋S)∩(∈\(SR°))`	the half of the row above that costs nothing
`P(min(R)) min(R)⊑union min(R)` `R` a preorder	a minimum in each set, then a minimum of those
`P(min(R)) min(R)=P(Dom(min(R))) union min(R)` `R` a preorder	the same as an equality, once empty sets are dropped

§13.1.1. `X⊑min(R)⟺X⊑∋` and `X°∋⊑R`

(13.1.1a)

$$\frac{X}{\mathrm{min(R)}} \underset{\Delta\dashv\cap}{\Longleftrightarrow} \begin{array}{c}\frac{X}{\ni}\\[4pt] \frac{X}{\in\backslash R^\circ}\end{array} \underset{T\cdot\dashv T\backslash}{\Longleftrightarrow} \begin{array}{c}\frac{X}{\ni}\\[4pt] \frac{\in X}{R^\circ}\end{array} \underset{\circ}{\Leftrightarrow} \begin{array}{c}\frac{X}{\ni}\\[4pt] \frac{X^\circ\ni}{R}\end{array}$$

§13.1.2. $\frac{1}{\ni}$ `(∈\R)=R`

(13.1.2a)

$$\frac{X}{\frac{1}{\ni}\,(\in\backslash R)} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \frac{\frac{1}{\ni}{}^\circ X}{\in\backslash R} \underset{T\cdot\dashv T\backslash}{\Longleftrightarrow} \frac{\in\frac{1}{\ni}{}^\circ X}{R} \underset{\circ}{\Leftrightarrow} \frac{(\frac{1}{\ni}\ \ni)^\circ X}{R} \underset{i\dashv E}{\Longleftrightarrow} \frac{X}{R}$$

§13.1.3. $\frac{S}{\ni}$ `(∈\R)=S°\R`

(13.1.3a)

$$\frac{X}{\frac{S}{\ni}\,(\in\backslash R)} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \frac{\frac{S}{\ni}{}^\circ X}{\in\backslash R} \underset{T\cdot\dashv T\backslash}{\Longleftrightarrow} \frac{\in\frac{S}{\ni}{}^\circ X}{R} \underset{\circ}{\Leftrightarrow} \frac{(\frac{S}{\ni}\ \ni)^\circ X}{R} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \frac{S^\circ X}{R} \underset{T\cdot\dashv T\backslash}{\Longleftrightarrow} \frac{X}{S^\circ\backslash R}$$

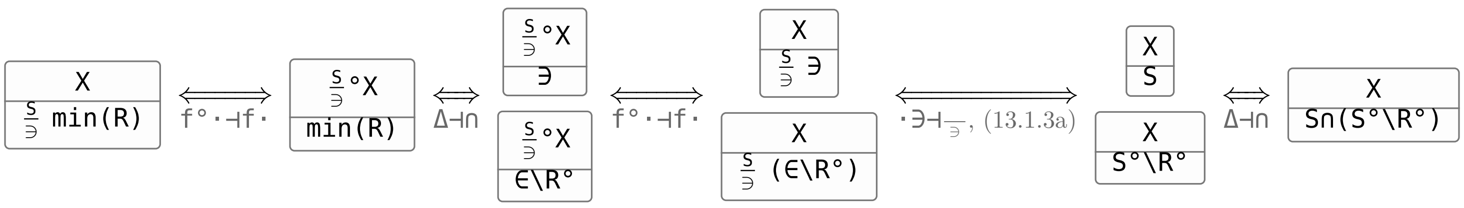
§13.1.4. `union (∈\R)=∈\(∈\R)`

(13.1.4a)

$$\frac{X}{\mathrm{union}\ (\in\backslash R)} \underset{f^\circ\cdot\dashv f\cdot}{\Longleftrightarrow} \frac{\mathrm{union}^\circ\ X}{\in\backslash R} \underset{T\cdot\dashv T\backslash}{\Longleftrightarrow} \frac{\in\mathrm{union}^\circ\ X}{R} \underset{\circ}{\Leftrightarrow} \frac{(\mathrm{union}\ni)^\circ X}{R} \underset{\cdot\ni\dashv\frac{}{\ni}}{\Longleftrightarrow} \frac{(\ni\ni)^\circ X}{R} \underset{\circ,\ T\cdot\dashv T\backslash}{\Longleftrightarrow} \frac{X}{\in\backslash(\in\backslash R)}$$

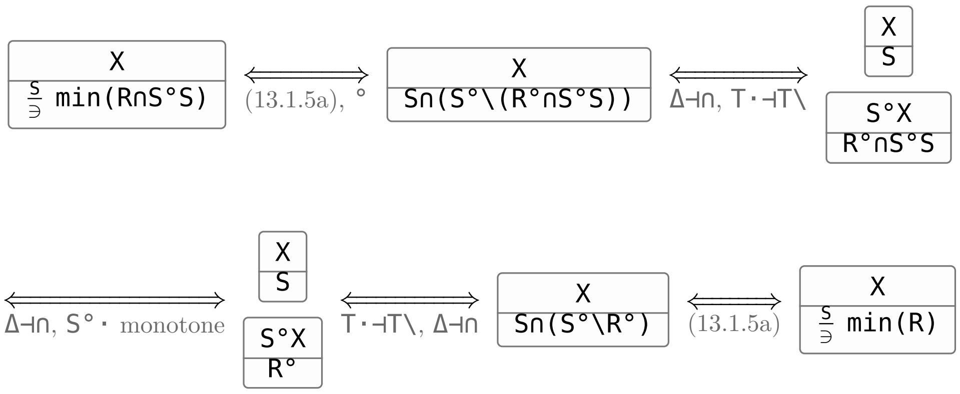
§13.1.5. $\frac{\mathtt{S}}{\ni}$ min(R)=S∩(S°\R°)

(13.1.5a)



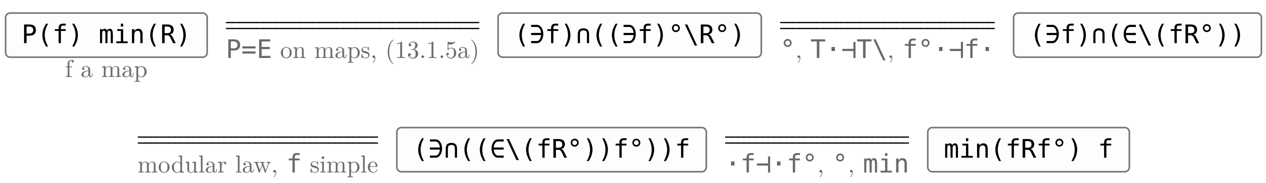
§13.1.6. $\frac{\mathtt{S}}{\ni}$ min(R)= $\frac{\mathtt{S}}{\ni}$ min(R∩S°S)

(13.1.6a)



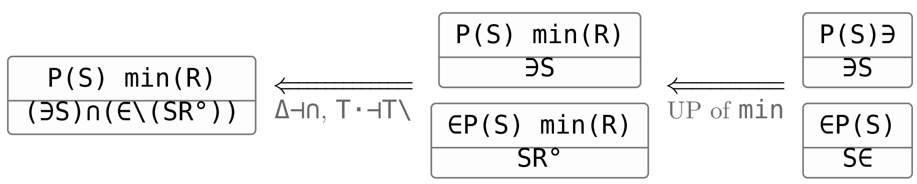
§13.1.7. P(f) min(R)=min(fRf°) f

(13.1.7a)



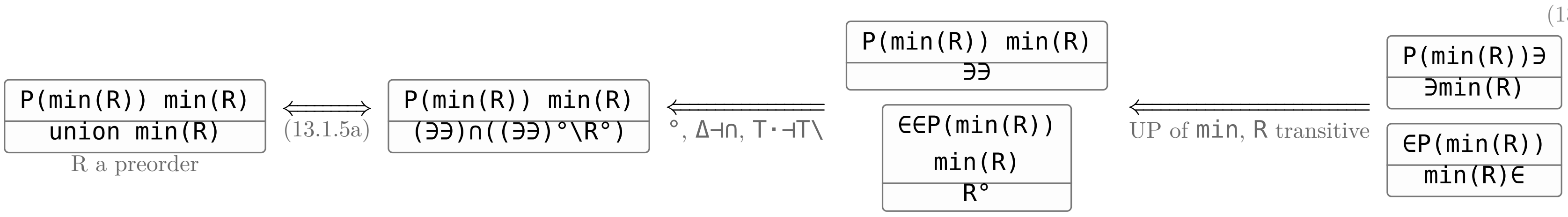
§13.1.8. P(S) min(R)⊑(∃S)∩(∈\(SR°))

(13.1.8a)



§13.1.9. P(min(R)) min(R)⊑union min(R)

(13.1.9a)



§13.2. Lax natural transformations (LaT)

(13.2a)

For relators $G,F : \mathcal{C}\to\mathcal{D}$ and components $\varphi_A : GA\to FA$, φ is **lax at** $R : A\to B$ when $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ — both sides $GA\to FB$, the left through the component at B, the right through the one at A.

$\varphi : G\Rightarrow F$ is a **lax natural transformation** (LaT) when it is lax at **every** R. (5.13)

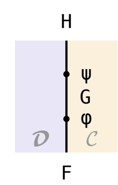
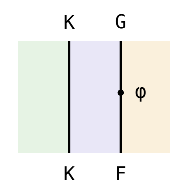
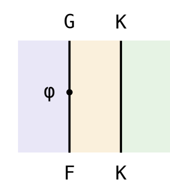
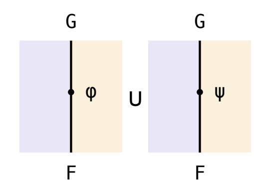
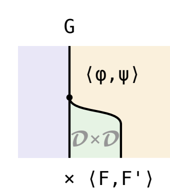
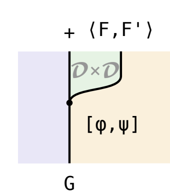
Lax at every **map** already gives LaT, and at a map the inequation is an equality $G(f)\varphi = \varphi F(f)$: laxness is about relations only. Theorem 5.2

(13.2b)

$$\begin{tikzcd} GA \arrow[r, "\varphi_A"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, "F(R)"] \\ GB \arrow[r, "\varphi_B"'] & FB \end{tikzcd}$$

$G(R)\varphi_B \sqsubseteq \varphi_A F(R)$

(13.2c)

closed under	commutative diagram	string diagram
$\psi : H\Rightarrow G$ and $\varphi : G\Rightarrow F$ give $\psi\varphi : H\Rightarrow F$	$\begin{tikzcd} HA \arrow[r, "\psi_A"] \arrow[d, "H(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & GA \arrow[r, "\varphi_A"] \arrow[d, "G(R)"] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, "F(R)"] \\ HB \arrow[r, "\psi_B"'] & GB \arrow[r, "\varphi_B"'] & FB \end{tikzcd}$	
$\varphi : G\Rightarrow F$ and K a relator give $K(\varphi) : K\circ G\Rightarrow K\circ F$, K composed on the outside	$\begin{tikzcd} K(GA) \arrow[r, dash, "K(\varphi_A)"] \arrow[d, "K(G(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & K(FA) \arrow[d, "K(F(R))"] \\ K(GB) \arrow[r, dash, "K(\varphi_B)"'] & K(FB) \end{tikzcd}$	
$\varphi : G\Rightarrow F$ gives $\varphi K : G\circ K\Rightarrow F\circ K$, K composed on the inside	$\begin{tikzcd} G(KA) \arrow[r, dash, "\varphi_{KA}"] \arrow[d, "G(K(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & F(KA) \arrow[d, "F(K(R))"] \\ G(KB) \arrow[r, dash, "\varphi_{KB}"'] & F(KB) \end{tikzcd}$	
$\varphi,\psi : G\Rightarrow F$ give $\varphi\cup\psi : G\Rightarrow F$	$\begin{tikzcd} GA \arrow[r, "\varphi_A"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, "F(R)"] \\ GB \arrow[r, "\varphi_B"'] & FB \end{tikzcd} \cup \begin{tikzcd} GA \arrow[r, "\psi_A"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA \arrow[d, "F(R)"] \\ GB \arrow[r, "\psi_B"'] & FB \end{tikzcd}$	
$\varphi : G\Rightarrow F$ and $\psi : G\Rightarrow F'$ give $\langle\varphi,\psi\rangle : G\Rightarrow F\times F'$ $R\lhd\sqsubseteq\lhd(R\times R)$ — (2b) — opens $G(R)\langle\varphi_B,\psi_B\rangle$; $\langle X,Y\rangle(R\times S)=\langle XR,YS\rangle$ closes it — 2 of (11.1.2b)	$\begin{tikzcd} GA \arrow[r, dash, "\langle\varphi,\psi\rangle_A"] \arrow[d, "G(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & FA\times F'A \arrow[d, "F(R)\times F'(R)"] \\ GB \arrow[r, dash, "\langle\varphi,\psi\rangle_B"'] & FB\times F'B \end{tikzcd}$	
$\varphi : F\Rightarrow G$ and $\psi : F'\Rightarrow G$ give $[\varphi,\psi] : F+F'\Rightarrow G$	$\begin{tikzcd} FA+F'A \arrow[r, "{[\varphi,\psi]_A}"] \arrow[d, "F(R)+F'(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & GA \arrow[d, "G(R)"] \\ FB+F'B \arrow[r, "{[\varphi,\psi]_B}"'] & GB \end{tikzcd}$	
$\varphi\cap\psi : G\Rightarrow F$ fails the step it would need is $(\varphi_A F(R))\cap(\psi_A F(R))\sqsubseteq(\varphi\cap\psi)_A F(R)$, the wrong direction of (4c)		

§13.3. Monotonic algebras

An F-algebra $\varphi : FA \to A$ is **monotonic on** $R : A \to A$ when it is lax at R at $G{:=}F$, $F{:=}Id$: $F(R)\varphi \sqsubseteq \varphi R$. R is an **endorelation**, so $B{=}A$ and the two components φ_A, φ_B are the one arrow φ — an algebra, not a family. (13.3a)

For a map $f : FA \to A$ that is $f^\circ F(R) f \sqsubseteq R$, equivalently $F(R) \sqsubseteq f R f^\circ$.

$(\leq \times \leq){+} \sqsubseteq {+} \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c{=}a{+}b \wedge a \leq a' \wedge b \leq b' \Longrightarrow c \leq a'{+}b'$.

(13.3b)

$$\begin{tikzcd}
FA \arrow[r, "\varphi"] \arrow[d, "F(R)"'] \arrow[dr, phantom, "\sqsubseteq"] & A \arrow[d, "R"] \\
FA \arrow[r, "\varphi"'] & A
\end{tikzcd}$$

$F(R)\varphi \sqsubseteq \varphi R$

$f : FA \to A$ **distributes over** R if $F(min(R))f \sqsubseteq \frac{F(\ni)f}{\ni}\, min(R)$. (13.3c)

${+}$ distributes over $\leq$, at the point level $min\ xs{+}min\ ys{=}min\{x{+}y \mid x{\in}xs \wedge y{\in}ys\}$ for xs, ys non-empty and $min{\triangleq}min(\leq)$.

(13.3d)

$$\begin{tikzcd}
F(EA) \arrow[r, "\frac{F(\ni)f}{\ni}"] \arrow[d, "F(min(R))"'] \arrow[dr, phantom, "\sqsubseteq"] & EA \arrow[d, "min(R)"] \\
FA \arrow[r, "f"'] & A
\end{tikzcd}$$

$F(min(R))f \sqsubseteq \frac{F(\ni)f}{\ni}\ min(R)$

$$\begin{tikzcd}
\underset{(xs,ys)}{E\ Nat \times E\ Nat} \arrow[r, "\frac{(\ni\times\ni)+}{\ni}"] \arrow[d, "min(\leq)\times min(\leq)"'] \arrow[dr, phantom, "\sqsubseteq"] & \underset{\{x+y \mid x\in xs \wedge y \in ys\}}{E\ Nat} \arrow[d, "min(\leq)"] \\
\underset{(min\ xs,min\ ys)}{Nat \times Nat} \arrow[r, "+"'] & \underset{min\ xs+min\ ys}{Nat}
\end{tikzcd}$$

$(min(\leq)\times min(\leq)){+} \sqsubseteq \frac{(\ni\times\ni)+}{\ni}\ min(\leq)$

(13.3e)

the law	what it says
$f^\circ F(R) f \sqsubseteq R \Longleftrightarrow f^\circ F(R^\circ) f \sqsubseteq R^\circ$	a **map** is monotonic on an order and on its opposite together — a relation is not
$f^\circ F(R) f \sqsubseteq R \Longleftrightarrow F(min(R))f \sqsubseteq \frac{F(\ni)f}{\ni}\ min(R)$ Theorem 7.1, f a map	for a map the two definitions above are one condition
$\llparenthesis \frac{S}{\ni}\ min(R) \rrparenthesis \sqsubseteq \frac{\llparenthesis S \rrparenthesis}{\ni}\ min(R)$ Theorem 7.2, R a preorder and $F(R^\circ)S \sqsubseteq SR^\circ$	the **greedy theorem**: keeping one minimum at every step beats no more than taking the minimum at the end

§13.3.1. Ordered objects and monotone relations

An **ordered object** (A,T) is an object A of the allegory with an endorelation $T : A \to A$; T is asked to be neither reflexive nor transitive. 3.3.1a)

A **monotone relation** $(A,T_A) \to (B,T_B)$ is a relation $\varphi : A \to B$ with $T_A \varphi \sqsubseteq \varphi T_B$.

That inequation is (13.2a)'s at $G{:=}F{:=}Id$, and (13.3a)'s algebra φ is the case $T_A{:=}F(R)$, $T_B{:=}R$.

(13.3.1b)

it has	**why**
identity and composition — so it IS a category	`T𝟙=T=𝟙T`; composition is the sliding rule, (13.2c)'s first row at `G=F=Id`
hom-sets ordered by ⊑, pointwise, and composition monotone in both arguments — **enriched over posets**	inherited from the allegory: ⊑ is reflexive, transitive and antisymmetric there, and `R⊑R'` gives `RS⊑R'S`
binary union `φ∪ψ`	(13.2c)'s union row at `G=F=Id`
a tensor $(A,T_A)\times(B,T_B)$, **not** a categorical product	the projections are monotone and so is `⟨φ,ψ⟩`, but `⟨φ,ψ⟩` π_1`=Dom(ψ)φ⊑φ` — (11.1b) — with equality only when `ψ` is entire, the same reason `R×S` is not a categorical product in `Rel` — (11.1.1a)
no meet `φ∩ψ`	the counterexample (13.3.1c)
no converse `φ°`	° of $T_A\varphi\sqsubseteq\varphi T_B$ is $\varphi^\circ T_A{}^\circ\sqsubseteq T_B{}^\circ\varphi^\circ$ — the orders are converted AND the inclusion points the other way, so `φ°` lands in the op-lax category, not this one
a zero object — the empty object is both terminal and initial	every relation into it is the empty one, so there is exactly one arrow
binary products, but they are ⊔ and not the × this chapter uses	a relation into a disjoint union splits into its two halves, `Hom(C,A⊔B)≅Hom(C,A)×Hom(C,B)`, and the projections are the converses of the injections
no equalizers, hence not regular	the counterexample (13.3.1d)

$$A=\{0,1\} \qquad T_A=\{(0,1)\}$$
$$B=\{p,q,r\} \qquad T_B=\{(p,r),(q,r)\}$$
$$\varphi=\{(0,p),(1,r)\} \qquad \psi=\{(0,q),(1,r)\} \tag{13.3.1c}$$

`φ` monotone	$0T_A1$ and `1φr`, witnessed by `p`: `0φp` and pT_Br
`ψ` monotone	the same step through `q`: `0ψq` and qT_Br
`φ∩ψ={(1,r)}` not	that step wants a `b` with `0(φ∩ψ)b`, and `(φ∩ψ)[0]=∅`
the moral	⊑ asserts a witness **exists**; the two sides have witnesses but different ones, `p` and `q`, and the meet keeps neither — which is also why (4c) runs one way only

$$A=\{0,1\} \qquad R=\top=\{(0,*),(1,*)\}$$
$$B=\{*\} \qquad S=\{(0,*)\} \tag{13.3.1d}$$

the order on `A` and `B`	**empty**, and every relation is monotone for it — so the count below is a count in this category, not only in `Rel`
`e : C⇸A` equalizes `R` and `S`, at a one-element `C`	`eR=eS`, which comes to `xe1⟹xe0` for every `x`
exactly three such `e`	`∅`, `{0}`, `{0,1}` — `{1}` is the one subset the condition rules out
so no equalizer	the equalizer would have exactly three arrows from `C`, and `Hom(C,X)` is the power set of `X` — no power set has three elements

§13.3.2. `takewhile(p)=⦇[nil,(π₁p→cons,⊸ nil)]⦈`

(13.3.2a)

`FX=1+A×X`, `α≙[nil,cons]`, `p` a coreflexive on `A`, `R≙length≤length°`, a preorder, `S≙[nil,⊸ nil∪(p×𝟙) cons] : F([A])→[A]`.

`⊸ nil` is the constant `nil`, the second `nil` of `prefix=⦇[nil,⊸ nil∪cons]⦈`; `(g→X,Y)` is `X` where `g` is defined and `Y` where it is not.

`nil R=⊤` `nil` is the shortest list, so it loses every `max(R)`

$$\boxed{\texttt{α prefix list(p)}} \;\overset{}{\underset{\text{defining equation}}{=\!=\!=}}\; \boxed{\texttt{F(prefix) [nil,⊸ nil∪cons] list(p)}} \tag{13.3.2b}$$

$$\overset{\text{list(p) through cons}}{=}\ \mathtt{F(prefix)}\ [\mathtt{nil}, \multimap \mathtt{nilu}(\mathtt{p}\times\mathtt{list(p)})\ \mathtt{cons}]$$

$$\overset{\text{relator, prefix entire}}{=}\ [\mathtt{nil}, \multimap \mathtt{nilu}(\mathtt{p}\times(\mathtt{prefix\ list(p)}))\ \mathtt{cons}]\ \overset{\text{prefix list(p) entire}}{=}\ \mathtt{F(prefix\ list(p))S}$$

(11.6.1a) reads that off as $\mathtt{prefix\ list(p)}=\llparenthesis \mathtt{S}\rrparenthesis$. (11.4.2a) cannot: $\mathtt{list(p)}$ is not entire, $(\mathbb{1}\times\mathtt{list(p)})\multimap \mathtt{nil}\sqsubset\multimap \mathtt{nil}$, and no algebra meets the side condition.

$$(\mathbb{1}\times R)(\multimap \mathtt{nilu}(\mathtt{p}\times\mathbb{1})\ \mathtt{cons})\ \overset{\text{relator, composition over }\cup}{=}\ (\mathbb{1}\times R)\multimap \mathtt{nilu}(\mathtt{p}\times R)\ \mathtt{cons}\ \overset{(5.1\text{a}),\ (\mathbb{1}\times R)\ \mathtt{cons}\sqsubseteq\mathtt{cons}\ R}{\sqsubseteq}\ \multimap \mathtt{nilu}(\mathtt{p}\times\mathbb{1})\ \mathtt{cons}\ R\ \overset{R\text{ reflexive}}{\sqsubseteq}\ (\multimap \mathtt{nilu}(\mathtt{p}\times\mathbb{1})\ \mathtt{cons})R \qquad (13.3.2\text{c})$$

the $\mathtt{cons}$ branch of $\mathtt{F(R)S}\sqsubseteq\mathtt{SR}$; the $\mathtt{nil}$ branch is $\mathtt{nil}\sqsubseteq\mathtt{nil}\ R$. $(\mathbb{1}\times R)\ \mathtt{cons}\sqsubseteq\mathtt{cons}\ R$ is $\mathtt{cons\ length}=(\mathbb{1}\times\mathtt{length})\pi_2\ \mathtt{succ}$ with $\mathtt{succ}$ monotone — a longer tail makes a longer list.

$$\frac{\mathtt{S}}{\ni}\ \mathtt{max(R)}\ \overset{\text{coproduct of maps}}{=}\ \left[\frac{\mathtt{nil}}{\ni}\ \mathtt{max(R)},\ \frac{\multimap \mathtt{nilu}(\mathtt{p}\times\mathbb{1})\ \mathtt{cons}}{\ni}\ \mathtt{max(R)}\right]\ \overset{\text{singleton, }R^\circ\text{ reflexive}}{=}\ \left[\mathtt{nil},\ \frac{\multimap \mathtt{nilu}(\mathtt{p}\times\mathbb{1})\ \mathtt{cons}}{\ni}\ \mathtt{max(R)}\right] \qquad (13.3.2\text{d})$$

$$\overset{\mathtt{nil}\ R=\top}{=}\ [\mathtt{nil},(\pi_1\mathtt{p}\to\mathtt{cons},\multimap \mathtt{nil})]$$

the set is $\{\mathtt{nil}\}$ where $\mathtt{p}$ fails on the head and $\{\mathtt{nil},\mathtt{cons(a,xs)}\}$ where it holds, and $\mathtt{nil}$ loses the second — (13.1a) at a two-element set.

(13.3.2e)

the law	**what it says**
$\llparenthesis \frac{\mathtt{S}}{\ni}\ \mathtt{max(R)}\rrparenthesis\sqsubseteq \frac{\llparenthesis\mathtt{S}\rrparenthesis}{\ni}\ \mathtt{max(R)}$ Theorem 7.2, (13.3e) at R°; $\mathtt{max(R)}=\mathtt{min}(R^\circ)$, so the hypothesis is monotonicity on $(R^\circ)^\circ=R$, which is (13.3.2c)	greedy: one longest $\mathtt{p}$-prefix kept at each $\mathtt{cons}$, instead of every $\mathtt{p}$-prefix collected and one chosen at the end
$\mathtt{prefix}^\circ\ \mathtt{prefix}\cap R\cap R^\circ\sqsubseteq\mathbb{1}$ context, (13.1b): only R between prefixes of one list counts	two prefixes of one list of equal length are equal, so $\frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{max(R)}$ is simple — **the** longest, not **a** longest
$\frac{\mathtt{prefix\ list(p)}}{\ni}\ \mathtt{max(R)}$ entire	$\mathtt{nil}$ is always a $\mathtt{p}$-prefix and R is connected on the prefixes of one list, so the longest exists
$X\sqsubseteq Y$, X entire, Y simple $\Longrightarrow X=Y$ $\llparenthesis[\mathtt{nil},(\pi_1\mathtt{p}\to\mathtt{cons},\multimap \mathtt{nil})]\rrparenthesis$ is a reduce of maps, hence entire	what turns the greedy $\sqsubseteq$ into the heading's $=$

§13.3.3. $\mathtt{mss}=\llparenthesis[\mathtt{zero\ wrap},\langle(\mathbb{1}\times\mathtt{head})\oplus,\pi_2\rangle\ \mathtt{cons}]\rrparenthesis\ \mathtt{max}(\leq)$

.3.3a)

$\mathtt{FX}=\mathtt{1}+\mathtt{A}\times\mathtt{X}$, $\mathtt{A}:=\mathtt{Int}$, $\alpha\triangleq[\mathtt{nil},\mathtt{cons}]$, $\mathtt{sum}=\llparenthesis[\mathtt{zero},\mathtt{plus}]\rrparenthesis$ and $\mathtt{segment}=\mathtt{suffix\ prefix}$ from (7a) and (12a).

$\mathtt{head}\triangleq\mathtt{cons}^\circ\ \pi_1$, $\mathtt{wrap}\triangleq\langle\mathbb{1},\multimap \mathtt{nil}\rangle\ \mathtt{cons}$ the head of a list and the one-element list, beside (12a)'s $\mathtt{tail}\triangleq\mathtt{cons}^\circ\ \pi_2$

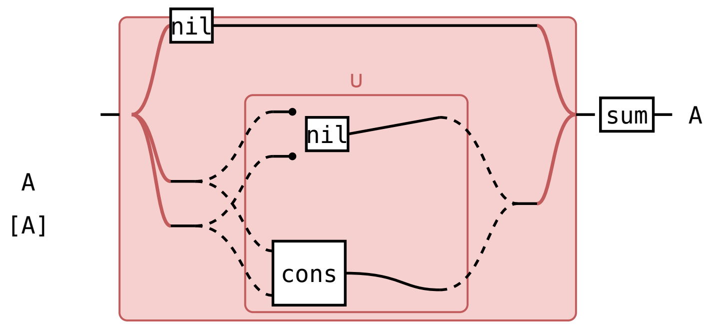
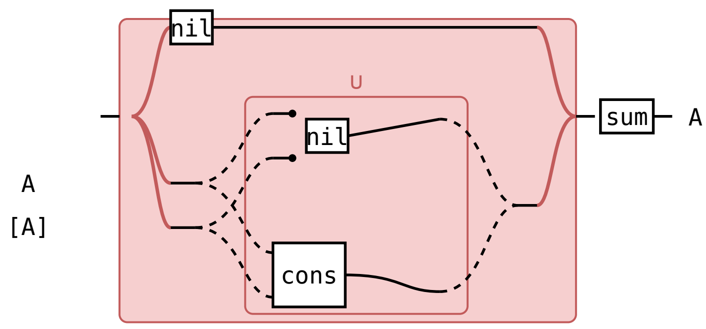
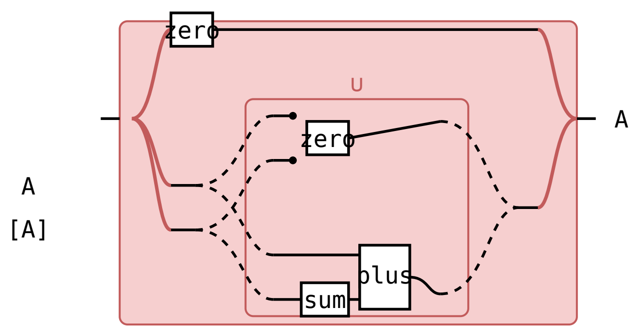
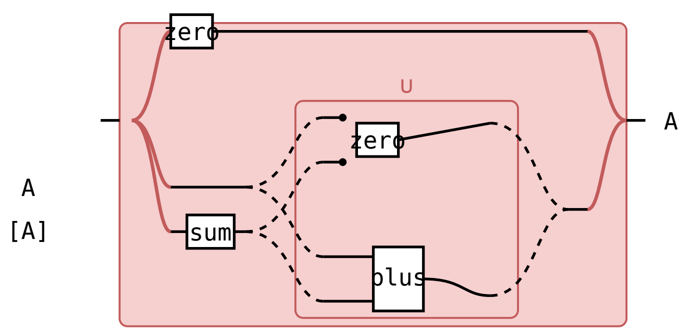
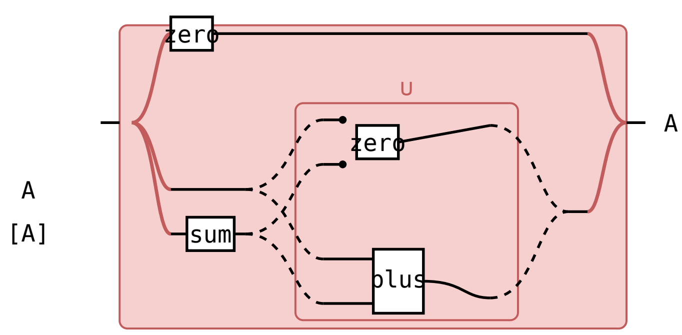
$\oplus\triangleq\frac{\multimap \mathtt{zerouplus}}{\ni}\ \mathtt{max}(\leq)$ B&dM's $\mathtt{oplus}=\mathtt{max}(\Lambda(\mathtt{zerouplus}))$; the set at $\mathtt{(a,b)}$ is $\{\mathtt{0},\mathtt{a+b}\}$, so $\oplus$ is the larger of the two

$\frac{\texttt{segment sum}}{\ni}\ \texttt{max(≤)}=\frac{\texttt{suffix}}{\ni}\ \texttt{E(}\frac{\texttt{prefix sum}}{\ni}\ \texttt{max(≤)) max(≤)}$ 3.3.3b)

formula — one wire from `[A]` to `A`, its type written along it		**reason**
$\cdot$ [A] $\boxed{\frac{\texttt{segment sum}}{\ni}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{segment sum}}{\ni}$ `max(≤)`	
= $\cdot$ [A] $\boxed{\frac{\texttt{suffix (prefix sum)}}{\ni}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{suffix (prefix sum)}}{\ni}$ `max(≤)`	`segment=suffix prefix` (12a), (13.3.3a)
= $\cdot$ [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E([A]) $\boxed{\texttt{E(prefix sum)}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{suffix}}{\ni}$ `E(prefix sum) max(≤)`	absorption (10b) — `frac(S,∋)E(R)=frac(SR,∋)` at `S:=suffix`, `R:=prefix sum`
= $\cdot$ [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E([A]) $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\texttt{)}}$ E(EA) $\boxed{\texttt{union}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{suffix}}{\ni}$ `E(`$\frac{\texttt{prefix sum}}{\ni}$`)` `union max(≤)`	$\frac{\texttt{R}}{\ni}$ `∋=R`, `union=E(∋)` (10b)'s `frac(R,∋)∋=R` at `R:=prefix sum` and `E(R)≙frac(∋R,∋)`; (13.1b)'s `union≙frac(∋∋,∋)`; the middle equality is (11a)'s `F(RS)=F(R)F(S)` at `F:=E`
= $\cdot$ [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E([A]) $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\texttt{)}}$ E(EA) $\boxed{\texttt{E(max(≤))}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{suffix}}{\ni}$ `E(`$\frac{\texttt{prefix sum}}{\ni}$`)` `E(max(≤)) max(≤)`	(13.1b), the sets non-empty
= $\cdot$ [A] $\boxed{\frac{\texttt{suffix}}{\ni}}$ E([A]) $\boxed{\texttt{E(}\frac{\texttt{prefix sum}}{\ni}\ \texttt{max(≤))}}$ EA $\boxed{\texttt{max(≤)}}$ A $\cdot$	$\frac{\texttt{suffix}}{\ni}$ `E(`$\frac{\texttt{prefix sum}}{\ni}$ `max(≤))` `max(≤)`	relator (11a) — `F(RS)=F(R)F(S)` at `F:=E`

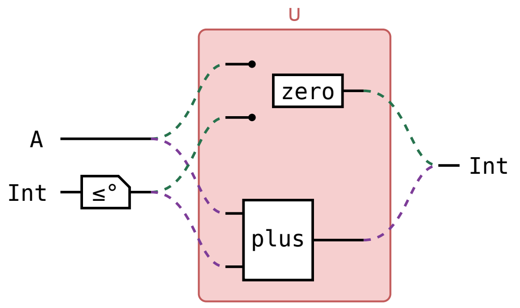
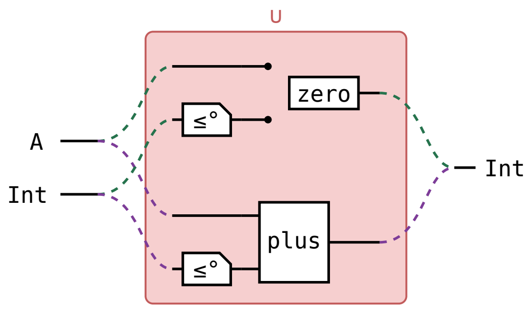
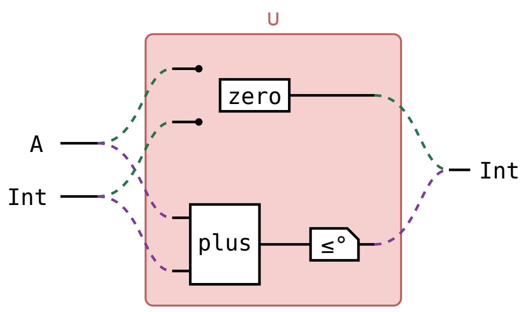
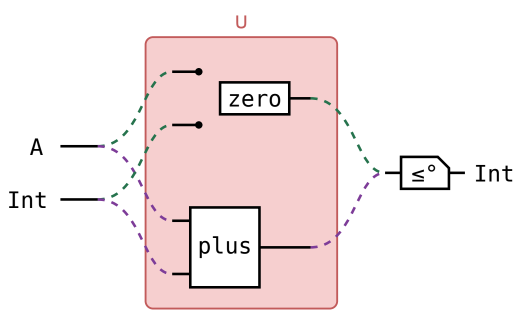
B&dM's `max(P(max(Λ(sum prefix))))Λsuffix`, mirrored. The `union` step is (13.1b)'s `P(min(R)) min(R)=P(Dom(min(R))) union min(R)` — every suffix has the empty prefix, so `Dom` is 𝟙 here — and `P(f)=E(f)` at the map it is applied to ((11.3d)).

`[nil,⊸ nil∪cons] sum=F(sum)[zero,⊸ zero∪plus]` (13.3.3c)

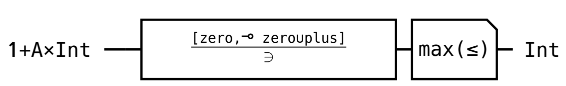
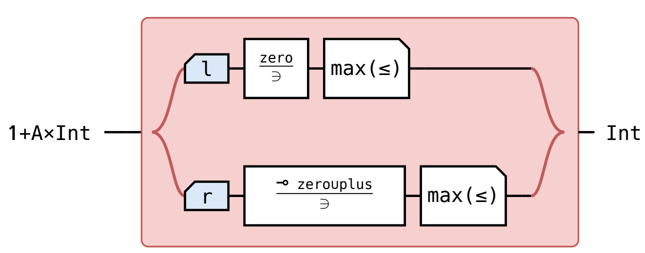
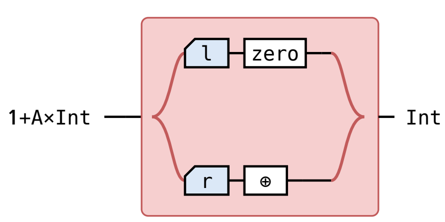
formula — the fork is the bracket's case split `F([A])=𝟙+A×[A]`: `nil` above, the pair and its `∪` below	**reason**
 `[nil,⊸ nil∪cons] sum`	
=  `[nil sum,⊸ nil sum∪cons sum]`	coproduct of maps, composition over `∪`
=  `[zero,⊸ zero∪(𝟙×sum) plus]`	`sum`'s defining equation
=  `[zero,(𝟙×sum)(⊸ zero∪plus)]`	`(𝟙×sum)⊸=⊸`, `sum` entire
=  `F(sum) [zero,⊸ zero∪plus]`	relator

(11.4.2a) at α_B `:=[nil,⊸ nil∪cons]`, `S:=sum`: the side condition, so `prefix sum=⦇[zero,⊸ zero∪plus]⦈`. `prefix` is the reduce, `sum` the map fused into it — the intermediate list is gone.

(13.3.3d)

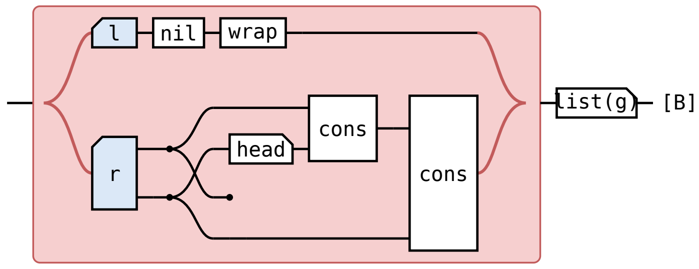
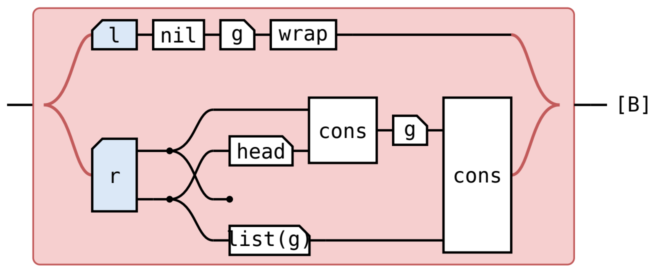
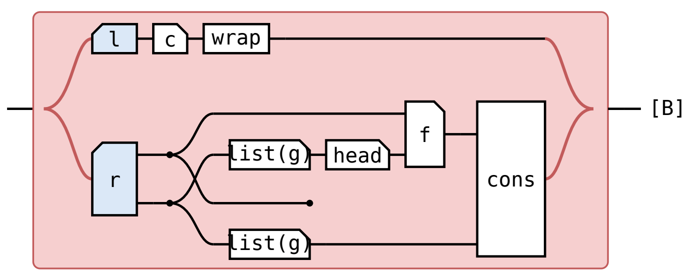
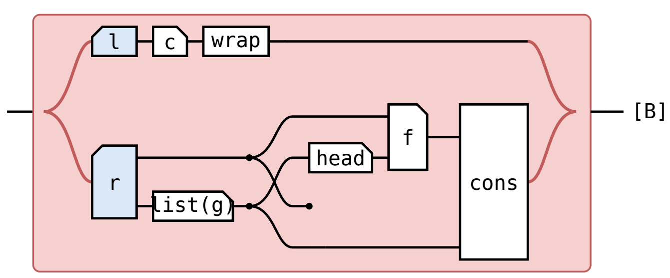
$(\mathbb{1}\times\le^\circ)(\multimap\ \mathtt{zerouplus})\sqsubseteq(\multimap\ \mathtt{zerouplus})\le^\circ$ the $\mathtt{plus}$ branch of $F(\le^\circ)S\sqsubseteq S\le^\circ$; the $\mathtt{zero}$ branch is $\mathtt{zero}\sqsubseteq\mathtt{zero}\le^\circ$		
formula — the head above, the running sum below; the tape is the $\cup$		**reason**
	$(\mathbb{1}\times\le^\circ)(\multimap\ \mathtt{zerouplus})$	
$=$ 	$(\mathbb{1}\times\le^\circ)\multimap\ \mathtt{zero}\cup(\mathbb{1}\times\le^\circ)\ \mathtt{plus}$	relator, composition over $\cup$
$\sqsubseteq$ 	$\multimap\ \mathtt{zerouplus}\le^\circ$	(5.1a), $(\le^\circ\times\le^\circ)\ \mathtt{plus}\sqsubseteq\mathtt{plus}\le^\circ$ $(\le\times\le)\ \mathtt{plus}\sqsubseteq\mathtt{plus}\le$ is (13.3a), written + there, and (13.3e)'s first row carries it to $\le^\circ$ — $\mathtt{plus}$ is a map, so it is monotonic on an order and on its opposite together.
$\sqsubseteq$ 	$(\multimap\ \mathtt{zerouplus})\le^\circ$	$\le^\circ$ reflexive

(13.3.3e)

$\frac{[\mathtt{zero},\multimap\ \mathtt{zerouplus}]}{\ni}\ \mathtt{max}(\le)=[\mathtt{zero},\oplus]$		
formula — the tape is the coproduct: $\mathtt{zero}$'s branch above, $\mathtt{plus}$'s below		**reason**
	$\frac{[\mathtt{zero},\multimap\ \mathtt{zerouplus}]}{\ni}\ \mathtt{max}(\le)$	
$=$ 	$[\frac{\mathtt{zero}}{\ni}\ \mathtt{max}(\le),\ \frac{\multimap\ \mathtt{zerouplus}}{\ni}\ \mathtt{max}(\le)]$	coproduct of maps (11.2.1a) at $T:=[\mathtt{zero},\multimap\ \mathtt{zerouplus}]$, then $[U,V]Z=[UZ,VZ]$ — (11.2a), composition over $\cup$
$=$ 	$[\mathtt{zero},\oplus]$	singleton, $\le^\circ$ reflexive (13.1b)'s $\frac{1}{\ni}\ \mathtt{min}(R)=\mathbb{1}\cap R$ at $R:=\le^\circ$, $\mathtt{zero}$ a map; the lower branch is $\oplus$'s definition, (13.3.3a), and no law

with (13.3.3d) the greedy theorem gives $(\![\mathtt{zero},\oplus]\!)\sqsubseteq\frac{\mathtt{prefix\ sum}}{\ni}\ \mathtt{max}(\le)$ — B&dM's own containment — and (13.3.3g)'s second row makes it an equality.

(13.3.3f)

$[\mathtt{nil\ wrap},\langle(\mathbb{1}\times\mathtt{head})\ \mathtt{cons},\pi_2\rangle\ \mathtt{cons}]\ \mathtt{list(g)}=F(\mathtt{list(g)})[\mathtt{c\ wrap},\langle(\mathbb{1}\times\mathtt{head})\mathtt{f},\pi_2\rangle\ \mathtt{cons}]$	
formula — the 1 branch of $1+A\times[[A]]$ above, the root and its list of tails below	**reason**
 $[\mathtt{nil\ wrap},\langle(\mathbb{1}\times\mathtt{head})\ \mathtt{cons},\pi_2\rangle\ \mathtt{cons}]\ \mathtt{list(g)}$	$\mathtt{g}\triangleq\llparenthesis[\mathtt{c,f}]\rrparenthesis$ and $\mathtt{tails}=\llparenthesis[\mathtt{nil\ wrap},\langle(\mathbb{1}\times\mathtt{head})\ \mathtt{cons},\pi_2\rangle\ \mathtt{cons}]\rrparenthesis$, since $\mathtt{tails(cons(a,xs))}$ is $\mathtt{cons(cons(a,head(tails\ xs)),tails\ xs)}$
=  $[\mathtt{nil\ g\ wrap},\langle(\mathbb{1}\times\mathtt{head})\ \mathtt{cons\ g},\pi_2\ \mathtt{list(g)}\rangle\ \mathtt{cons}]$	coproduct of maps, wrap and cons natural
=  $[\mathtt{c\ wrap},\langle(\mathbb{1}\times(\mathtt{list(g)\ head}))\mathtt{f},(\mathbb{1}\times\mathtt{list(g)})\pi_2\rangle\ \mathtt{cons}]$	g's defining equation, $\mathtt{list(g)\ head=head\ g}$ the π_2 slide $\pi_2\mathtt{list(g)}=(\mathbb{1}\times\mathtt{list(g)})\pi_2$ is 1 and 4 of (11.1.2b), $\mathrm{Dom}(\pi_1)=\mathbb{1}$ ((5c)); $\mathtt{list(g)\ head=head\ g}$ is the one step of (13.3.3f) the note has no law for — (13.3.3g)'s third row
=  $F(\mathtt{list(g)})[\mathtt{c\ wrap},\langle(\mathbb{1}\times\mathtt{head})\mathtt{f},\pi_2\rangle\ \mathtt{cons}]$	fork, relator the fork slide is 7 of (11.1.2b), $\mathbb{1}\times\mathtt{list(g)}$ a map there — and it is at $\mathtt{c,f}:=\mathtt{zero},\oplus$. (11.4.2a) then reads off tails $\mathtt{list(g)}=\llparenthesis[\mathtt{c\ wrap},\langle(\mathbb{1}\times\mathtt{head})\mathtt{f},\pi_2\rangle\ \mathtt{cons}]\rrparenthesis$, the heading's fold.

(13.3.3g)

the law	**what it says**
$\llparenthesis\frac{S}{\ni}\ \mathtt{max}(\le)\rrparenthesis\sqsubseteq\frac{\llparenthesis S\rrparenthesis}{\ni}\ \mathtt{max}(\le)$ (13.3e) at $\le^\circ$, $S:=[\mathtt{zero},\multimap\ \mathtt{zerouplus}]$, condition (13.3.3d)	greedy: one running maximum kept at each cons, instead of every prefix sum collected and one chosen at the end
$X\sqsubseteq Y$, X entire, Y simple $\Longrightarrow X=Y$ (13.3.2e)'s last row at this S	$\llparenthesis[\mathtt{zero},\oplus]\rrparenthesis$ is a reduce of maps, and the prefix sums of one list are finitely many integers with one greatest, so (13.3.3e)'s $\sqsubseteq$ is an equality
$\mathtt{list(g)\ head=head\ g}$ the one step of (13.3.3f) the note has no law for	head is a partial map, undefined at nil; the equality holds because tails never returns an empty list, which is a fact about tails, not a naturality square
tails implements $\frac{\mathtt{suffix}}{\ni}$, $\mathtt{list(f)}$ implements $E(\mathtt{f})$ (12a)	what makes (13.3.3b) a program: one fold builds the $n+1$ running maxima, and the final $\mathtt{max}(\le)$ reads them in one more pass, so mss is linear

§13.3.4. $\mathtt{filter(p)}=\llparenthesis[\mathtt{nil},(\pi_1\mathtt{p}\to\mathtt{cons},\pi_2)]\rrparenthesis$

(13.3.4a)

> F, α, p, R as in (13.3.2a); $S\triangleq[\mathtt{nil},\pi_2\cup(\mathtt{p}\times\mathbb{1})\ \mathtt{cons}]\ :\ F([A])\to[A]$, π_2 where (13.3.2a) has $\multimap$ nil.
>
> $\mathbb{1}\sqsubseteq\pi_2 R\ \mathtt{cons}^\circ$ the tail is one shorter than the cons, so π_2 loses the $\mathtt{max}(R)$ at every step — where (13.3.2a)'s loser is nil

$$\boxed{\alpha\ \mathtt{subseq\ list(p)}} \;\underset{\text{defining equation}}{=\!=\!=}\; \boxed{\begin{array}{c}\mathtt{F(subseq)}\\ \mathtt{[nil,cons\cup\pi_2]\ list(p)}\end{array}} \;\underset{\mathtt{list(p)}\text{ through }\mathtt{cons},\ \pi_2\text{ natural}}{=\!=\!=}\; \boxed{\begin{array}{c}\mathtt{F(subseq)}\\ \mathtt{[nil,(p\times list(p))}\\ \mathtt{cons\cup(1\!\!1\times list(p))\pi_2]}\end{array}}$$
$$\;\underset{\text{relator, }\mathtt{subseq\ list(p)}\text{ entire}}{=\!=\!=}\; \boxed{\mathtt{F(subseq\ list(p))S}} \tag{13.3.4b}$$

(11.6.1a) reads that off as $\mathtt{subseq\ list(p)=(\!|S|\!)}$ — (13.3.2b)‘s chain with π_2 for $\multimap\ \mathtt{nil}$. Here (11.4.2a) is blocked for the same reason, and π_2 is the branch that carries $\mathtt{list(p)}$ out by naturality ((12.1f) at $\ni\mathtt{:=list(p)}$).

$$\boxed{\begin{array}{c}\mathtt{(1\!\!1\times R)}\\ \mathtt{(\pi_2\cup(p\times 1\!\!1)}\\ \mathtt{cons)}\end{array}} \;\underset{\text{composition over }\cup,\ \pi_2\text{ natural}}{=\!=\!=}\; \boxed{\begin{array}{c}\mathtt{\pi_2R\cup(p\times R)}\\ \mathtt{cons}\end{array}} \;\underset{\text{(13.3.2c)‘s }\mathtt{cons}\text{ branch}}{\sqsubseteq}\; \boxed{\begin{array}{c}\mathtt{\pi_2R\cup(p\times 1\!\!1)}\\ \mathtt{cons\ R}\end{array}} \;\underset{\text{composition over }\cup}{=\!=\!=}\; \boxed{\begin{array}{c}\mathtt{(\pi_2\cup(p\times 1\!\!1)}\\ \mathtt{cons)R}\end{array}} \tag{13.3.4c}$$

$\mathtt{F(R)S\sqsubseteq SR}$, the $\mathtt{nil}$ branch again $\mathtt{nil\sqsubseteq nil\ R}$. The first step is an equality: (13.3.2c) spends $\mathtt{R}$ reflexive on getting $\multimap\ \mathtt{nil}$ past $\mathtt{1\!\!1\times R}$, and π_2 needs only its naturality square.

$$\boxed{\tfrac{\mathtt{S}}{\ni}\ \mathtt{max(R)}} \;\underset{\text{(13.3.2d)‘s first two steps}}{=\!=\!=}\; \boxed{\mathtt{[nil,}\ \tfrac{\mathtt{\pi_2\cup(p\times 1\!\!1)\ cons}}{\ni}\ \mathtt{max(R)]}} \;\underset{\mathtt{1\!\!1\sqsubseteq\pi_2R\ cons^\circ}}{=\!=\!=}\; \boxed{\mathtt{[nil,(\pi_1p\to cons,\pi_2)]}} \tag{13.3.4d}$$

at $\mathtt{(a,xs)}$ the set is $\mathtt{\{xs\}}$ where $\mathtt{p}$ fails on $\mathtt{a}$ and $\mathtt{\{xs,cons(a,xs)\}}$ where it holds, and $\mathtt{xs}$ loses the second — (13.1a) at a two-element set. The head is dropped, not the whole tail: that is the one place π_2 shows against (13.3.2d)‘s $\multimap\ \mathtt{nil}$.

(13.3.4e)

the law	**what it says**
$\mathtt{(\!\vert\tfrac{S}{\ni}\ max(R)\vert\!)\sqsubseteq \tfrac{(\!\vert S\vert\!)}{\ni}\ max(R)}$ (13.3.2e)‘s first row at this $\mathtt{S}$, its condition (13.3.4c)	greedy: one longest $\mathtt{p}$-subsequence kept at each $\mathtt{cons}$
$\mathtt{S^\circ S\cap R\cap R^\circ\sqsubseteq 1\!\!1}$ fails at $\mathtt{S:=subseq\ list(p)}$ (13.3.2e)‘s uniqueness row does not transfer	two $\mathtt{p}$-subsequences of one list can have equal length and differ: the prefixes of one list are linearly ordered by length, its subsequences are not
$\tfrac{\mathtt{subseq\ list(p)}}{\ni}\ \mathtt{max(R)}$ simple (12a)	a $\mathtt{p}$-subsequence that drops a passing element is beaten by the one that keeps it, so only the subsequence keeping **exactly** the passing elements survives $\mathtt{max(R)}$ — **the** longest, not **a** longest
$\mathtt{X\sqsubseteq Y}$, $\mathtt{X}$ entire, $\mathtt{Y}$ simple $\Longrightarrow\mathtt{X=Y}$ (13.3.2e)‘s last row at this $\mathtt{S}$	what turns the greedy $\sqsubseteq$ into the heading’s $=$; $\mathtt{(\!\vert[nil,(\pi_1p\to cons,\pi_2)]\vert\!)}$ is again a reduce of maps, hence entire

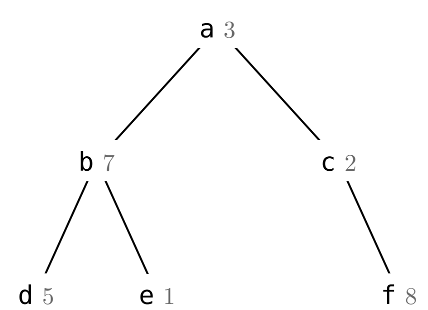
§13.4. Planning a company party

(13.4a)

definition	type	note
`tree A::=node (A,[tree A])`	$\mathcal{A}\to\mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
`F(A,B)=A×[B]`	$\mathcal{A}\times\mathcal{A}\to\mathcal{A}$	The base functor `tree` folds: an employee beside the recursive position, one layer deep.
`rating`	`A→Real`	What one employee is worth as a guest.
`cost≙list(rating) sum`	`[A]→Real`	What a guest list is worth.
`R≙cost≤cost°`	`[A]→[A]`	The preorder the guest list is maximised over.
`choose≙π₁∪π₂`	`[A]×[A]→[A]`	Takes one of the two parties a subtree returns.
`include≙(𝟙×(list(π₂) concat)) cons`	`F(A,[A]×[A])→[A]`	The party that invites the root, which puts every immediate subtree's root out. A map.
`exclude≙(𝟙×(list(choose) concat))π₂`	`F(A,[A]×[A])→[A]`	The party that leaves the root out, so each subtree is free to choose. Not a map.
`S≙⟨include,exclude⟩`	`F(A,[A]×[A])→[A]×[A]`	The algebra: one step returns both parties of a subtree at once.
`party≙⦇S⦈ choose`	`tree A→[A]`	Every guest list the president's ruling allows.
the specification $\underset{\ni}{\texttt{party}}$ `max(R)`	`tree A→[A]`	A guest list of greatest total conviviality.

§13.4.1. `include` and `exclude`

(13.4.1a)



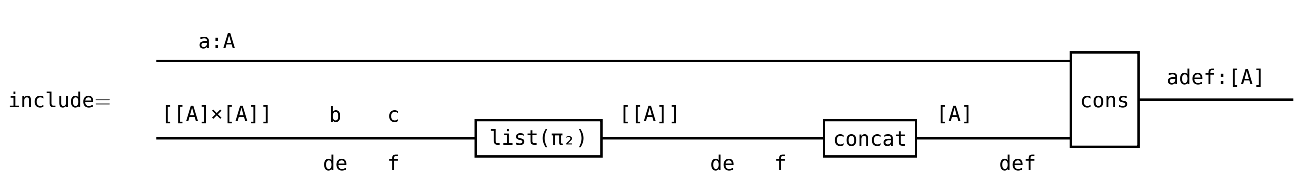
the small grey number is the employee's rating

(13.4.1b)

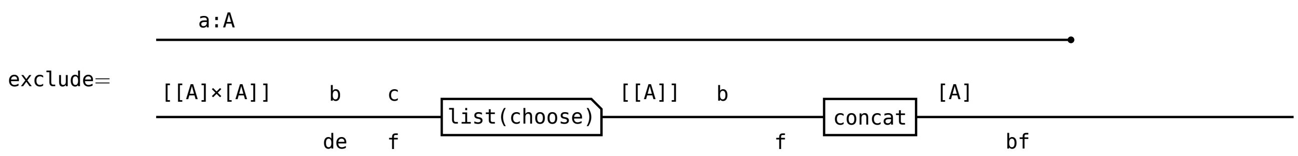
node	the algebra's input `A×[[A]×[A]]`	`include`	`exclude`
`d`, `e`, `f`	`(d,[])`	`[d]`	`[]`
`b`	`(b,[([d],[]),([e],[])])`	`[b]`	`[d,e]`, `[d]`, `[e]`, `[]`
`c`	`(c,[([f],[])])`	`[c]`	`[f]`, `[]`
`a`	`(a,[([b],…),([c],…)])`	`[a]⧺exclude(b)⧺exclude(c)`	`choose(b)⧺choose(c)`

with those ratings, `max(R)` keeps `([b]=7,[d,e]=6)` at `b`, where `include` wins, and `([c]=2,[f]=8)` at `c`, where `exclude` wins, so at the root `include=[a,d,e,f]=17` beats `exclude=[b,f]=15`, and `choose` takes 17.

(13.4.1c)



(13.4.1d)



(13.4.1e)

```
list(choose) [([d],[]),([e],[])]
  1st item              ([d],[])  choose↦[d]  or  []
  2nd item              ([e],[])  choose↦[e]  or  []
  2×2=4 combinations:   [[d],[e]]   [[d],[]]   [[],[e]]   [[],[]]
  concat flattens:        [d,e]       [d]        [e]        []
```

§13.4.2. list(R×R)

(13.4.2a)

```
[   ([b],[d])   ,   ([c],[])   ]
        |               |
        ▼               ▼                list(R×R), elementwise
[  ([b],[d,e])  ,  ([c],[f])   ]
```

```
([b],[d])
 |    └── exclude: the best party in b's subtree when b does not come
 └─────── include: the best party in b's subtree when b comes
```

(13.4.2b)

element	1st component include	2nd component exclude
1st, ([b],[d])	[b] R [b] 7≤7, reflexivity	[d] R [d,e] 5≤6, and ([b],[d,e]) is the pair max(R) keeps at b
2nd, ([c],[])	[c] R [c] 2≤2, reflexivity	[] R [f] 0≤8, and ([c],[f]) is the pair max(R) keeps at c

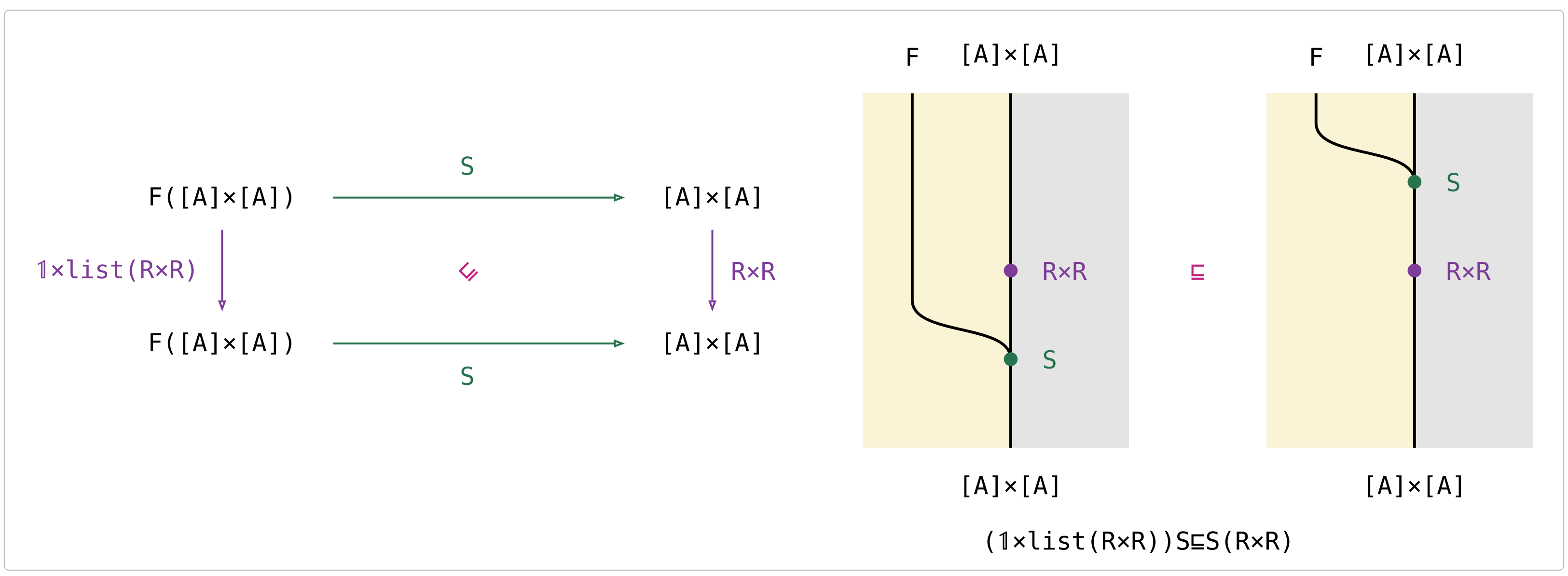
the first component of 𝟙×list(R×R) is the root employee.

R≜cost≤cost° is a preorder, so the components that do not move are related by reflexivity — R×R demands a relation in **both** components, it cannot skip one.

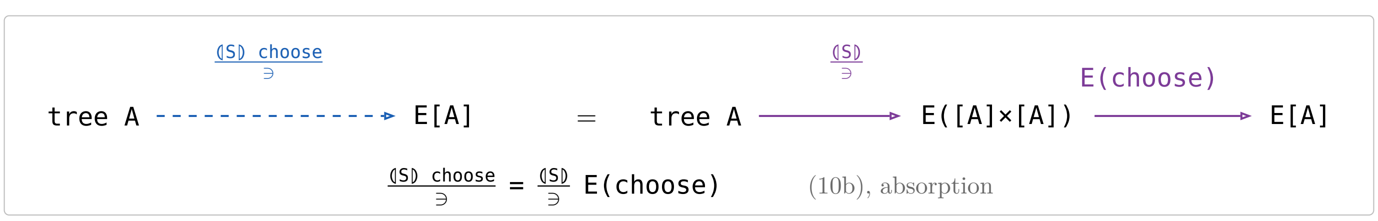
R compares cost only, not contents: [] R [f] is legal though [f] has an element [] does not. That is why the three leaves of §13.4.3′s proof all reduce to “cost is a sum”.

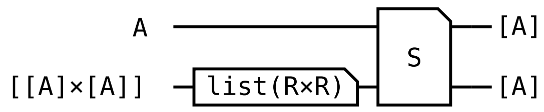
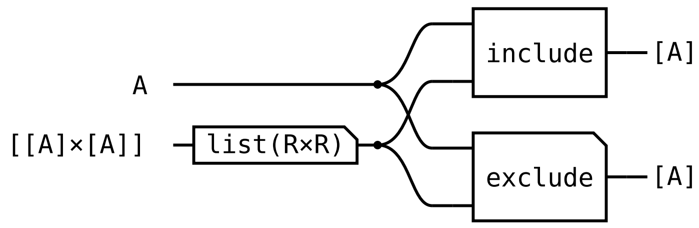
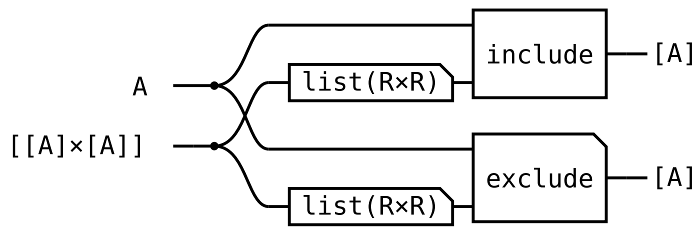
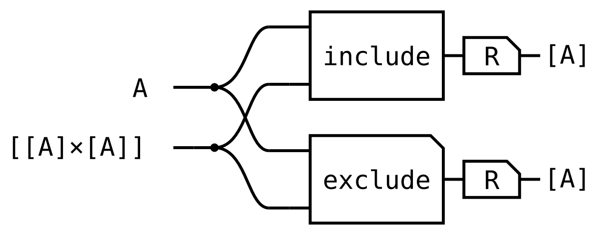
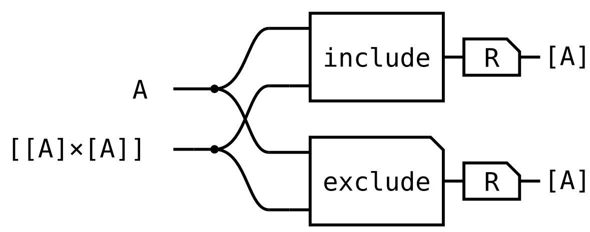
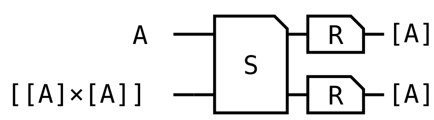
§13.4.3. (𝟙×list(R×R))S⊑S(R×R) — S : F([A]×[A])→[A]×[A] monotonic on R×R

(13.4.3a)



(13.4.3b)

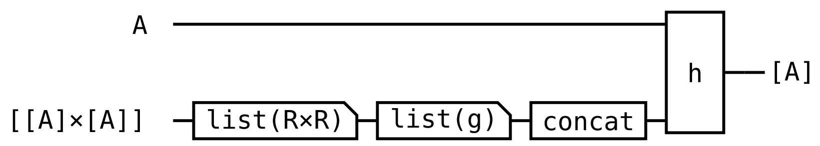


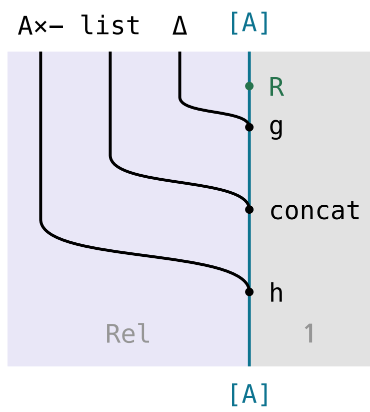
(𝟙×list(R×R))S⊑S(R×R)				(13.4.3c)
formula — a product is two wires: the root above, the subtrees' pairs below			**reason**	
		(𝟙×list(R×R))S		
=		(𝟙×list(R×R))⟨include,exclude⟩	S≙⟨include,exclude⟩	
⊑		⟨(𝟙×list(R×R))include, (𝟙×list(R×R))exclude⟩	(11.1b), (4c) ⊑, not =: 7 of (11.1.2b) asks for a map and 𝟙×list(R×R) is not one. B&dM write the step =.	
⊑		⟨include R,exclude R⟩	(13.4.3d) (11.1b): a fork is monotonic in both components	
=		⟨include,exclude⟩(R×R)	2 of (11.1.2b), ⟨X,Y⟩(R×S)=⟨XR,YS⟩	
=		S(R×R)	S≙⟨include,exclude⟩	

(𝟙×list(R×R))include⊑include R (𝟙×list(R×R))exclude⊑exclude R 3.4.3d)

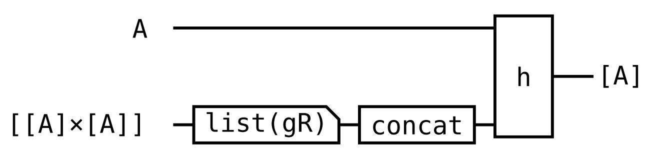
include,exclude = (𝟙×(list(g) concat))h

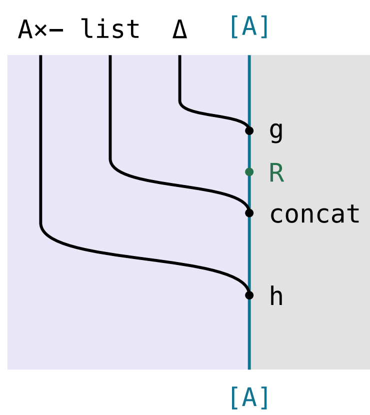
circuit	Hinze–Marsden



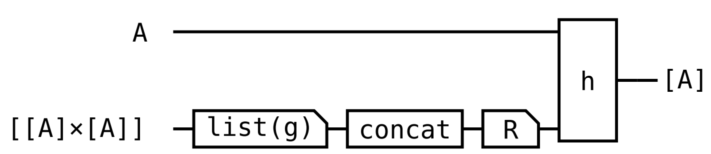


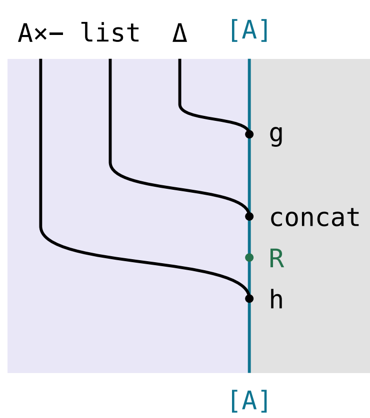
⊑



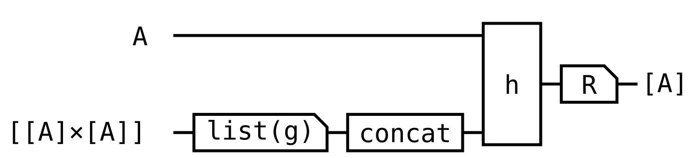


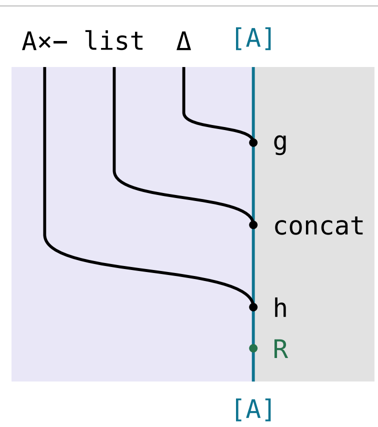
⊑





⊑





g (R×R)g⊑gR

g:=π_2 is (R×R)π_2=(Dom(π_1R))π_2R ⊑π_2R — 1 and 4 of (11.1.2b), then Dom⊑𝟙; g:=choose is §13.4.4.

list monotonic, (11a)

concat list(R)concat⊑concat R

a LEAF: no law above it. cost is a sum, so cost(concat xss)=sum(list(cost)xss) and a costlier part makes a costlier whole. B&dM's exercise.

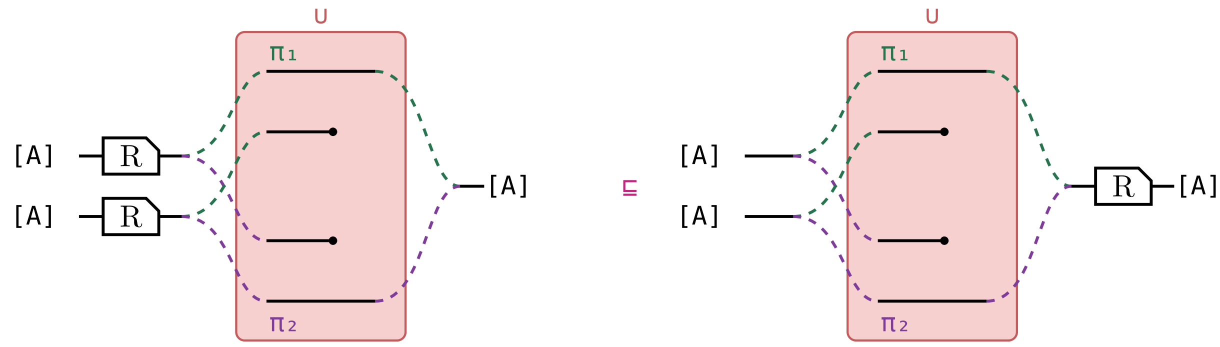
h (𝟙×R)h⊑hR

a LEAF: cost(cons(a,xs)) =rating(a)+cost(xs), so a costlier tail makes a costlier list. B&dM's exercise. For h:=π_2 it is an EQUALITY (𝟙×R)π_2=(Dom(π_1))π_2R =π_2R: π_1 is a map, hence entire, so Dom(π_1)=𝟙 — (5c)

	g	h
include	π_2	cons
exclude	choose	π_2

§13.4.4. $(R\times R)\mathrm{choose}\sqsubseteq\mathrm{choose}\ R$ — choose monotonic on R

(13.4.4a)

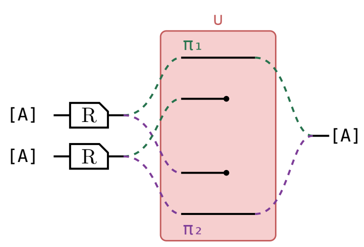
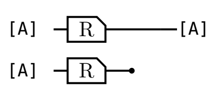
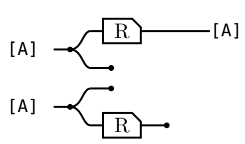
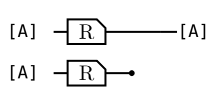
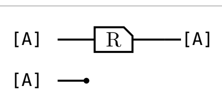
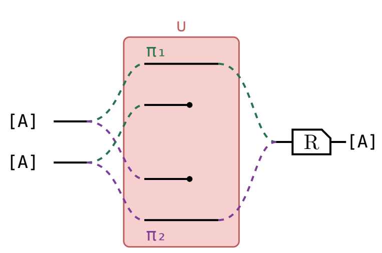
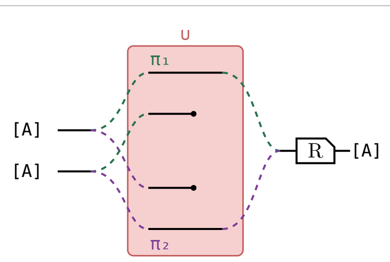


$(R\times R)\mathrm{choose}\sqsubseteq\mathrm{choose}\ R$

(13.4.4b)

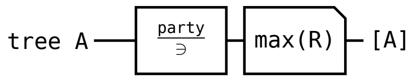
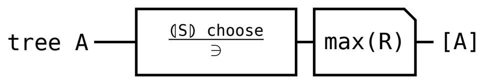
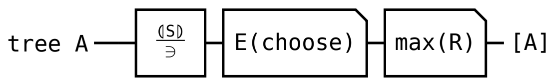
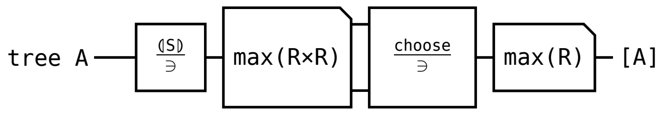
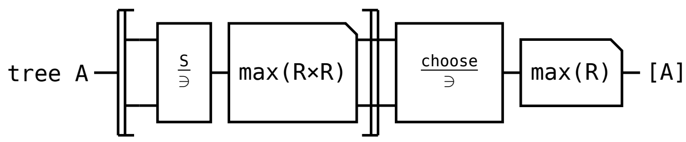
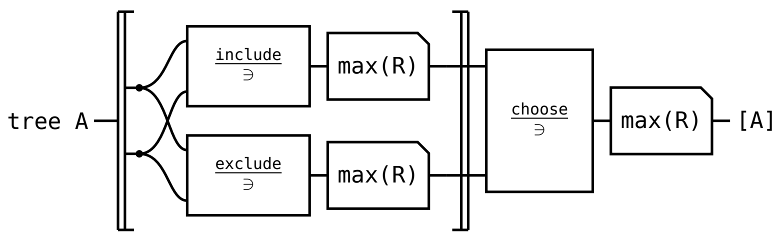
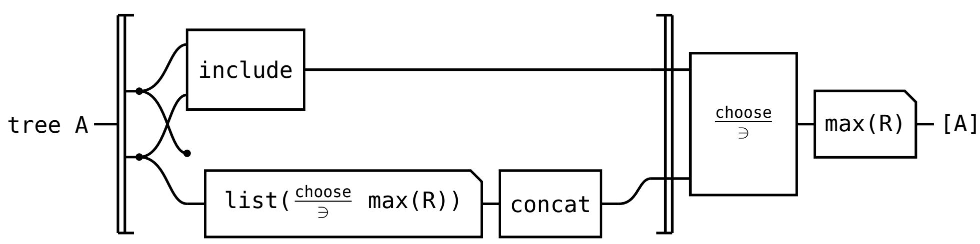
$R\triangleq\{(0,0)\} : \{0,1\}\to\{0,1\}$	not entire — undefined at 1
$(R\times R)\mathrm{choose}$ at $(0,1)$	nothing — $R\times R$ needs BOTH components related, and R is undefined at 1
choose R at $(0,1)$	0 — choose picks the first component 0, and R 0 0 holds
$((0,1),0)$	in choose R, not in $(R\times R)\mathrm{choose}$
$(R\times R)\mathrm{choose}=\mathrm{choose}\ R$	iff R is entire — $\mathrm{Dom}\sqsubseteq\mathbb{1}$ is the calculation's only inequality step, and it is an equality iff $\mathrm{Dom}(R)=\mathbb{1}$

(13.4.4c)

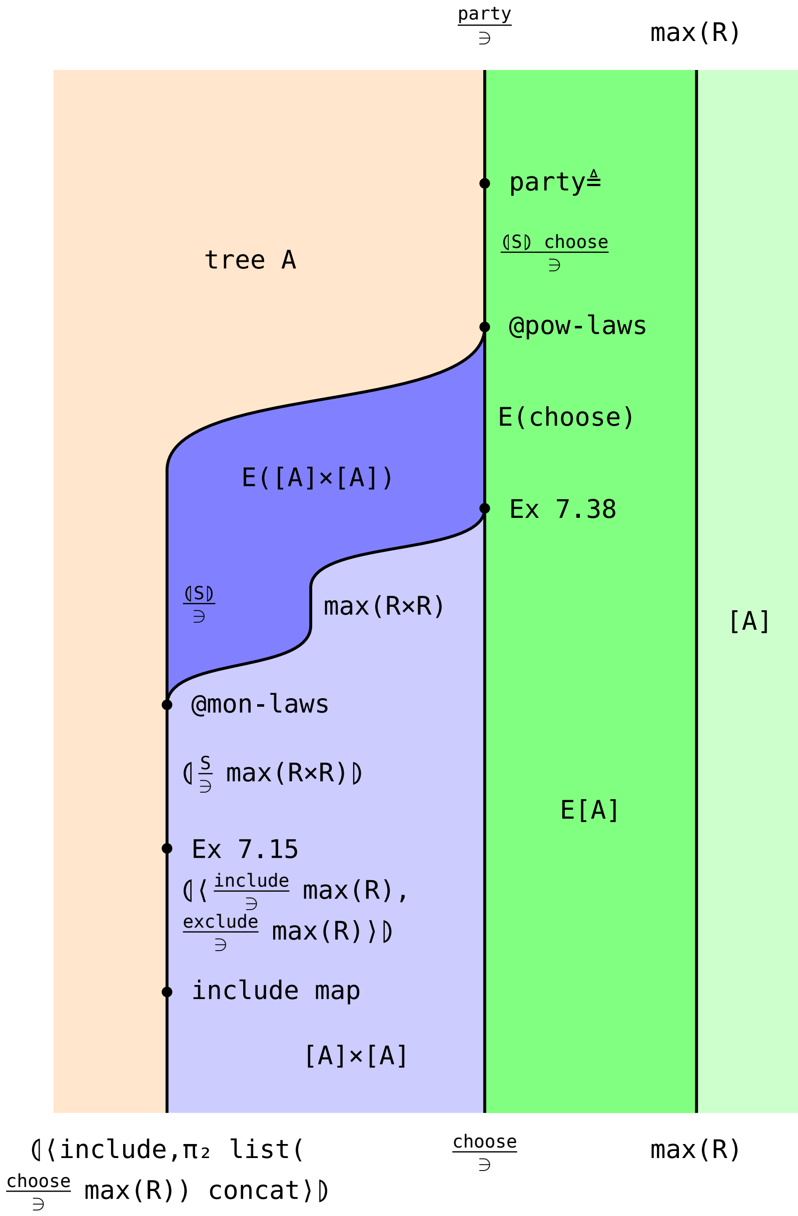
$(R\times R)\mathrm{choose}\sqsubseteq\mathrm{choose}\ R$		
formula — one branch of the ∪; the other is symmetric		**reason**
	$(R\times R)\mathrm{choose}$	
= 	$(R\times R)(\pi_1\cup\pi_2)$	$\mathrm{choose}\triangleq\pi_1\cup\pi_2$
= 	$(R\times R)\pi_1\cup(R\times R)\pi_2$	$T(R\cup S)=TR\cup TS$
= 	$\langle\pi_1R,\pi_2R\rangle\pi_1\cup\langle\pi_1R,\pi_2R\rangle\pi_2$	1 of (11.1.2b), $R\times S=\langle\pi_1R,\pi_2S\rangle$
= 	$(\mathrm{Dom}(\pi_2R))\pi_1R\cup(\mathrm{Dom}(\pi_1R))\pi_2R$	3 and 4 of (11.1.2b)
$\sqsubseteq$ 	$\pi_1R\cup\pi_2R$	$\mathrm{Dom}\sqsubseteq\mathbb{1}$
= 	$(\pi_1\cup\pi_2)R$	$(R\cup S)T=RT\cup ST$
= 	choose R	$\mathrm{choose}\triangleq\pi_1\cup\pi_2$

§13.4.5. The derivation

$$\frac{\texttt{party}}{\ni}\ \texttt{max(R)} \sqsupseteq (\!|\langle\texttt{include},\pi_2\ \texttt{list(}\frac{\texttt{choose}}{\ni}\ \texttt{max(R))}\ \texttt{concat}\rangle|\!)\ \frac{\texttt{choose}}{\ni}\ \texttt{max(R)} \qquad (13.4.5a)$$

	formula — one wire, `tree A` to `[A]`; the pair `[A]×[A]` is where it runs as two	**reason**
	 $\frac{\texttt{party}}{\ni}\ \texttt{max(R)}$	
=	 $\frac{(\!|\texttt{S}|\!)\ \texttt{choose}}{\ni}\ \texttt{max(R)}$	def. `party`
=	 $\frac{(\!|\texttt{S}|\!)}{\ni}\ \texttt{E(choose)}\ \texttt{max(R)}$	(10b), absorption
⊒	 $\frac{(\!|\texttt{S}|\!)}{\ni}\ \texttt{max(R×R)}\ \frac{\texttt{choose}}{\ni}\ \texttt{max(R)}$	§13.4.4
⊒	 $(\!|\frac{\texttt{S}}{\ni}\ \texttt{max(R×R)}|\!)\ \frac{\texttt{choose}}{\ni}\ \texttt{max(R)}$	(13.3e), `(𝟙×list(R×R))S⊑S(R×R)`
⊒	 $(\!|\langle\frac{\texttt{include}}{\ni}\ \texttt{max(R)},\ \frac{\texttt{exclude}}{\ni}\ \texttt{max(R)}\rangle|\!)\ \frac{\texttt{choose}}{\ni}\ \texttt{max(R)}$	Ex 7.15, the fork splits
⊒	 $(\!|\langle\texttt{include},\pi_2\ \texttt{list(}\frac{\texttt{choose}}{\ni}\ \texttt{max(R))}\ \texttt{concat}\rangle|\!)\ \frac{\texttt{choose}}{\ni}\ \texttt{max(R)}$	`include` a map, `max(R)` into each branch

(13.4.5b)



include map	⊑	include a map, max(R) into each branch
Ex 7.15	⊑	the fork splits
@mon-laws	⊑	(𝟙×list(R×R))S⊑S(R×R)
Ex 7.38	⊑	(R×R)choose⊑choose R
@pow-laws	=	absorption
party≙	=	definition of party

§13.5. Shortest paths on a cylinder

(13.5a)

definition	type	note
`F(A,X)=A+A×X`	$\mathcal{A}\times\mathcal{A}\to\mathcal{A}$	The base functor: one square of the new column, alone or in front of the path so far.
`L=list⁺`	$\mathcal{A}\to\mathcal{A}$	A path is a non-empty list of squares, one per column crossed.
`α` the initial algebra	`F(A,LA)→LA`	`α=[wrap,cons]`: a path is started by one square, or extended by one.
`N` the `n`-tuple relator	$\mathcal{A}\to\mathcal{A}$	One component per row of a column, so the fold carries `n` answers at once.
`R≜sum≤sum°`	`L Nat→L Nat`	The cost of a path, which the cheapest minimises.
`setify`	`NA→EA`	Forgets which row a component came from, so the `n` answers become one set.
`moves`	`NA→E(NA)`	The three columns a path may step to: rotated up, unrotated, rotated down.
`trans`	`E(NA)→N(EA)`	Transposes a set of tuples, so every row collects its own paths.
`zip`	`F(NA,NB)→NF(A,B)`	Commutes `N` with the base functor, pairing each new square with its own row's paths.
`cp≜` $\frac{\mathtt{F}(\mathbb{1},\ni)}{\ni}$	`F(A,EB)→E(F(A,B))`	Turns a square paired with a set of paths into the set of extensions of those paths.
`generate≜F(𝟙,moves trans N(union)) zip N(cp P(α))`	`F(NA,N(E(LA)))→N(E(LA))`	One fold step: extends every path of every row by the new column.
`paths≜⦇generate⦈ setify union`	`L N Nat→E(L Nat)`	Every path across the cylinder.
the specification `paths min(R)`	`L N Nat→L Nat`	A cheapest path from the entry side to the exit side.

§13.5.1. **`N(E(L A))`**

```
1   5
2   6      [(1,2,3,4),(5,6,7,8)] : L N Nat
3   7
4   8
```
(13.5.1a)

```
⦇generate⦈[(5,6,7,8)] = ({[5]},{[6]},{[7]},{[8]})
```
(13.5.1b)

```
generate((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) =
    ( {[1,5],[1,6],[1,8]},
      {[2,5],[2,6],[2,7]},
      {[3,6],[3,7],[3,8]},
      {[4,5],[4,7],[4,8]} )
```

(13.5.1c)

`N`	the 4-tuple, one component per row of the entry column
`E`	the set of paths that can start in that row
`L`	a path is the non-empty list of the squares it crosses

row 1′s set has no `[1,7]` — from row 1 only rows 4, 1 and 2 are reachable — and it does have `[1,8]`, because the cylinder glues the bottom row to the top. That is `moves trans N(union)` of (13.5a): component `k` collects the paths of rows `k-1`, `k`, `k+1`.

§13.5.2. The derivation

$$\boxed{\texttt{paths min(R)}} \;\underset{\text{definition of }\texttt{paths}}{=}\; \boxed{\begin{array}{c}\texttt{⦇generate⦈ setify union}\\ \texttt{min(R)}\end{array}} \;\underset{\text{(13.1b) at }\texttt{union}\text{, }\mathsf{R}\text{ a preorder}}{\sqsupseteq}\; \boxed{\begin{array}{c}\texttt{⦇generate⦈ setify}\\ \texttt{P(min(R)) min(R)}\end{array}} \tag{13.5.2a}$$

$$\underset{\texttt{setify}\text{ lax natural}}{\sqsupseteq}\; \boxed{\texttt{⦇generate⦈N(min(R)) setify min(R)}} \;\underset{\text{(11.4.2a) at }\texttt{Q}}{\sqsupseteq}\; \boxed{\texttt{⦇Q⦈ setify min(R)}}$$

$$\boxed{\texttt{generate N(min(R))}} \;\underset{\text{(7.13), then }\texttt{zip}\text{, }\texttt{trans}\text{, }\texttt{moves}\text{ lax natural}}{\sqsupseteq}\; \boxed{\texttt{F(𝟙,N(min(R)))Q}}$$

$$\boxed{\texttt{Q}} \;\underset{\text{the fusion condition read as a definition}}{=}\; \boxed{\texttt{F(𝟙,moves trans N(min(R))) zip N(α)}}$$

$$\underset{\texttt{zip=𝟙+zip'}\text{, }\texttt{α=[wrap,cons]}}{=}\; \boxed{\texttt{[N(wrap),(𝟙×moves trans N(min(R))) zip' N(cons)]}}$$

(7.13) is $\texttt{F(𝟙,min(R))α⊑cp P(α) min(R)}$, Theorem 7.1 at the map α with $\frac{\texttt{F(1,∋)α}}{\ni}$ $\texttt{=cp P(α)}$: extending every path in a set and then taking a minimum is beaten by extending one minimum. It is the crux here, not the greedy theorem.

§13.6. The security van problem

(13.6a)

definition	type	note
`R≜length≤length°`	`[[Int]]→[[Int]]`	The order the schedule is minimised over: fewer van visits is better.
`ceiling≜` $\frac{\texttt{prefix sum}}{\ni}$ `max(≤)`	`[Int]→Int`	The highest the bank's balance reaches over a stretch of transactions.
`floor≜` $\frac{\texttt{prefix sum}}{\ni}$ `min(≤)`	`[Int]→Int`	The lowest it reaches, so `ceiling-floor` is the cash the stretch has to carry.

`secure`
the coreflexive on `x` with
`bmax(ceiling x,ceiling x-floor x)≤N`
	`[Int]→[Int]`	The stretches one van visit can serve: some starting reserve keeps the cash between `0` and `N`.

`ok` the coreflexive on `(a,xs)` with `xs` non-empty and `[a]++head xs` secure	`Int×[[Int]]` `→Int×[[Int]]`	The test the final program runs, in place of `old`'s `secure`.
`new≜(wrap×𝟙) cons`	`Int×[[Int]]→[[Int]]`	Calls the van: the transaction opens a segment of its own.
`glue≜(𝟙×cons°) assocl (cons×𝟙) cons`	`Int×[[Int]]→[[Int]]`	Puts the transaction on the front of the first segment.
`old≜(𝟙×cons°) assocl` `((cons secure)×𝟙) cons`	`Int×[[Int]]→[[Int]]`	`glue` restricted to a first segment that stays secure.
`partition=⦇[nil,new∪glue]⦈`	`[Int]→[[Int]]`	Every splitting of the transactions into consecutive segments.
`S≜[nil,new∪old]`	`1+Int×[[Int]]→[[Int]]`	The algebra whose fold is every splitting of the transactions into secure segments.
`partition list(secure)=⦇S⦈`	`[Int]→[[Int]]`	Fusion: keeping only secure segments is what turns `glue` into `old`.
`H≜(head prefix° head°)∪(nil° nil)`	`[[Int]]→[[Int]]`	One schedule's first segment is a prefix of the other's, or both are empty.
`R;H≜R∩(R°⇒H)`	`[[Int]]→[[Int]]`	The refinement of `R` that makes both halves of `S` monotonic.
`|R|≜R∩¬R°`	`[[Int]]→[[Int]]`	The strict part `R` splits into: `R;H=|R|∪(R∩H)`.
the specification $\frac{\texttt{partition list(secure)}}{\ni}$ `min(R)`	`[Int]→[[Int]]`	A schedule with the fewest secure segments.

(13.6b)

$\boxed{\frac{\texttt{partition list(secure)}}{\ni}\ \texttt{min(R)}}$

$=$ { (11.4.2a), `secure prefix⊑prefix secure` }

$\boxed{\frac{\texttt{⦇S⦈}}{\ni}\ \texttt{min(R)}}$

$\sqsupseteq$ { (7.14) holds, (7.15) FALSE; `R;H⊑R` }

$\boxed{\frac{\texttt{⦇S⦈}}{\ni}\ \texttt{min(R;H)}}$

$\sqsupseteq$ { (13.3e), `S` monotonic on `R;H` by (7.16) and (7.17) }

$\boxed{⦇\frac{\texttt{S}}{\ni}\ \texttt{min(R;H)}⦈}$

$\sqsupseteq$ { `old⊑new (R;H)°` }

$\boxed{\texttt{⦇[nil,(ok→glue,new)]⦈}}$

$\boxed{\texttt{(𝟙×(R;H)) old}}$

$=$ { `R;H=|R|∪(R∩H)`, `∪` distributes }

$\boxed{\texttt{(𝟙×|R|) old∪(𝟙×(R∩H)) old}}$

$\sqsubseteq$ { (7.19) and (7.20) on `|R|`, (7.21) on `R∩H` }

$\boxed{\texttt{new (R∩H)∪old (R∩H)}}$

$\sqsubseteq$ { `X∩Y⊑X;Y`, converses }

$\boxed{\texttt{(new∪old)(R;H)}}$

(7.15) `(𝟙×R) old⊑(new∪old)R` is FALSE — the shorter partition need not stay secure — and that is what refining `R` to `R;H` buys; the second chain is (7.17), and (7.16) rests the same way on (7.18) `(𝟙×⊤) new⊑new H`.

§14. Thinning Algorithms

§14.1. Thinning

For $\mathtt{Q} : \mathtt{A} \to \mathtt{A}$, $\mathtt{thin\ Q} \triangleq (\ni/\ni) \cap (\in \backslash (\mathtt{Q}^{\circ} \in)) : \mathtt{EA} \to \mathtt{EA}$ (8.1). (14.1a)

$\mathtt{ys\ (thin\ Q)\ xs} \Longleftrightarrow \mathtt{xs} \subseteq \mathtt{ys} \wedge (\forall \mathtt{a} \in \mathtt{ys}.\ \exists \mathtt{b} \in \mathtt{xs}.\ \mathtt{b\ Q\ a})$

(14.1b)

the law	what it says
$\mathtt{X} \sqsubseteq \frac{\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \Longleftrightarrow \mathtt{X} \ni \sqsubseteq \mathtt{S}$ and $\mathtt{S}^{\circ}\mathtt{X} \sqsubseteq \mathtt{Q}^{\circ} \in$	the universal property: everything kept is an S-value, and every S-value has a Q-lower bound among the kept ones
$\mathtt{Q} \sqsubseteq \mathtt{R} \Longrightarrow \mathtt{thin\ Q} \sqsubseteq \mathtt{thin\ R}$	the fewer pairs Q relates, the fewer subsets count as thinnings
$\mathbb{1} \sqsubseteq \mathtt{thin\ Q}$, and $\mathtt{thin\ Q}$ is a preorder if $\mathtt{Q}$ is	keeping everything is always a legal thinning
$\mathtt{min(R)} = \mathtt{thin\ Q\ min(R)}$ $\mathtt{Q} \sqsubseteq \mathtt{R}$, both preorders	**thin-introduction**: thinning first cannot lose an R-minimum
$\mathtt{thin\ Q} \sqsupseteq \mathtt{min(Q)}\ \frac{\mathbb{1}}{\ni}$ (8.2)	**thin-elimination**: keeping one element is a thinning, but its domain is the sets min(Q) is defined on
$\frac{\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \sqsupseteq \frac{\mathtt{S}}{\ni}\ \mathtt{min(R)}\ \frac{\mathbb{1}}{\ni}$ (8.3), $\mathtt{R} \cap (\mathtt{S}^{\circ}\mathtt{S}) \sqsubseteq \mathtt{Q}$	the usable variant: R need only refine Q between values S gives one argument
$\mathtt{union\ thin\ Q} \sqsupseteq \mathtt{P(thin\ Q)\ union}$ (8.4)	thinning each member set is a thinning of the union
$\llparenthesis \frac{\mathtt{F}(\ni)\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \rrparenthesis \sqsubseteq \frac{\llparenthesis \mathtt{S} \rrparenthesis}{\ni}\ \mathtt{thin\ Q}$ Theorem 8.1, S monotonic on Q	the **thinning theorem**: thinning at every step beats thinning only at the end
$\llparenthesis \frac{\mathtt{F}(\ni)\mathtt{S}}{\ni}\ \mathtt{thin\ Q} \rrparenthesis\ \mathtt{min(R)} \sqsubseteq \frac{\llparenthesis \mathtt{S} \rrparenthesis}{\ni}\ \mathtt{min(R)}$ Corollary 8.1, $\mathtt{Q} \sqsubseteq \mathtt{R}$ as well	the same against the optimisation problem itself, by thin-introduction

§14.2. Paths in a layered network

$\mathtt{F(A,X)} = \mathtt{A} + \mathtt{A} \times \mathtt{X}$, $\mathtt{L} = \mathtt{list}^{+}$ with initial algebra $\alpha \triangleq [\mathtt{wrap},\mathtt{cons}] : \mathtt{F(A,LA)} \to \mathtt{LA}$. (14.2a)

$\mathtt{wrapz} \triangleq \langle \mathtt{wrap},\mathtt{zero} \rangle$, $\mathtt{consw\ (a,(xs,n))} = (\mathtt{cons\ (a,xs)},\mathtt{wt\ (a,head\ xs)} + \mathtt{n})$.

$\mathtt{cost} \triangleq \llparenthesis [\mathtt{wrapz},\mathtt{consw}] \rrparenthesis \pi_2$, $\llparenthesis [\mathtt{wrapz},\mathtt{consw}] \rrparenthesis = \langle \mathbb{1},\mathtt{cost} \rangle$, $\mathtt{R} \triangleq \mathtt{cost} \leq \mathtt{cost}^{\circ}$.

$\mathtt{Q} \triangleq \mathtt{R} \cap (\mathtt{head\ head}^{\circ})$, $\mathtt{S} \triangleq \mathtt{F}(\mathbb{1},\ni)\alpha$, $\frac{\mathtt{F}(\ni,\mathbb{1})}{\ni} = \mathbb{1} + \mathtt{cpl}$, $\frac{\mathtt{F}(\mathbb{1},\ni)}{\ni} = \mathbb{1} + \mathtt{cpr}$, $\mathtt{step} \triangleq \mathtt{cpr\ P(cons)\ min(R)}$.

(14.2b)

the law	what it says
$\frac{\llparenthesis \mathtt{F}(\ni,\mathbb{1})\alpha \rrparenthesis}{\ni}\ \mathtt{min(R)}$ $= \frac{\mathtt{L}(\ni)}{\ni}\ \mathtt{min(R)}$	the specification: a least-cost path across the network
$\mathtt{F}(\ni,\mathtt{Q})\alpha \sqsubseteq \mathtt{F}(\ni,\mathbb{1})\alpha\mathtt{Q}$	S is monotonic on Q; on R it is not, since the next edge can cost arbitrarily much
$\llparenthesis \frac{\mathtt{F}(\ni,\ni)\alpha}{\ni}\ \mathtt{thin\ Q} \rrparenthesis\ \mathtt{min(R)} \sqsubseteq \frac{\llparenthesis \mathtt{F}(\ni,\mathbb{1})\alpha \rrparenthesis}{\ni}\ \mathtt{min(R)}$ Corollary 8.1	thinning applies: one partial path per starting vertex is enough
$\mathtt{S\ head} \sqsubseteq [\mathbb{1},\pi_2]$ $\mathtt{S\ head}$ simple, so $\mathtt{R} \cap (\mathtt{S}^{\circ}\mathtt{S}) \sqsubseteq \mathtt{Q}$	the side condition of (8.3): between two paths S builds from one argument, equal cost and equal head already means Q
$\frac{\mathtt{F}(\ni,\ni)\alpha}{\ni}\ \mathtt{thin\ Q}$ $\sqsupseteq \frac{\mathtt{F}(\ni,\mathbb{1})}{\ni}\ \mathtt{P}(\frac{\mathtt{F}(\mathbb{1},\ni)}{\ni}\ \mathtt{P}(\alpha)\ \mathtt{min(R)})$ (8.4), (8.3), $\mathtt{P}(\frac{\mathbb{1}}{\ni})\ \mathtt{union} = \mathbb{1}$	thin eliminated: split the algebra at $\mathtt{F}(\ni,\mathbb{1})\mathtt{F}(\mathbb{1},\ni)$, thin each part, one minimum per part
$\frac{\mathtt{F}(\mathbb{1},\ni)}{\ni}\ \mathtt{P}(\alpha)\ \mathtt{min(R)} = [\mathtt{wrap},\mathtt{step}]$	that inner part read off the two branches of α
$\frac{\llparenthesis \mathtt{F}(\ni,\mathbb{1})\alpha \rrparenthesis}{\ni}\ \mathtt{min(R)} \sqsupseteq \llparenthesis [\mathtt{P(wrap)},\mathtt{cpl\ P(step)}] \rrparenthesis\ \mathtt{min(R)}$	the program: one fold, a best path per vertex of the current layer

§14.3. Implementing thin

(14.3a)

$\mathtt{setify} : [A]\to EA$, $\mathtt{cup} : EA\times EA\to EA$, $\mathtt{cp}(F)\triangleq \frac{F(\ni)}{\ni}$, $\mathtt{listcp}(F) : F([A])\to[FA]$, $\mathtt{sort}\ P\triangleq\mathtt{setify}^\circ\ \mathtt{ordered}\ P$ for P a connected preorder.

$\mathtt{thinlist}\ Q$ is any $\mathtt{thinlist}\ Q\sqsubseteq\mathtt{subseq}$ with $\mathtt{thinlist}\ Q\ \mathtt{setify}\sqsubseteq\mathtt{setify}\ \mathtt{thin}\ Q$; one is $(\!|[\mathtt{nil},\mathtt{bump}\ Q]|\!)$, $\mathtt{bump}\ Q\ (a,[])=[a]$, $\mathtt{bump}\ Q\ (a,[b]\mathbin{+\!\!+}xs)=(b\ Q\ a\to[a]\mathbin{+\!\!+}xs, a\ Q\ b\to[b]\mathbin{+\!\!+}xs, [a]\mathbin{+\!\!+}[b]\mathbin{+\!\!+}xs)$.

Binary thinning data: $S=(f_1p_1)\cup(f_2p_2)$ with p_1, p_2 coreflexive; Q a preorder with $Q\sqsubseteq R$ and both f_1p_1, f_2p_2 monotonic on Q; P a connected preorder with both f_1, f_2 monotonic on P; $g_i\triangleq\mathtt{list}(f_i)\ \mathtt{filter}(p_i)$.

(14.3b)

the law	what it says
$\mathtt{thinlist}\ Q\ xs=[\mathtt{minlist}\ Q\ xs]$ (8.5), Q connected, xs non-empty	what thinning should come to when it can: one element
$\mathtt{sort}\ P\ \mathtt{thinlist}\ Q\sqsubseteq\mathtt{thin}\ Q\ \mathtt{sort}\ P$ (8.6)	thinning a sorted list is a thinning of the set — this is what $\mathtt{thinlist}\ Q\sqsubseteq\mathtt{subseq}$ buys
$\mathtt{sort}\ P\ \mathtt{minlist}\ Q\sqsubseteq\mathtt{min}(Q)$ (8.7)	a minimum of the sorted list is a minimum of the set
$\mathtt{sort}(fPf^\circ)\ \mathtt{list}(f)\sqsubseteq P(f)\ \mathtt{sort}\ P$ (8.8)	shunt a function through a sort
$\mathtt{sort}\ P\ \mathtt{filter}(p)\sqsubseteq E(p)\ \mathtt{sort}\ P$ (8.9), p coreflexive	filtering a sorted list sorts the restricted set
$(\mathtt{sort}\ P\times\mathtt{sort}\ P)\ \mathtt{merge}\ P\sqsubseteq\mathtt{cup}\ \mathtt{sort}\ P$ (8.10)	merging two sorted lists sorts their union
$F(\mathtt{sort}\ P)\ \mathtt{listcp}(F)\sqsubseteq\mathtt{cp}(F)\ \mathtt{sort}(FP)$ (8.11), F linear	$\mathtt{listcp}(F)$ is the list implementation of the cartesian product $\mathtt{cp}(F)$
$F(\mathtt{sort}\ P)\ \mathtt{listcp}(F)\ \mathtt{list}(f)\ \mathtt{filter}(p)\sqsubseteq\frac{F(\ni)fp}{\ni}\ \mathtt{sort}\ P$ Lemma 8.1, f monotonic on P, p coreflexive	one sorted list built from sorted arguments, instead of a set built and then sorted
$(\!|\mathtt{listcp}(F)\ \langle g_1,g_2\rangle\ \mathtt{merge}\ P\ \mathtt{thinlist}\ Q|\!)\ \mathtt{minlist}\ R$ $\sqsubseteq\frac{(\!|S|\!)}{\ni}\ \mathtt{min}(R)$ Theorem 8.2	the **binary thinning theorem**: a fold on sorted lists of partial solutions, thinned at every step

§14.4. The knapsack problem

(14.4a)

$\mathtt{vol},\mathtt{wt} : \mathtt{Item}\to\mathtt{Real}$, $\mathtt{value}\triangleq\mathtt{list}(\mathtt{vol})\ \mathtt{sum}$, $\mathtt{weight}\triangleq\mathtt{list}(\mathtt{wt})\ \mathtt{sum}$.

$\mathtt{subseq}=(\!|[\mathtt{nil},\mathtt{cons}]\cup[\mathtt{nil},\pi_2]|\!)$, $\mathtt{within}\ w$ the coreflexive on xs with $\mathtt{weight}\ xs\le w$.

$\ge\triangleq\le^\circ$, $R\triangleq\mathtt{value}\ge\mathtt{value}^\circ$, $Q\triangleq R\cap(\mathtt{weight}\le\mathtt{weight}^\circ)$, $P\triangleq R$.

$FA=1+\mathtt{Item}\times A$, $\mathtt{listcp}(F)=\mathtt{wrap}+\mathtt{cpr}$, $g_1\triangleq\mathtt{list}([\mathtt{nil},\mathtt{cons}])\ \mathtt{filter}(\mathtt{within}\ w)=[\mathtt{list}(\mathtt{nil}),h_1]$, $g_2\triangleq\mathtt{list}([\mathtt{nil},\pi_2])=[\mathtt{list}(\mathtt{nil}),h_2]$.

$h_1\triangleq\mathtt{list}(\mathtt{cons})\ \mathtt{filter}(\mathtt{within}\ w)$, $h_2\triangleq\mathtt{list}(\pi_2)$.

(14.4b)

the law	what it says
$\frac{\mathtt{subseq}\ (\mathtt{within}\ w)}{\ni}\ \mathtt{min}(R)$	the specification: a selection of greatest total value that stays within the capacity w
$\mathtt{subseq}\ (\mathtt{within}\ w)=(\!|([\mathtt{nil},\mathtt{cons}]\ (\mathtt{within}\ w))\cup[\mathtt{nil},\pi_2]|\!)$ fusion, weights non-negative	the selections that fit are themselves a fold, and the fold's algebra already has binary thinning's shape
$(\mathbb{1}\times R)\ (\mathtt{cons}\ (\mathtt{within}\ w))\sqsubseteq\mathtt{cons}\ (\mathtt{within}\ w)R$ FALSE	not monotonic on R: a selection of greater value need not still fit once one more item goes in
$(\mathbb{1}\times Q)\ (\mathtt{cons}\ (\mathtt{within}\ w))\sqsubseteq\mathtt{cons}\ (\mathtt{within}\ w)Q$ $(\mathbb{1}\times Q)\pi_2\sqsubseteq\pi_2Q$	both halves are monotonic on Q once ties in value are broken by weight
$(\!|\mathtt{listcp}(F)\ \langle g_1,g_2\rangle\ \mathtt{merge}\ R\ \mathtt{thinlist}\ Q|\!)\ \mathtt{minlist}\ R$ Theorem 8.2 at $P\triangleq R$, F linear	binary thinning, sorting in descending order of value
$\mathtt{knapsack}\ w=(\!|[\mathtt{nil},\mathtt{cpr}\ \langle h_1,h_2\rangle\ \mathtt{merge}\ R\ \mathtt{thinlist}\ Q]|\!)\ \mathtt{minlist}\ R$	the program; $\mathtt{minlist}\ R$ becomes $\mathtt{head}$, since packings come out in descending value

§14.5. The paragraph problem

(14.5a)

`Line=list⁺ Word`, `Para=list⁺ Line`, `FA=Word+Word×A`, `listcp(F)=wrap+cpr`.

`new (a,xs)=[[a]]⧺xs`, `glue (a,xs)=[[a]⧺head xs]⧺tail xs`, `partition≜⦇[wrap wrap,new∪glue]⦈ : list⁺ Word→Para`.

`width≜⦇[length,(length×𝟙) plus succ]⦈`, `fits w` the coreflexive on a line `x` with `width x≤w`, `ok w` the coreflexive on `[x]⧺xs` with `width x≤w`.

`white w x=w−width x`, `collect≜list(sqr) sum`, `waste w≜init list(white w) collect`.

`R≜(waste w)≤(waste w)°`, `Q≜Rn(head head°)`, `P≜⊤`.

g_1`≜list([wrap wrap,new])`, g_2`≜list([wrap wrap,glue]) filter(ok w)`, `start≜wrap wrap wrap`, h_1`≜list(new)`, h_2`≜list(glue) filter(ok w)`.

(14.5b)

the law	what it says
$\frac{\mathtt{partition\ list^{+}(fits\ w)}}{\ni}\ \mathtt{min(R)}$	the specification: least waste among the paragraphs whose every line fits
`partition list⁺(fits w)=⦇[wrap wrap,new∪(glue (ok w))]⦈` fusion, every word fits on a line by itself	the fitting paragraphs are themselves a fold
`[wrap wrap,new∪(glue (ok w))]` `=[wrap wrap,new]∪([wrap wrap,glue] (ok w))`	rewritten into binary thinning's shape $(f_1 p_1)\cup(f_2 p_2)$
`(𝟙×R) glue⊑glue R` FALSE	`glue` is not monotonic on `R`: the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this

`(𝟙×Q) new⊑new Q`
`(𝟙×Q) (glue (ok w))⊑glue (ok w)Q`
`cons` monotonic on `collect≤collect°`
	both halves are monotonic on `Q` once ties in waste are broken by the first line

`merge ⊤=cat`; `P≜head prefix head°` also serves	`⊤` needs no sorting at all, and `prefix` is a linear order on first lines of paragraphs of one input
`⦇listcp(F) ⟨`g_1`,`g_2`⟩ cat thinlist Q⦈ minlist R` Theorem 8.2 at `P≜⊤`	binary thinning, one candidate kept per first line
`paragraph w=⦇[start,cpr ⟨`h_1`,`h_2`⟩ cat thinlist Q]⦈ minlist R`	the program, g_1 and g_2 split along the coproduct

§14.6. Bitonic tours

(14.6a)

`FA=(City×City)+(City×A)`, the base functor of cons-lists of length at least two; `listcp(F)=wrap+cpr`; `tc : City×City→Real`, neither positive nor symmetric.

`start (a,b)=([a,b],[a,b])`, `dropl (a,([b]⧺xs,ys))=([a]⧺xs,[a]⧺ys)`, `dropr (a,(xs,[b]⧺ys))=([a]⧺xs,[a]⧺ys)`, `tour≜⦇[start,dropl∪dropr]⦈`.

`cost (xs,ys)=outcost xs+incost ys`, `outcost` $[a_0,\ldots,a_n]$`=tc` $(a_0,a_1)+\cdots+$`tc` (a_{n-1},a_n), `incost` $[a_0,\ldots,a_n]$`=tc` $(a_1,a_0)+\cdots+$`tc` (a_n,a_{n-1}).

`next≜tail head`, `next2≜next×next`, `R≜cost≤cost°`, `Q≜Rn(next2 next2°)`, `P≜⊤`, g_1`≜list([start,dropl])`, g_2`≜list([start,dropr])`.

the law	what it says
$\frac{\texttt{tour}}{\ni}$ `min(R)`	the specification: a least-cost bitonic tour, outward journey and return kept as a pair of lists
`(𝟙×R) dropl⊑dropl R` FALSE `(𝟙×R) dropr⊑dropr R` FALSE	neither drop is monotonic on `R`: the two edges it adds and removes depend on `head` and `next` of both lists
`(𝟙×Q) dropl⊑dropl Q` `(𝟙×Q) dropr⊑dropr Q`	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input
`⦇listcp(F) ⟨g1,g2⟩ cat thinlist Q⦈ minlist R` Theorem 8.2 at `P≙T`, `merge T=cat`	binary thinning, one candidate per pair of second cities
`mintour=⦇[start wrap,cpr ⟨list(dropl),list(dropr)⟩ cat thinlist Q]⦈ minlist R`	the program: quadratic, because each step adds just two tours to the list kept

(14.6b)

§15. Dynamic Programming

§15.1. Theory

$\mathtt{h : FB{\rightharpoonup}B}$ a map, $\mathtt{T : FA{\rightharpoonup}A}$ an F-algebra, $\mathtt{R : B{\rightharpoonup}B}$. (15.1a)

$\mathtt{H}\triangleq(\!|\mathtt{T}|\!)^\circ(\!|\mathtt{h}|\!)\ \mathtt{: A{\rightharpoonup}B}$, $\mathtt{M}\triangleq\frac{\mathtt{H}}{\ni}\ \mathtt{min(R)}$ the problem to be solved, $(\mu\mathtt{X : G(X)})$ the least fixed point of $\mathtt{G}$.

(15.1b)

the law	what it says
$(\mu\mathtt{X} : \frac{\mathtt{T}^\circ}{\ni}\ \mathtt{P(F(X)h)\ min(R)})\sqsubseteq\mathtt{M}$ Theorem 9.1, $\mathtt{h}$ monotonic on $\mathtt{R}$	**dynamic programming**: decompose in every way, solve each part, keep one optimum per part
$\frac{\mathtt{T}^\circ}{\ni}\ \mathtt{P(F(M)h)\ min(R)}\sqsubseteq\mathtt{M}$ (9.1)	all Knaster–Tarski leaves to prove
$\frac{\mathtt{T}^\circ}{\ni}\ \mathtt{P(F(M)h)\ min(R)}\sqsubseteq\mathtt{H}$ (9.2) $\mathtt{H}^\circ\ \frac{\mathtt{T}^\circ}{\ni}\ \mathtt{P(F(M)h)\ min(R)}\sqsubseteq\mathtt{R}$ (9.3)	(9.1) split by the universal property of `min`
$\mathtt{P(X)\ min(R)}\sqsubseteq(\ni\mathtt{X})\cap(\in\backslash(\mathtt{XR}^\circ))$ (9.4) = (7.10), (13.1b)	the only fact about `min` either proof uses
$(\mu\mathtt{X} : \frac{\mathtt{T}^\circ}{\ni}\ \mathtt{thin\ Q\ P(F(X)h)\ min(R)})\sqsubseteq\mathtt{M}$ Theorem 9.2, $\mathtt{Q}$ a preorder with $\mathtt{QF(H)h}\sqsubseteq\mathtt{F(H)hR}$; `thin Q` as in (14.1b)	the same with a thinning step: decompositions that can never win are dropped
the fixed point is unique, and entire Theorem 6.3, $\mathtt{T}^\circ$ followed by $\mathtt{F}$'s membership relation inductive; $\frac{\mathtt{T}^\circ}{\ni}$ finite and non-empty, $\mathtt{R}$ connected	when the recursion can be refined to a recursive function
$\frac{[\mathtt{V_1,V_2}]^\circ}{\ni}\ \mathtt{thin(Q_1{+}Q_2)\ P([U_1,U_2])\ min(R)}$ $=(\mathtt{Ran\ V_1}\rightarrow\mathtt{W_1,W_2})$ $\mathtt{W_i}\triangleq\frac{\mathtt{V_i}^\circ}{\ni}\ \mathtt{thin(Q_i)\ P(U_i)\ min(R)}$ Proposition 9.1, $\mathtt{V_2V_1}^\circ=\bot$	`FA` is usually a coproduct, and disjoint ranges split the fixed point into one branch per summand
$\mathtt{F(R)h}\sqsubseteq\mathtt{hR}$ $\mathtt{QF(H)h}\sqsubseteq\mathtt{F(H)hR}$	the two conditions to be checked, in the order the three propositions below discharge them
$\mathtt{F(R)h}\sqsubseteq\mathtt{hR}$ Proposition 9.2, $\mathtt{R}\triangleq\mathtt{cost{\le}cost}^\circ$, $\mathtt{h\ cost{=}F(cost)k}$, $\mathtt{F({\le})k}\sqsubseteq\mathtt{k{\le}}$	monotonicity when the cost is itself a fold with a step `k` monotonic on $\le$
$\mathtt{F(R\cap(H^\circ H))h}\sqsubseteq\mathtt{hR}$ Proposition 9.3, $\mathtt{R}\triangleq\mathtt{cost{\le}cost}^\circ$, $\mathtt{h\ cost{=}F(\langle cost,H^\circ\rangle)k}$, $\mathtt{F({\le}{\times}\mathbb{1})k}\sqsubseteq\mathtt{k{\le}}$, $\mathtt{H}^\circ$ simple	monotonicity **in context**: `k` may also read the input the part was built from
$\mathtt{Q}\triangleq\mathtt{F(U,V)}$ Proposition 9.4, $\mathtt{U}$, $\mathtt{V}$ preorders, $\mathtt{F(U,R)h}\sqsubseteq\mathtt{hR}$, $\mathtt{VH}\sqsubseteq\mathtt{HR}$	both conditions of Theorem 9.2 at once, split along the two arguments of a bifunctor

§15.2. The string edit problem

$\mathtt{Op::=cpy\ Char|del\ Char|ins\ Char}$, $\mathtt{F(A,B)=1+(A{\times}B)}$, $\alpha\triangleq\mathtt{[nil,cons]}$. (15.2a)

$\mathtt{edit}\triangleq(\!|\mathtt{[base,step]}|\!)\ \mathtt{: [Op]{\rightharpoonup}[Char]{\times}[Char]}$, `base` returning `([],[])`, `step (cpy a,(xs,ys))=([a]++xs,[a]++ys)`, `step (del a,(xs,ys))=([a]++xs,ys)`, `step (ins a,(xs,ys))=(xs,[a]++ys)`.

$\mathtt{length}\triangleq(\!|\mathtt{[zero,\pi_2\ succ]}|\!)$, $\mathtt{R}\triangleq\mathtt{length{\le}length}^\circ$, $\mathtt{U}\triangleq\top$, $\mathtt{V}\triangleq\mathtt{suffix^\circ{\times}suffix^\circ}$, $\mathtt{Q}\triangleq\mathbb{1}\mathtt{+(U{\times}V)}$, `empty` the coreflexive on `(xs,ys)` with both lists empty.

`unstep` implements $\frac{\mathtt{step}^\circ}{\ni}\ \mathtt{thin(U{\times}V)}$: `unstep ([a]++xs,[])=[(del a,(xs,[]))]`, `unstep ([],[b]++ys)=[(ins b,([],ys))]`, `unstep ([a]++xs,[b]++ys)=(a=b→[(cpy a,(xs,ys))],[(del a,(xs,[b]++ys)),(ins b,([a]++xs,ys))])`.

(15.2b)

the law	what it says
$\frac{\texttt{edit}^\circ}{\ni}$ `min(R)`	the specification: a shortest edit sequence from which both strings can be reconstituted
`F(R)α⊑αR` Proposition 9.2 at `length`, `succ` monotonic on `≤`	`cons` is monotonic on `R`, so Theorem 9.1 already applies
`QF(𝟙,edit°)α⊑F(𝟙,edit°)αR`	the thinning condition sought, over `F(Op,[Char]×[Char])`
`F(⊤,R)α⊑αR` left as an exercise	`U≜⊤` costs nothing: any two operations may be compared
`edit (𝟙×suffix)⊑R° edit` `edit (suffix×𝟙)⊑R° edit`	the other half of Proposition 9.4 for `V≜suffix°×suffix°`

`edit (𝟙×tail)⊑R° edit`
`edit (tail×𝟙)⊑R° edit`
`suffix=tail*`, and `BA⊑CB⟹BA*⊑C*B`
	one step is enough: drop the operation that produced the head, or weaken its `cpy` to a `del` — never lengthening the sequence

`(μX :` $\frac{\texttt{[base,step]}^\circ}{\ni}$ `thin Q P([nil,(𝟙×X) cons]) min(R))⊑` $\frac{\texttt{edit}^\circ}{\ni}$ `min(R)` Theorem 9.2	a copy, when available, beats a delete or an insert
`X=(empty→nil,` $\frac{\texttt{step}^\circ}{\ni}$ `thin(U×V) P((𝟙×X) cons) min(R))` Proposition 9.1	`base` and `step` have disjoint ranges
`mle=(empty→nil,unstep list((𝟙×mle) cons) minlist R)`	the program: at most two decompositions survive, but the same subproblem is solved many times, so the running time is exponential
`column xs ys=[mle (u,ys)|u←tails xs]` `column xs=⦇[fstcol xs,nextcol xs]⦈, fstcol=list(del) tails`	the tabulation: `mle (xs,ys)` needs `mle (u,v)` for every tail `u` of `xs` and `v` of `ys`, so the columns are built right to left

`column xs ([b]⧺ys)=nextcol xs (b,column xs ys)`
`nextcol xs (b,us)`
`=⦇[base (b,last us),step b]⦈ xus`
`xus=zip (xs,zip (init us,tail us))`
	each column is a fold built bottom to top, over `xs` zipped with the adjacent pairs of the column to its right

`base (b,u)=[[ins b]⧺u]`
`step b ((a,(u,v)),ws)=`
`(a=b→[[cpy a]⧺v]⧺ws,`
`[bmin(R) ([del a]⧺w,[ins b]⧺u)]⧺ws)`
`w=head ws`; these `base`, `step` are not `edit`'s
	an entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§15.3. Optimal bracketing

(15.3a)

`tree A::=tip A|bin (tree A,tree A)`, `FX=A+X²`, so `F(R)=𝟙+R²`; `h≜[tip,bin]`, `flatten≜⦇[wrap,cat]⦈ : tree A→list⁺ A`, `H=flatten°`.

`⟨cost,size⟩≜⦇[opt,opb]⦈`, `opt≜⟨zero,st⟩`, `opb ((cx,sx),(cy,sy))=(cb (sx,sy)+cx+cy,sb (sx,sy))`.

`sb` associative, so `size=flatten sz` for a map `sz`; `R≜cost≤cost°`, `g≜[zero,(𝟙×sz)² opb π₁]`, `single` the coreflexive on singleton lists.

`splits≜⟨inits⁺,tails⁺⟩ zip`, an implementation of $\frac{\texttt{cat}^\circ}{\ni}$; `array≜inits list(row)`, `row≜tails list(mct)`, `col≜inits list(mct)`.

`mix≜zip list(bin) minlist R`, `next≜⟨π₁,mix⟩ snoc`, `process≜((tip wrap)×𝟙) loop(next)`.

(15.3b)

the law	what it says
$\frac{\mathtt{flatten}^\circ}{\ni}$ `min(R)`	the specification: a least-cost bracketing of $\mathtt{a}_1\oplus\cdots\oplus\mathtt{a}_n$, the tree whose flattening is the given list
$\mathtt{[tip,bin]\ cost=(\mathbb{1}+\langle cost,flatten\rangle^2)\ g}$ (9.5)	the cost of a node reads only the cost and the flattening of its two subtrees
$\mathtt{(\mathbb{1}+(\leq\times\mathbb{1})^2)\ g\sqsubseteq g\leq}$ (9.6)	`g` is monotonic on `≤` in its two cost arguments
`F(R∩(flatten flatten°))h⊑hR` Proposition 9.3, `H°=flatten` a map	monotonicity in context: only trees with the same flattening are compared
$\mathtt{(\mu X\ :}\ \frac{\mathtt{[wrap,cat]}^\circ}{\ni}\ \mathtt{P([tip,(X\times X)\ bin])\ min(R))\sqsubseteq}\ \frac{\mathtt{flatten}^\circ}{\ni}\ \mathtt{min(R)}$ Theorem 9.1	split the list in every way, bracket both halves, join
$\mathtt{X=(single\rightarrow wrap^\circ\ tip,}\ \frac{\mathtt{cat}^\circ}{\ni}\ \mathtt{P((X\times X)\ bin)\ min(R))}$ Proposition 9.1	`wrap` and `cat` have disjoint ranges
`mct=(single→head tip,splits list((mct×mct) bin) minlist R)`	the program; exponential, since the segments of one list overlap
`mct=(single→head tip,⟨init col,tail row⟩ mix)` (9.7)	the tabulation: `mct xs` is needed for every non-empty segment `xs`, so the values are held as an array of rows
`col=(single→head tip wrap,⟨init col,tail row⟩ next)` (9.8) `cons col=(𝟙×array) process` (9.9) `row=(single→head tip wrap,⟨mct,tail row⟩ cons)` (9.10)	a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop
`array=⦇[fstcol,addcol]⦈, fstcol≜tip wrap wrap` $\mathtt{addcol\triangleq\langle\pi_1\ tip\ wrap,step\rangle\ cons}$ $\mathtt{step\triangleq\langle process\ tail,\pi_2\rangle\ zip\ list(cons)}$	the program: one fold building the array column by column, cubic in the length of the input

§15.4. Data compression

(15.4a)

$\mathtt{[A]::=nil|snoc\ \ ([A],A),\ \ list^+\ \ A::=wrap\ \ A|snoc\ \ (list^+\ \ A,A),\ \ String=[Char],\ \ Code::=sym\ \ Char|ptr\ (String,String^+),\ \ F(Code,String)=\mathbb{1}+(String\times Code),\ \ \alpha\triangleq[nil,snoc].}$

`decode≜⦇[nil,extend]⦈ : [Code]→String`, `extend (xs,sym a)=xs⧺[a]`, `extend (xs,ptr (ys,zs))=xs⧺zs` when `ys⧺zs` is a proper prefix of `xs⧺zs`; `H=decode°`.

$\mathtt{size\triangleq(\![zero,distr\ \ [\mathbb{1}\times c,\mathbb{1}\times p]\ \ plus]\!)}$ with `c`, `p` the constant costs of a symbol and a pointer; `R≜size≤size°`, $\mathtt{Q\triangleq F(\top+\top,prefix^\circ)=\mathbb{1}+(prefix^\circ\times(\top+\top))}$, the two `⊤` on symbols and on pointers.

`lrt ws=min(prefix°×(⊤+⊤)) {(xs,(ys,zs))|xs⧺zs=ws, ys⧺zs` a proper prefix of `ws}`, the longest repeated tail; `reduce (ws⧺[a])=(zs≠[]→[(ws,sym a),(xs,ptr (ys,zs))],[(ws,sym a)])` with `(xs,(ys,zs))=lrt (ws⧺[a])`.

(15.4b)

the law	**what it says**
$\frac{\mathtt{decode}^\circ}{\ni}$ `min(R)`	the specification: a smallest code sequence decoding to the given string
`F(R)α⊑αR` `(⊤+⊤)[c,p]=[c,p]`	monotonicity is routine, the two costs being constants

`F(⊤+⊤,R)α⊑αR`
`decode prefix⊑R° decode`
Proposition 9.4 at `Q≙F(⊤+⊤,prefix°)`
	the thinning condition split in two: pointers are ordered only by how much input they consume, symbols and pointers not at all

`decode init⊑R° decode`	enough for the second: drop the last character of the output by shortening or removing the last code element, never raising the cost
`(μX :` $\frac{\mathtt{[nil,extend]}^\circ}{\ni}$ `thin Q P([nil,(X×𝟙) snoc]) min(R))⊑` $\frac{\mathtt{decode}^\circ}{\ni}$ `min(R)` Theorem 9.2	between a symbol and a pointer nothing can be decided in advance; between two pointers the longer match wins
`X=(null→nil,` $\frac{\mathtt{extend}^\circ}{\ni}$ `thin(prefix°×(⊤+⊤)) P((X×𝟙) snoc) min(R))` Proposition 9.1	`nil` and `extend` have disjoint ranges
$\frac{\mathtt{extend}^\circ}{\ni}$ `(ws⧺[a])={(ws,sym a)}∪` `{(xs,ptr (ys,zs))|xs⧺zs=ws⧺[a]`, `ys⧺zs` a proper prefix of `ws}`	the decompositions of one string: take the last character as a symbol, or end with a pointer
`reduce` implements $\frac{\mathtt{extend}^\circ}{\ni}$ `thin(prefix°×(⊤+⊤))`	thinning leaves at most two: the symbol, and the pointer of `lrt`
`encode=(null→nil,reduce list((encode×𝟙) snoc) minlist R)`	the program, again exponential; the book gives no tabulation for it

§16. Greedy Algorithms

§16.1. Theory

h, T, R, H, M as in (15.1a); additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\ni}$ returns, so that $\frac{T^\circ}{\ni}$ min(Q) is entire. (16.1a)

(16.1b)

the law	what it says
(μX : $\frac{T^\circ}{\ni}$ min(Q) F(X)h)⊑M Theorem 10.1, the hypotheses of Theorem 9.2	**greedy**: one decomposition is kept at every step, so P and $\frac{}{\ni}$ disappear from the recursion
$\frac{[V_1,V_2]^\circ}{\ni}$ min(Q_1+Q_2) [U_1,U_2] =(Ran V_1→W_1,W_2) W_i≙ $\frac{V_i^\circ}{\ni}$ min(Q_i) U_i Proposition 10.1, $V_2V_1^\circ$=⊥	Proposition 9.1 with min for thin
Q≙F(U,V) Proposition 9.4	still the way to get both conditions, but $\frac{T^\circ}{\ni}$ min(Q) must now be entire as well
Theorem 7.2, (13.3e)	that greedy theorem chooses among the **results** of one relational step of a reduce; Theorem 10.1 chooses among the **decompositions** of the input, for an arbitrary T rather than an initial algebra, at the cost of h being a map

§16.2. The detab-entab problem

detab≙⦇[nil,expand]⦈ : String→String over snoc-lists, α≙[nil,snoc], H=detab°; expand (xs,a)=(a=TB→fill xs,xs⧺[a]), fill xs=xs⧺blanks (n−(col xs) mod n). (16.2a)

col≙⦇[zero,count]⦈, count (c,a)=(a=NL→0,c+1); TB the tab, BL the blank, NL the newline, tab stops every n columns.

R≙length≤length°, U the preorder with a U b⟺a=TB∨a=b, V≙prefix°∩(fill fill°), Q≙𝟙+(V×U).

unfill xs the shortest prefix of xs with fill (unfill xs)=fill xs; tbc the trailing blank count, triple≙⟨unfill entab,⟨tbc,col⟩⟩.

(16.2b)

the law	what it says
$\frac{\mathtt{detab}^\circ}{\ni}$ `min(R)`	the specification: a shortest input `detab` expands to the given output — `detab entab=𝟙` and nothing shorter does
`F(⊤,R)α⊑αR` left as an exercise	`U` may be any preorder on characters; `a U b⟺a=TB∨a=b` puts `TB` below every character, so `min` prefers a tab to a blank
`detab prefix⊑R° detab` FALSE	at `n=8`, `detab [a,b,c,d,e,TB]=[a,b,c,d,e,BL,BL,BL]`, whose prefix `[a,b,c,d,e,BL,BL]` is longer than any input giving it
`detab V°⊑R° detab` `V≙prefix°∩(fill fill°)`	true once only the prefixes that do not cross a tab stop are allowed
`nil V°=nil` `fill V°=fill` `snoc V°⊑snoc∪(`π_1`V°)` left as exercises	the three properties of `V` the derivation rests on; the second is what `fill fill°` was added for
`expand V°⊑expand∪(`π_1`V°)`	the claim they add up to: shortening the output either leaves the last step alone or discards it
`detab V°⊑detab∪(init detab V°)` `init` inductive, so the greatest solution is the unique one	hence `detab V°⊑prefix detab`, and `prefix⊑R°` finishes Proposition 9.4
`(μX :` $\frac{\mathtt{[nil,expand]}^\circ}{\ni}$ `min(Q) (𝟙+(X×𝟙)) [nil,snoc])⊑` $\frac{\mathtt{detab}^\circ}{\ni}$ `min(R)` Theorem 10.1	greedy: one character of input is decided at each step
`X=(null→nil,` $\frac{\mathtt{expand}^\circ}{\ni}$ `min(V×U) (X×𝟙) snoc)` Proposition 10.1	`nil` and `expand` have disjoint ranges
$\frac{\mathtt{expand}^\circ}{\ni}$ `(xs#[a])={(ys,TB)|fill ys=xs#[a]}∪{(xs,a)}` the first set is non-empty iff `a=BL` and `col (xs#[a]) mod n=0`	the last output character came from a tab only if it is a blank landing exactly on a tab stop
$\frac{\mathtt{expand}^\circ}{\ni}$ `min(V×U) (xs#[a])=` `(a=BL∧col (xs#[a]) mod n=0→(unfill xs,TB),(xs,a))`	the greedy step: emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop
`entab xs=entab (unfill xs)#blanks (tbc xs)` (10.1)	what makes `triple` a snoc-list reduce: the output splits at the last tab stop
`triple=⦇[base,op]⦈` and `entab=triple assocl` π_1 `(𝟙×blanks) cat`	the program: one pass carrying the column and the count of pending blanks

`base` returns `([],(0,0))`
`op ((xs,(t,c)),a)=`
`(a=BL∧(c+1) mod n≠0→(xs,(t+1,c+1)),`
`a=BL→(xs#[TB],(0,c+1)),`
`a=NL→(xs#blanks t#[NL],(0,0)),`
`(xs#blanks t#[a],(0,c+1)))`
	hold a blank back, cash the held blanks in for a tab at a tab stop, flush them at a newline, flush them before anything else

§16.3. The minimum tardiness problem

(16.3a)

> `FX=1+(X×Job)`, `α≙[nil,snoc]` on schedules, `β≙[nil,snag]` on bags, `snag` putting a job into a bag; `bagify≙⦇β⦈ : [Job]→Bag Job`, `H=bagify°`.
>
> `ct`, `dt`, `wt : Job→Real` the completion, due and weighting quantities of a job; `penalty (xs,j)=(sum (list(ct) xs)+ct j−dt j)×wt j`.
>
> `cost≙` $\frac{\mathtt{prefix}}{\ni}$ `P(α° [zero,penalty]) max(≤)`, `cost []=0`, `cost (xs#[j])=bmax (cost xs,penalty (xs,j))`, `R≙cost≤cost°`.
>
> `perm≙bagify bagify°=⦇[nil,add]⦈`, `add (xs,j)=ys#[j]#zs` for some `xs=ys#zs`.
>
> `k≙[zero,assocr (𝟙×((bagify°×𝟙) penalty)) bmax]`, `f≙[zero,(bagify°×𝟙) penalty]`, `Q≙f≤f°`.

(16.3b)

the law	what it says
$\frac{\mathtt{bagify}^\circ}{\ni}\ \mathtt{min}(R)$	the specification: an ordering of the given bag of jobs with least maximum penalty
$\mathtt{F}(R\cap(\mathtt{bagify}\ \mathtt{bagify}^\circ))\alpha\sqsubseteq\alpha R$ (10.2)	monotonicity in context — only schedules of one bag are compared
$(Q\cap(\beta\beta^\circ))\mathtt{F}(\mathtt{bagify}^\circ)\alpha\sqsubseteq\mathtt{F}(\mathtt{bagify}^\circ)\alpha R$ (10.3)	the greedy condition, also in context
$\mathtt{cost}\ (xs{+\!\!+}[j])=\mathtt{bmax}\ (\mathtt{cost}\ xs,\mathtt{penalty}\ (\mathtt{perm}\ xs,j))$ $\alpha\ \mathtt{cost}=\mathtt{F}(\langle\mathtt{cost},\mathtt{bagify}\rangle)k,\ \ \mathtt{F}(\leq\times\mathbb{1})k\sqsubseteq k\leq$ Proposition 9.3	(10.2): `penalty` sums the completion times of `xs`, and a sum does not see the order
$\alpha\ \mathtt{cost}=\langle g,h\rangle\ \mathtt{bmax}$ (10.4) $g\triangleq[\mathtt{zero},\mathtt{penalty}]$ (10.5) $h\triangleq[\mathtt{zero},\pi_1\ \mathtt{cost}]$ (10.6)	the cost of a schedule split into the penalty of its last job and the cost of the rest
$\mathtt{add}\sqsubseteq\pi_1 R$ (10.7) $\beta\ \mathtt{bagify}^\circ=\mathtt{F}(\mathtt{bagify}^\circ)\ [\mathtt{nil},\mathtt{add}]$ (10.8)	adding a job never lowers the cost; and $\mathtt{bagify}^\circ$ is itself a reduce on bags. Together: $\beta\ \mathtt{bagify}^\circ\sqsubseteq\mathtt{F}(\mathtt{bagify}^\circ)h\leq\mathtt{cost}^\circ$
$\mathtt{F}(\mathtt{bagify})Q^\circ\mathtt{F}(\mathtt{bagify}^\circ)=g\geq g^\circ$, met by $Q\triangleq f\leq f^\circ,\ f\triangleq[\mathtt{zero},(\mathtt{bagify}^\circ\times\mathbb{1})\ \mathtt{penalty}]$	the specification Q has to satisfy, and the choice that meets it: a Q-minimum is a job of least penalty
no greedy **reduce** exists	a greedy snoc-list reduce would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole
$X=(\mathtt{null}\to\mathtt{nil},\ \frac{\mathtt{snag}^\circ}{\ni}\ \mathtt{min}(Q)\ \ (X\times\mathbb{1})\ \ \mathtt{snoc})$ Theorem 10.1, Proposition 10.1	`nil` and `snag` have disjoint ranges
$\mathtt{schedule}=(\mathtt{null}\to\mathtt{nil},\mathtt{pick}\ (\mathtt{schedule}\times\mathbb{1})\ \mathtt{snoc})$ $\mathtt{pick}\sqsubseteq\frac{\mathtt{snag}^\circ}{\ni}\ \mathtt{min}(Q)$, a partial function	the program: repeatedly remove a job of least penalty and put it last; quadratic in the number of jobs

§16.4. The TeX problem

(16.4a)

$\mathtt{intern}\triangleq\mathtt{val}\ \mathtt{round}\ :\ \mathtt{Decimal}\to[0,2^{16})$, $\mathtt{val}\triangleq(\![\mathtt{zero},\mathtt{shift}]\!)$, $\mathtt{shift}\ (d,r)=(d+r)/10$, $\mathtt{round}\ r$ rounds $2^{16}r$ to the nearest integer: $\mathtt{round}\ r=n\Longleftrightarrow 2n-1<2^{17}r<2n+1$.

$\mathtt{interval}\ n=((2n-1)/2^{17},(2n+1)/2^{17})$, $r\ \mathtt{inrange}\ (a,b)\Longleftrightarrow a<r<b$, $\mathtt{round}^\circ=\mathtt{interval}\ \mathtt{inrange}$, $R\triangleq\mathtt{length}\leq\mathtt{length}^\circ$.

$\mathtt{Interval}$ the pairs (a,b) with $0<b<1$ and $a<b$ (10.9) $[\mathtt{arb},\mathtt{step}]\ :\ 1+(\mathtt{Digit}\times\mathtt{Interval})\to\mathtt{Interval}$, $\mathtt{step}\ (d,(a,b))=((d+a)/10,(d+b)/10)$.

$\mathtt{F}X=1+(\mathtt{Digit}\times X)$, $\alpha\triangleq[\mathtt{nil},\mathtt{cons}]$, $h\triangleq(\![\mathtt{arb},\mathtt{step}]\!)=H^\circ$, $!\ :\ \mathtt{Digit}\times\mathtt{Interval}\to 1$, $Q\triangleq(l^\circ\,!^\circ r)\cup\mathbb{1}$, $w\triangleq 2^{17}$.

(16.4b)

the law	**what it says**
$\frac{\mathtt{intern}^\circ}{\ni}\ \mathtt{min(R)=interval}\ \frac{\mathtt{inrange\ val}^\circ}{\ni}\ \mathtt{min(R)}$	the specification: a shortest decimal whose internal representation is the given multiple of 2^{-16}. `round°` is not a map, but `interval` is, so it comes out of the $\frac{}{\ni}$
`val inrange°=⦇[arb,step]⦈` fusion; `zero inrange°=arb`, `shift inrange°=(𝟙×inrange°) step`	the converse of `val`, cut down to intervals, is a reduce on cons-lists
`F(R)α⊑αR`	`cons` is monotonic on `R` — routine
`α°F(h)Q°⊑R°α°F(h)`	the greedy condition of Theorem 10.1, converted
$\frac{\mathtt{arb}^\circ}{\ni}$ `(a,b)=(a<0→{*},{})` $\frac{\mathtt{step}^\circ}{\ni}$ `(a,b)={(d,(10a−d,10b−d))|0<10b−d<1}`	the decompositions of an interval: stop, or take one more digit
$\mathtt{0<10b-d_1<1}$ and $\mathtt{0<10b-d_2<1}$ imply $\mathtt{d_1=d_2}$	`step°` is in fact a map, `d` the digit with `0<10b−d<1`, so $\frac{\mathtt{[arb,step]}^\circ}{\ni}$ returns at most two elements and `Q` need only choose between them
`Q≙(l°!°r)∪𝟙`, and `! nil⊑cons R°` from `! nil length⊑cons length≥`	stop whenever stopping is legal: the empty decimal is shorter than any other
$\mathtt{(\mu X\ :}\ \frac{\mathtt{[arb,step]}^\circ}{\ni}\ \mathtt{min(Q)\ F(X)\alpha)\sqsubseteq}\ \frac{\mathtt{intern}^\circ}{\ni}\ \mathtt{min(R)}$ Theorem 10.1	greedy: emit the one digit the interval allows, until the interval contains zero
`extern=interval f` `f(a,b)=(a<0→[],[d]⧺f(10a−d,10b−d))`	the program, with `d` the digit above

`extern n=f(2n−1,2n+1)`
 `f (p,q)=(p≤0→[],[d]⧺f (10p−w d,10q−w d))`
 `d=(10q) div w`, $\mathtt{w=2^{17}}$
	the same in integer arithmetic only, as chapter 3 required of `intern`: every interval reached is `(p/w,q/w)`

§17. Appendix

§17.1. P(S) min R=(∋S)∩(∈\(SR°))

(p,q) tabulates W≜(∋S)∩(∈\(SR°)), and y≜ $\frac{(p∋S)∩(qR)}{∋}$, a map. (17.1a)

(17.1b)

$$\frac{W}{P(S)\ \min\ R} \xLeftarrow[p^\circ q=W,\ 𝟙⊑yy^\circ]{} \frac{p^\circ y}{P(S)} \quad \frac{y^\circ q}{\min\ R}$$

(17.1c)

$$\frac{y^\circ q}{∋} \xLeftarrow[f^\circ\cdot⊣f\cdot,\ \cdot∋⊣_{\overline{∋}}]{} \frac{q}{(p∋S)∩(qR)} \xLeftarrow[Δ⊣∩,\ f^\circ\cdot⊣f\cdot]{} \frac{p^\circ q}{∋S} \quad \frac{𝟙}{R}$$

R reflexive

$$\frac{∈y^\circ q}{R^\circ} \xLeftarrow[\cdot∋⊣_{\overline{∋}},\ ^\circ,\ \text{meet}]{} \frac{R^\circ q^\circ q}{R^\circ}$$

q simple

(17.1d)

$$\frac{p^\circ y}{P(S)} \xLeftarrow[Δ⊣∩,\ \cdot T⊣/T,\ ^\circ,\ f^\circ\cdot⊣f\cdot]{} \frac{p^\circ y∋}{∋S} \quad \frac{p∋}{y∋S^\circ}$$

$$\frac{p^\circ y∋}{∋S} \xLeftarrow[\cdot∋⊣_{\overline{∋}},\ \text{meet}]{} \frac{p^\circ p∋S}{∋S}$$

p simple

$$\frac{p∋}{y∋S^\circ} \xLeftarrow[\text{modular law}]{} \frac{p∋}{(p∋)∩(qRS^\circ)} \xLeftarrow[𝟙⊑qq^\circ]{} \frac{q^\circ p∋}{RS^\circ} \xLeftarrow[^\circ,\ T\cdot⊣T\backslash]{} \frac{W}{∈\backslash(SR^\circ)}$$

W's right half

§17.2. P(min R) min R=P(Dom (min R)) union min R

(17.2a)

$$\boxed{P(\min\ R)\ \min\ R} \ \overset{}{=\!=}_{\text{Dom (min R) min R=min R}} \ \boxed{P(\text{Dom (min R) min R) min R}}$$

$$=\!=_{\text{P a relator}} \ \boxed{P(\text{Dom (min R))P(min R) min R}} \ \sqsubseteq_{(13.1.9a)} \ \boxed{P(\text{Dom (min R)) union min R}}$$

(17.2b)

$$\frac{P(\text{Dom (min R)) union min R}}{P(\text{min R) min R}} \xLeftarrow[(17.1b)\text{ at } S:=\min\ R,\ Δ⊣∩,\ T\cdot⊣T\backslash]{} \frac{P(\text{Dom (min R)) union min R}}{∋\text{min R}} \quad \frac{∈P(\text{Dom (min R)) union min R}}{\text{min RR}^\circ}$$

$$\frac{P(\text{Dom (min R)) union min R}}{∋\text{min R}} \xLeftarrow[P(\text{Dom (min R)})⊑𝟙]{} \frac{\text{union min R}}{∋\text{min R}}$$

$$\xLeftarrow[T(U∩V)⊑TU∩TV,\ \cdot∋⊣_{\overline{∋}},\ (13.1.4a)]{} \frac{(∋∋)∩(∈\backslash(∈\backslash R^\circ))}{∋\text{min R}} \xLeftarrow[\text{modular law}]{} \frac{∋(∋∩(∈(∈\backslash(∈\backslash R^\circ))))}{∋(∋∩(∈\backslash R^\circ))}$$

counit of $T\cdot⊣T\backslash$

$$\frac{∈P(\text{Dom (min R)) union min R}}{\text{min RR}^\circ} \xLeftarrow[∈\text{ lax natural}]{} \frac{\text{Dom (min R)}∈\text{union min R}}{\text{min RR}^\circ}$$

⟸ Dom (min R)⊑min R (min R)°

min R (min R)°∈union min R
min RR°

⟸ (min R)°⊑∈, ∈∈union⊑∈

min R∈min R
min RR°

UP of min